Stochastic Simulation for Bayesian Modeling in Psychology

An Introduction to Computational Methods

Maarten Marsman and Lourens Waldorp

Development version, 30 October 2026

Table of contents

Welcome **4**
- Why we wrote this book . . . 4
- Who this book is for . . . 4
- How this book is organized . . . 5
- How to read this book . . . 5
- Software . . . 6
- Acknowledgements . . . 6
- License . . . 6
- About this website . . . 6

I Foundations 7

1 Probability and Simulation **8**
- 1.1 Random variables, densities, and expectations . . . 9
- 1.2 Pseudo-random numbers in R . . . 11
- 1.3 Monte Carlo integration . . . 12
- 1.4 Generating draws from a distribution . . . 16
- 1.5 Simulating a Bayesian model forward . . . 21
- 1.6 When independent draws are not enough . . . 24
- 1.7 Chapter summary . . . 25
- 1.8 Quiz answers . . . 26
- 1.9 Exercises . . . 26

2 Bayesian Inference, Revisited **29**
- 2.1 Prior, likelihood, posterior . . . 29
- 2.2 Conjugate families and kernel recognition . . . 32
- 2.3 Summarizing a posterior . . . 34
- 2.4 Comparing models: marginal likelihoods and Bayes factors . . . 37
- 2.5 Grid approximation and its limits . . . 42
- 2.6 Sampling instead of gridding . . . 48
- 2.7 Chapter summary . . . 48
- 2.8 Quiz answers . . . 48
- 2.9 Exercises . . . 49

II Markov Chains 52

3 Markov Chains **53**
- 3.1 Chains on a finite state space . . . 53
- 3.2 Stationary distributions . . . 57
- 3.3 Irreducibility, aperiodicity, and convergence . . . 60
- 3.4 The ergodic theorem . . . 63
- 3.5 Detailed balance . . . 65

3.6 From transition matrices to transition kernels . . . 67
3.7 A first hand-built chain . . . 69
3.8 Chapter summary . . . 73
3.9 Quiz answers . . . 73
3.10 Exercises . . . 74

4 Diagnosing Markov Chains **77**
4.1 Inspecting a run . . . 78
4.2 Dependence and the accuracy of an average . . . 82
4.3 Effective sample size and Monte Carlo standard error . . . 83
4.4 Comparing several chains . . . 86
4.5 A worked analysis: Dutch-language radio . . . 89
4.6 What a reassuring run can miss . . . 94
4.7 Chapter summary . . . 96
4.8 Quiz answers . . . 96
4.9 Exercises . . . 96

III Sampling Algorithms **101**

5 The Metropolis Algorithm **102**
5.1 The idea: propose, compare, accept or reject . . . 102
5.2 A first Metropolis sampler in R . . . 104
5.3 Why it works . . . 108
5.4 Step size and efficiency . . . 109
5.5 Targets beyond the Normal . . . 113
5.6 Common mistakes . . . 115
5.7 Metropolis in more than one dimension . . . 115
5.8 Chapter summary . . . 118
5.9 Quiz answers . . . 119
5.10 Exercises . . . 119

References **122**

News **123**
Development version . . . 123

Appendices **124**

A Software Setup **124**
A.1 R and RStudio . . . 124
A.2 R packages . . . 124
A.3 Installing Stan . . . 125
A.4 Checking your installation . . . 125
A.5 If rstan will not compile . . . 126
A.6 Reproducibility: seeds and sessions . . . 127

B Notation **129**
B.1 Conventions . . . 129
B.2 Data, parameters, and densities . . . 129
B.3 Markov chains and samplers . . . 130
B.4 Model uncertainty . . . 131
B.5 Hamiltonian Monte Carlo . . . 131
B.6 Distributions . . . 131

C Technical Notes on MCMC **133**
C.1 The variance of a dependent average . . . 133
C.2 Calculating basic and split R-hat . . . 133
C.3 Rank normalization and folding . . . 134
C.4 Deriving the Metropolis acceptance probability . . . 136

D Deriving Conjugate Posteriors **138**
D.1 What we need to recognize . . . 138
D.2 A Binomial likelihood and a Beta prior . . . 138
D.3 A Poisson likelihood and a Gamma prior . . . 139
D.4 A Normal likelihood and a Normal prior . . . 140

Welcome

> **Work in progress.** This website holds the chapters of the book that are ready for readers. Online now: Chapter 1 (Probability and Simulation), Chapter 2 (Bayesian Inference, Revisited), Chapter 3 (Markov Chains), Chapter 4 (Diagnosing Markov Chains) and Chapter 5 (The Metropolis Algorithm). Further chapters are added as they are completed, and the chapters already online may still change.

Stochastic simulation provides a way to study Bayesian models when the quantities of interest are difficult to calculate directly. By generating draws from a probability distribution, we can estimate posterior means, probabilities, and predictions. This book develops the methods behind those calculations and shows how to implement them in R.

Why we wrote this book

We previously taught two separate courses in the Research Master's Psychology program at the University of Amsterdam. One used *Bayes Rules!* (Johnson et al., 2022), and the other used *MCMC from Scratch* (Hanada & Matsuura, 2022). Both are excellent books, but they differ in the topics they cover, the depth of their treatment, and the technical background they expect. *Bayes Rules!* introduces Bayesian modeling through applications, while *MCMC from Scratch* develops the principles and implementation of Monte Carlo methods.

Bringing the two courses together required a text that connected introductory Bayesian modeling with a more detailed study of simulation algorithms. We could not find a book that matched that combination of topics and level. We wrote this book to fill the gap, working through the explanations, mathematical steps, and code needed to understand and implement the methods.

Who this book is for

This book is intended for research master's students in psychology who want to understand how stochastic simulation supports Bayesian analysis. You should be familiar with elementary probability and statistics, including means, variances, and probability distributions, and be comfortable writing basic R code using vectors, functions, and loops. Familiarity with Bayesian inference is helpful; the opening chapters introduce the probability and Bayesian concepts used throughout the book.

By working through the text and code, you will learn to

- explain how Monte Carlo simulation approximates posterior quantities and how its numerical accuracy is assessed;
- analyze Markov-chain transition rules and justify whether they preserve a specified target distribution;
- implement and compare Metropolis, Metropolis–Hastings, and Gibbs sampling algorithms in R;
- evaluate simulation output using diagnostics and Monte Carlo error estimates, and justify changes to the sampling procedure;

- apply Bayesian variable selection and model averaging, and evaluate how prior choices affect posterior conclusions and predictions; and
- implement, tune, and evaluate Hamiltonian Monte Carlo, and explain its relationship to Stan's sampling methods in the optional final chapter.

How this book is organized

The book has five parts. Each builds on the concepts and methods introduced earlier.

Foundations. Chapter 1 introduces Monte Carlo integration, explains its accuracy, and develops methods for generating random draws. Chapter 2 connects these ideas to Bayesian inference, including posterior summaries, marginal likelihoods, and Bayes factors. These chapters also explain why obtaining useful draws and evaluating integrals become more demanding as models grow.

Markov Chains. Chapter 3 develops the properties of Markov chains and explains how their transitions can be constructed to preserve a chosen stationary distribution. Chapter 4 uses simulations from Stan to introduce practical checks of chain behavior and estimates of Monte Carlo accuracy.

Sampling Algorithms. Chapter 5 develops the Metropolis algorithm using ratios of posterior densities. Chapter 6 (Metropolis–Hastings and Adaptive Proposals) extends it to asymmetric proposals, parameter transformations, and adaptive tuning. Chapter 7 (Gibbs Sampling and Metropolis-within-Gibbs) introduces updates from conditional distributions and combines them with Metropolis steps when direct conditional sampling is unavailable. The code makes the connection between each algorithm's mathematical definition and its implementation explicit.

Model Uncertainty. Chapter 8 (Bayesian Variable Selection) treats the inclusion of predictors as an unknown part of a regression model. Chapter 9 (Bayesian Model Averaging) uses posterior model probabilities to combine estimates and predictions across models, accounting for uncertainty about model choice.

Going Further. Chapter 10 (Hamiltonian Monte Carlo) introduces Hamiltonian Monte Carlo, the family of methods underlying Stan's sampling algorithms. This chapter is optional; the earlier chapters can be studied independently of it.

How to read this book

Each chapter introduces a problem and its learning goals, develops the methods through worked examples, and closes with a summary and exercises. Read the explanations alongside the code, run the examples, and examine how the results change when you alter the inputs or simulation settings.

The exercises take three forms. *Conceptual* exercises ask you to explain or derive an idea. *Practical* exercises develop your ability to implement and assess a method. *Applied* exercises use the methods to answer questions about data.

Definitions and algorithms

Gray boxes collect definitions, algorithms, and short summaries for reference while you work through the examples and exercises.

Quiz yourself!

Quiz boxes offer a question to consider before continuing. Try to formulate an answer, then compare it with the answers collected near the end of the chapter.

Software

The examples use R. The `tidyverse` supports data manipulation and plotting, `rstan` provides an interface to Stan, and `bayesplot` and `posterior` provide tools for examining simulation output. Appendix A gives installation instructions and help with common setup problems.

Acknowledgements

We acknowledge the influence of *Bayes Rules!* by Alicia Johnson, Miles Ott, and Mine Dogucu, and *MCMC from Scratch* by Masanori Hanada and So Matsuura. Their approaches to teaching Bayesian modeling and simulation helped shape this book, and we highly recommend both texts for further study (Hanada & Matsuura, 2022; Johnson et al., 2022).

This book was co-written with Anthropic's Claude Fable and OpenAI's ChatGPT models Sol and Astra, which helped develop and refine the explanations, examples, and code.

License

This book is licensed under CC BY 4.0.

About this website

The source of the chapters on this website is kept on GitHub, where every change to the text and code is recorded. If you find an error or have a suggestion, please open an issue there.

Part I

Foundations

Chapter 1

Probability and Simulation

Suppose you run a stop-signal task. A participant sees an arrow and presses a key as quickly as possible. On some trials, a tone sounds shortly after the arrow, telling the participant to withhold the response. One account of this task treats it as a race between a *go* process and a *stop* process (Logan & Cowan, 1984). The go process starts when the arrow appears; the stop process starts when the tone sounds. The participant stops successfully if the stop process finishes first.

If you specify distributions for the durations of these processes, how can you calculate the probability of stopping? One approach is to simulate many races and calculate the proportion in which the stop process wins. This proportion is a **Monte Carlo estimate**. It lets you study a model's predictions even when calculating them directly is difficult.

This chapter develops that approach. We begin with probability distributions and expectations, use simulation to approximate them, and examine the accuracy of the resulting estimates. We then study how to generate draws from a distribution and why some sampling methods become inefficient. For further background on probability, simulation, and Bayesian inference, see *Bayes Rules!* (Johnson et al., 2022, Chapters 2, 3 and 8).

The code assumes that you can assign objects, call functions, and work with vectors in R. Run the following setup before working through the examples. Every chunk that generates random numbers starts with `set.seed(84735)`, so you can run any chunk on its own and obtain the numbers printed in the book.

```
library(tidyverse)
library(patchwork)
```

We use N for the number of Monte Carlo draws and n for the number of observed data points. The distinction matters because increasing N makes a computation more precise, whereas increasing n adds information about the phenomenon being studied.

Goals

- Describe random variables, probability masses, densities, and expectations using sums and integrals.
- Approximate integrals with random draws and understand why the Law of Large Numbers makes this work.
- Quantify the error of a Monte Carlo estimate and see it shrink at rate $1/\sqrt{N}$.
- Generate draws from a distribution using inverse-transform and rejection sampling.
- Simulate a Bayesian model *forward* and recover the posterior by conditioning on the observed data.
- Recognize the situations in which independent Monte Carlo is not enough, which motivates the rest of the book.

1.1 Random variables, densities, and expectations

A **random variable** X is a quantity whose value is uncertain, and a probability distribution says how that uncertainty is spread over the possible values. When X is discrete, a probability mass function $f(x) = P(X = x)$ assigns a probability to each value. When X is continuous, a probability density function $f(x)$ assigns a density to each value, and probabilities are areas under it: $P(a < X < b) = \int_a^b f(x)\,dx$. In both cases f is non-negative and the total probability is one. We write densities and mass functions as $f(\cdot)$, with the argument indicating the variable concerned. Uppercase X denotes a random variable; lowercase x denotes a possible value. For a continuous variable, the density at a point is not a probability. Probabilities come from areas over intervals, and a density can exceed one.

Our running example concerns Nederlandse Spoorwegen (NS), the Dutch national railway operator. Sara commutes between Amsterdam and Utrecht and wants to learn the probability π that a train on her route arrives on time. For this example, she defines "on time" as no more than three minutes late and treats the journeys as independent, with the same on-time probability.

Before collecting data, Sara represents her uncertainty about π with a **prior distribution**, $\pi \sim \text{Beta}(7, 5)$. The symbol $\sim$ means "has distribution". A Beta distribution is defined on $[0, 1]$, making it useful for an unknown probability. Its two positive shape parameters determine its shape; its mean is $a/(a + b)$ for a $\text{Beta}(a, b)$ distribution. Sara's illustrative prior therefore has mean $7/12 \approx 0.58$. It describes uncertainty about the on-time probability, rather than the variation between individual trains.

```
prior_grid <- tibble(p = seq(0, 1, length.out = 501), density = dbeta(p, 7, 5))
ggplot(prior_grid, aes(p, density)) +
  geom_area(data = filter(prior_grid, p >= 0.8), fill = "#552E6B", alpha = 0.35) +
  geom_line(color = "#552E6B") +
  geom_vline(xintercept = 7/12, linetype = "dashed") +
  labs(x = "On-time probability", y = "Prior density")
```

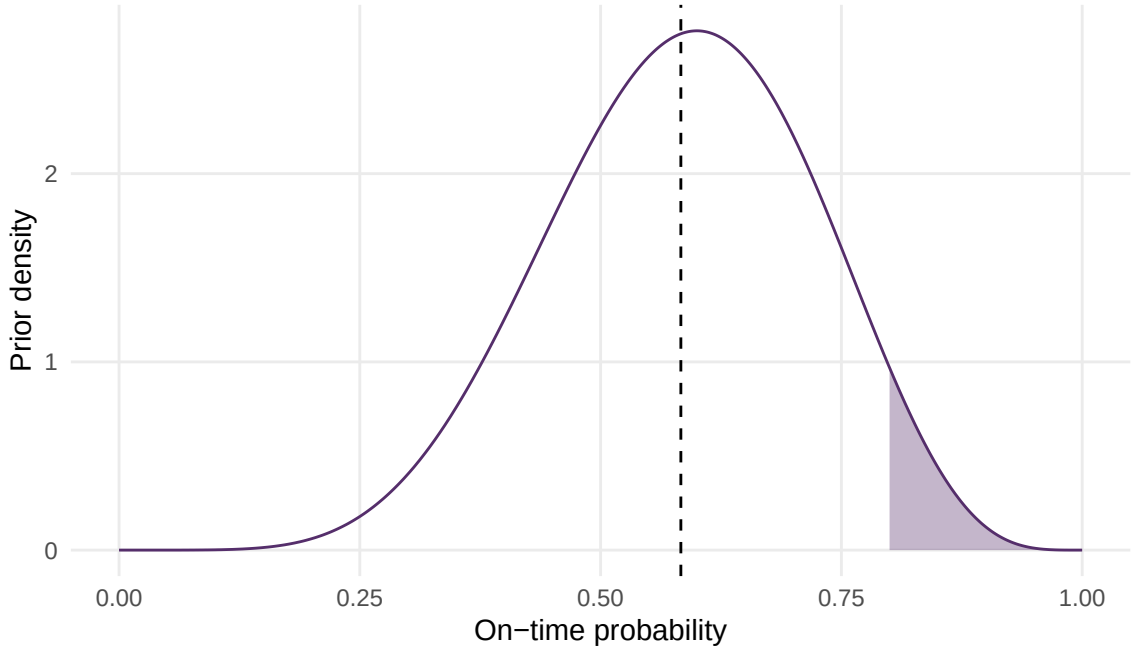


Figure 1.1: Sara's Beta(7,5) prior. The dashed line marks its mean, 7/12. The shaded area is the prior probability that the on-time rate exceeds 0.8; the height of the curve at 0.8 is not that probability.

In Figure 1.1, the whole area under the curve is one. The small shaded area expresses how little prior probability Sara assigns to on-time rates above 0.8.

The **expectation** of a function g of X is the average of $g(X)$, weighted by how probable each value is,

$$E[g(X)] = \sum_x g(x)\, f(x) \quad \text{(discrete)}, \qquad E[g(X)] = \int g(x)\, f(x)\, dx \quad \text{(continuous)}.$$

For discrete X, the sum runs over its possible values. Each term multiplies $g(x)$ by the probability mass $f(x) = P(X = x)$. For continuous X, we instead multiply $g(x)$ by the probability density $f(x)$ and integrate over the support of X, the set of possible values. The density at x is not the probability of that individual value; probabilities are obtained by integrating the density over intervals. For example, if X indicates whether a train is on time and $P(X = 1) = 0.8$, then $E[X] = 0 \times 0.2 + 1 \times 0.8 = 0.8$.

The function g specifies what we want to average. The mean is $E[X]$, with $g(x) = x$. The variance is $E[(X - \mu)^2]$. For a variable X supported on $[0, 1]$, a probability is an expectation too. If $\mathbb{1}\{X > 0.8\}$ is the indicator that equals one when $X > 0.8$ and zero otherwise, then

$$P(X > 0.8) = \int_{0.8}^{1} f(x)\, dx = \int_0^1 \mathbb{1}\{x > 0.8\}\, f(x)\, dx = E[\mathbb{1}\{X > 0.8\}]\,.$$

Expressing summaries as expectations

For discrete X, use $E[g(X)] = \sum_x g(x)f(x)$. For continuous X, use $E[g(X)] = \int g(x)f(x)\, dx$, integrating over its support. Choosing $g(x) = x$ gives the mean; choosing $g(x) = (x - \mu)^2$, where $\mu = E[X]$, gives the variance. An indicator function gives an event probability. These expectations must exist for the corresponding summary to be defined.

For Sara's prior we can compute these integrals exactly, because the Beta family is well studied. The probability that the on-time rate exceeds 0.8 is a Beta integral, and R's `pbeta()` knows the answer. We can also hand the integral to a numerical routine that knows nothing about the Beta family:

```
1 - pbeta(0.8, shape1 = 7, shape2 = 5)
#> [1] 0.05040957
integrate(function(p) dbeta(p, 7, 5), lower = 0.8, upper = 1)
#> 0.05040957 with absolute error < 5.6e-16
integrate(function(p) p * dbeta(p, 7, 5), lower = 0, upper = 1)
#> 0.5833333 with absolute error < 6.5e-15
```

The last line computes the prior mean, $7/12 \approx 0.583$. The function `dbeta()` evaluates a Beta density, whereas `pbeta()` returns the area to the left of its first argument. Thus `1 - pbeta(0.8, 7, 5)` gives the shaded area in Figure 1.1. In the second line, `integrate()` computes that same area numerically; its reported absolute error concerns numerical integration, not uncertainty about π.

Numerical integration is often effective for one parameter. Extending a simple grid to many parameters can be costly. With 100 evaluation points for each of ten parameters, a full grid contains $100^{10} = 10^{20}$ combinations. R's `integrate()` is an adaptive one-dimensional routine, rather than such a multidimensional grid. With many parameters, evaluating every combination of grid points quickly becomes impractical. Monte Carlo offers another approach: Draw combinations of parameter values from the distribution and use them to estimate the quantity of interest.

Quiz yourself!

Sara wants the probability that the on-time rate is between 0.6 and 0.8. Write this probability as an expectation $E[g(\pi)]$. What is g?

1.2 Pseudo-random numbers in R

Simulation needs random numbers, and lots of them. The generators we use produce **pseudo-random numbers**. These are deterministic sequences designed to behave like random draws for statistical computation. The classic recipe is the linear congruential generator, which starts from a seed x_0 and repeats

$$x_{t+1} = (a\,x_t + b) \bmod M,$$

so that each number is an arithmetic function of the previous one (Hanada & Matsuura, 2022, Section 2.1). The operation "modulo M" returns the remainder after division by M; R writes it as `%%`. For example, $(5 \times 7 + 3) \bmod 16 = 6$. Here is a toy version with only 16 possible states. It illustrates repetition, but is not suitable for statistical work:

```
lcg <- function(N, seed, a = 5, b = 3, M = 16) {
  x <- numeric(N)
  x[1] <- seed
  if (N > 1) for (t in 2:N) x[t] <- (a * x[t - 1] + b) %% M
  x / M
}
lcg(20, seed = 7)
#>  [1] 0.4375 0.3750 0.0625 0.5000 0.6875 0.6250 0.3125 0.7500 0.9375 0.8750
#> [11] 0.5625 0.0000 0.1875 0.1250 0.8125 0.2500 0.4375 0.3750 0.0625 0.5000
```

The same seed gives the same sequence. With $M = 16$, there are only 16 possible states, so a state must eventually repeat. In this example the full sequence repeats after 16 values. Practical generators use much larger state spaces and more sophisticated recurrences. R's default, the Mersenne Twister, has a period of $2^{19937} - 1$, so the short repeating pattern of this toy example is avoided.

In R, `runif(N)` gives N pseudo-random numbers that are uniform on $(0, 1)$, and `set.seed()` fixes the seed, which is what makes a simulation reproducible:

```
set.seed(2026)
runif(3)
#> [1] 0.6986735 0.5565305 0.1401400
set.seed(2026)
runif(3)
#> [1] 0.6986735 0.5565305 0.1401400
```

A seed reproduces a calculation when the generator, software, and sequence of random-number calls are the same. Resetting the same seed on every iteration restarts the sequence, so repeated calls can produce identical draws. Set the seed before a simulation rather than inside its sampling loop.

For reproducible work, record `sessionInfo()` and `RNGkind()` along with the code and seed. Changes in R or package versions can change results, and platform-dependent numerical calculations can differ slightly. R's `RNGversion()` can restore earlier generator settings when reproducing older code; a seed alone does not record the full computing environment see the R documentation.

Uniform draws can also be transformed into draws from other distributions. For example, a uniform draw is below 0.8 with probability 0.8, so `as.integer(runif(1) < 0.8)` simulates an on-time indicator. We develop this idea further in the section on inverse-transform sampling.

R uses a consistent naming system for distribution functions:

Prefix	Meaning	Beta example
`d`	Density, or probability mass for a discrete variable	`dbeta(0.8, 7, 5)`

Prefix	Meaning	Beta example
`p`	Cumulative probability $P(X \leq x)$	`pbeta(0.8, 7, 5)`
`q`	Quantile, the value at a specified cumulative probability	`qbeta(0.95, 7, 5)`
`r`	Random draws	`rbeta(1000, 7, 5)`

The same prefixes apply to `norm`, `binom`, `exp`, and many other families. A quantile answers the reverse of a cumulative-probability question. For instance, `qbeta(0.95, 7, 5)` is the value below which 95% of Sara's prior probability lies.

1.3 Monte Carlo integration

We first estimate the mathematical constant $\pi \approx 3.14159$ using points drawn in a square (Hanada & Matsuura, 2022, Section 2.2). In this example only, π denotes the circle constant, not Sara's on-time probability. A circle of radius r has area πr^2. A quarter of a circle of radius one therefore has area $\pi/4$. Place it in the unit square, whose horizontal and vertical coordinates both range from zero to one. The square has area one. If points are drawn uniformly across the square, the probability of falling in the quarter circle equals its fraction of the square's area, $(\pi/4)/1 = \pi/4$.

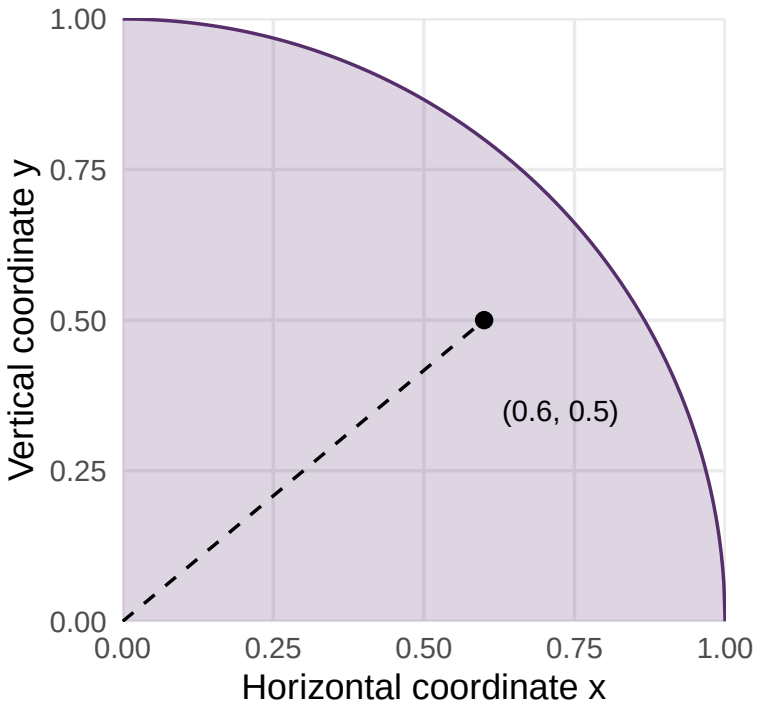


Figure 1.2: The geometry before simulation. A point (x,y) is inside the shaded quarter circle when its distance from the origin is at most one. The example point (0.6,0.5) satisfies 0.6 squared + 0.5 squared = 0.61 < 1.

The distance from the origin to (x, y) is $\sqrt{x^2 + y^2}$ by Pythagoras' theorem. Thus a point is inside when $x^2 + y^2 \leq 1$ (Figure 1.2). To simulate a point, draw its horizontal coordinate with `runif()` and its vertical coordinate independently with another call. Entry `i` of `x` and entry `i` of `y` form point `i`.

The expression `x^2 + y^2 <= 1` checks the condition for every point and returns `TRUE` or `FALSE`. When averaged, R treats these as one or zero. The resulting proportion estimates $\pi/4$, so multiplying by four estimates π.

```r
set.seed(84735)
N <- 10000
x <- runif(N)
y <- runif(N)
inside <- x^2 + y^2 <= 1
4 * mean(inside)
#> [1] 3.1616
```

```r
darts <- tibble(x, y, inside)
running <- tibble(n_darts = seq_len(N), estimate = 4 * cumsum(inside) / n_darts)

p_darts <- darts |>
  slice_head(n = 2000) |>
  ggplot(aes(x, y, color = inside)) +
  geom_point(size = 0.5, alpha = 0.6) +
  scale_color_manual(values = c("TRUE" = "#552E6B", "FALSE" = "#ECECEC"), guide =
    "none") +
  coord_equal() +
  labs(x = NULL, y = NULL)

p_running <- ggplot(running, aes(n_darts, estimate)) +
  geom_hline(yintercept = pi, linetype = "dashed") +
  geom_line(color = "#552E6B") +
  scale_x_log10() +
  labs(x = "Number of darts (log scale)", y = expression(hat(pi)))

wrap_plots(p_darts, p_running, widths = c(1, 1.6))
```

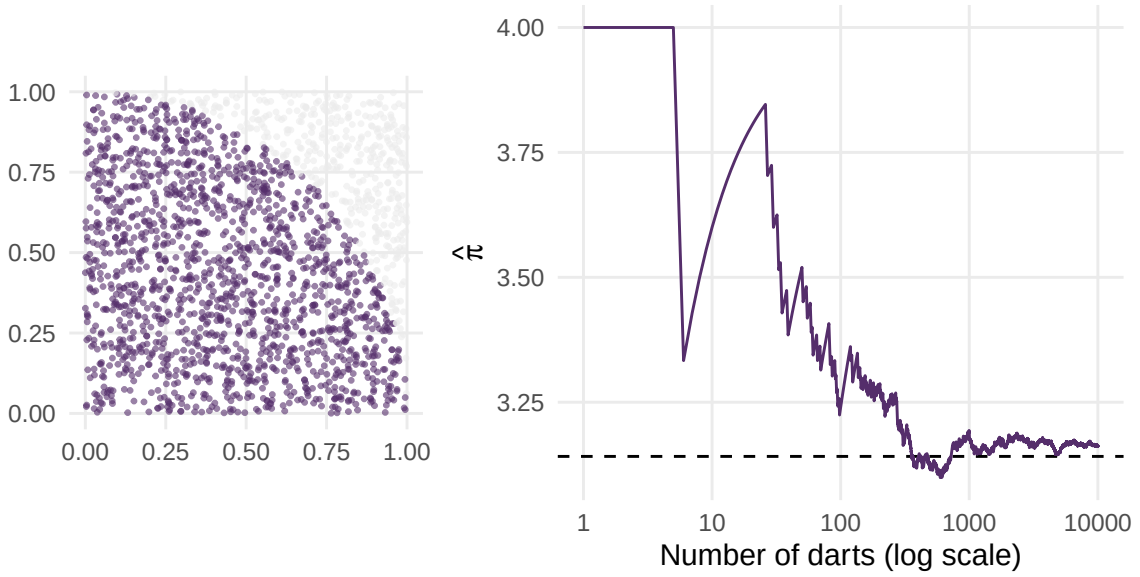


Figure 1.3: Estimating π by throwing darts. Left: the first 2,000 darts; those inside the quarter circle are purple. Right: four times the running proportion of darts inside, as the number of darts grows. The estimate wanders at first and settles down toward π (dashed line).

Why does this work? Write I_i for the indicator that dart i lands inside. Each I_i is a random variable with $E[I_i] = P(\text{inside}) = \pi/4$, and the darts are independent. The **Law of Large Numbers** states that an

average of independent, identically distributed draws converges to its expectation, provided the absolute expectation is finite. The condition holds here because the indicators are bounded between zero and one. So $\frac{1}{N}\sum_i I_i \to \pi/4$, and our estimate $4\bar{I} \to \pi$. The right panel of Figure 1.3 is the Law of Large Numbers in action.

Now look at what we did in the language of the previous section. The darts (X, Y) are draws from the uniform density on the unit square, and $\pi/4 = E[\mathbb{1}\{X^2 + Y^2 \le 1\}]$ is an expectation under that density. We estimated an integral by drawing from the density and averaging the function. The same averaging procedure applies to other distributions and functions.

Monte Carlo integration

To approximate $E[g(X)]$, defined by a sum or integral:

1. Draw $X^{(1)}, \ldots, X^{(N)}$ independently from f.
2. Compute the average $\bar{g} = \frac{1}{N}\sum_{i=1}^{N} g(X^{(i)})$.

When $E[|g(X)|] < \infty$, the Law of Large Numbers implies that $\bar{g} \to E[g(X)]$ as $N \to \infty$. If $g(X)$ also has finite variance, the **Monte Carlo standard error** of $\bar{g}$ is

$$\mathrm{SE}(\bar{g}) = \frac{\mathrm{sd}(g(X))}{\sqrt{N}} \approx \frac{1}{\sqrt{N}}\sqrt{\frac{1}{N-1}\sum_{i=1}^{N}\left(g(X^{(i)}) - \bar{g}\right)^2},$$

and for large N the estimate is approximately Normal, so $\bar{g} \pm 2\,\mathrm{SE}$ is an approximate 95% confidence interval for the expectation being estimated. This uses the Central Limit Theorem and a finite, nonzero variance; it is not a guarantee for every simulation size.

1.3.1 How accurate is it?

The right panel of Figure 1.3 shows the estimate settling down, but with 10,000 darts it has not necessarily arrived; our run gives 3.162 against $\pi \approx 3.1416$. How far off should we expect it to be? The estimate is an average of independent draws of $g(X, Y) = 4 \cdot \mathbb{1}\{X^2 + Y^2 \le 1\}$, so the **Central Limit Theorem** applies: the average is approximately Normal with mean π and standard deviation $\mathrm{sd}(g)/\sqrt{N}$. We can estimate $\mathrm{sd}(g)$ from the same draws:

```
g <- 4 * inside
estimate <- mean(g)
mc_se <- sd(g) / sqrt(N)
c(estimate = estimate, se = mc_se)
#>   estimate         se
#> 3.16160000 0.01628174
```

Our estimate of π is 3.162, with estimated standard error 0.016. Its approximate 95% Monte Carlo confidence interval is

$$[3.129,\ 3.194].$$

If we repeated the entire simulation many times and constructed a confidence interval from each simulation, approximately 95% of these intervals would contain the true value of π. This interval describes uncertainty from using finitely many simulated points. It does not describe uncertainty about a participant or a railway parameter.

The standard error decreases at rate $1/\sqrt{N}$. For example, increasing the number of darts from 10,000 to 40,000 halves the standard error; increasing it to 1,000,000 reduces it by a factor of ten. These statements concern the typical size of simulation error. An individual estimate need not move closer to the truth every time more draws are added.

Quiz yourself!

If you quadruple the number of darts, by what factor does the Monte Carlo standard error shrink? Using $\text{SE} = \sqrt{\pi(4-\pi)/N}$, how many darts are needed for an approximate 95% confidence interval with half-width 0.001? You may use $\pi \approx 3.14159$ for this planning calculation.

Let's check the $1/\sqrt{N}$ rule directly by repeating the whole experiment 200 times for three sample sizes and looking at the spread of the estimates.

```
set.seed(84735)
estimate_pi <- function(N) 4 * mean(runif(N)^2 + runif(N)^2 <= 1)

repeats <- expand_grid(N = c(100, 1000, 10000), rep = 1:200) |>
  mutate(estimate = map_dbl(N, estimate_pi))

repeats |>
  ggplot(aes(factor(N), estimate)) +
  geom_hline(yintercept = pi, linetype = "dashed") +
  geom_point(position = position_jitter(width = 0.25, seed = 1),
             alpha = 0.3, color = "#552E6B", size = 0.8) +
  labs(x = "Number of darts", y = expression(hat(pi)))
```

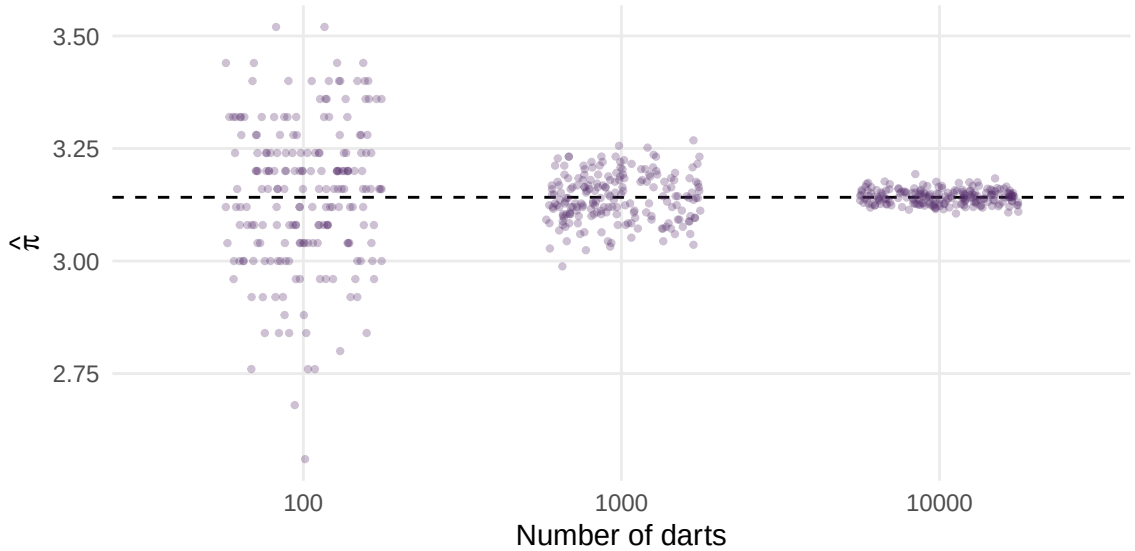


Figure 1.4: Two hundred independent estimates of π for each of three numbers of darts. The spread shrinks by a factor of about $\sqrt{10}$ each time the number of darts grows tenfold.

```
repeats |>
  group_by(N) |>
  summarize(observed_sd = sd(estimate)) |>
  mutate(theoretical_se = sqrt(pi * (4 - pi) / N))
#> # A tibble: 3 x 3
#>       N observed_sd theoretical_se
#>   <dbl>       <dbl>          <dbl>
#> 1   100      0.163          0.164
#> 2  1000      0.0527         0.0519
#> 3 10000      0.0158         0.0164
```

The observed standard deviation of the estimates matches the theoretical standard error $\sqrt{\pi(4-\pi)/N}$, which follows from the variance of the indicator, $\mathrm{Var}(4I) = 16 \cdot \frac{\pi}{4}(1-\frac{\pi}{4}) = \pi(4-\pi)$. Each row uses ten times as many darts as the one above it and is about $\sqrt{10} \approx 3.2$ times more precise.

1.3.2 The recipe in general

The Monte Carlo recipe applies whenever we can generate independent draws from the required distribution and evaluate a function with a finite expectation. Return to the stop-signal task. For illustration, let the go duration be the sum of a Normal variable with mean 500 ms and standard deviation 50 ms and an Exponential variable with mean 100 ms. Such a sum is called an *ex-Gaussian* variable. Let the stop duration be a Normal variable with mean 200 ms and standard deviation 30 ms plus an Exponential variable with mean 50 ms. These choices illustrate the computation rather than estimates from a particular experiment.

We assume that the components and the two processes are independent. All finishing times must be measured from the same starting point. The go process begins at the arrow, whereas the stop process begins 250 ms later. We therefore add 250 to the simulated stop duration before comparing the finishing times. In R, an Exponential mean of 100 requires `rate = 1/100`, since its mean is the reciprocal of its rate.

```
set.seed(84735)
N_races <- 100000
go_time   <- rnorm(N_races, mean = 500, sd = 50) + rexp(N_races, rate = 1 / 100)
stop_time <- 250 + rnorm(N_races, mean = 200, sd = 30) + rexp(N_races, rate = 1 / 50)
stop_wins <- stop_time < go_time
p_stop <- mean(stop_wins)
c(estimate = p_stop, se = sd(stop_wins) / sqrt(N_races))
#>    estimate          se
#> 0.811480000 0.001236858
```

Under these illustrative assumptions, the probability of stopping is approximately 81.1%, with Monte Carlo standard error 0.12 percentage points. We drew from the joint density of the two finishing times, which is easy because each is a sum of things R can draw, and averaged the indicator that the stop process won. We could just as easily estimate the mean finishing time of the winning process, the probability that a go response takes longer than 800 ms, or any other expectation, from the same draws. For each quantity, first specify the function to average and check that its expectation exists. The standard error concerns simulation under the chosen model, not uncertainty about whether the model is appropriate.

1.4 Generating draws from a distribution

So far, R has provided the required draws. In Bayesian analyses, however, the posterior distribution often does not belong to a familiar family with a built-in random generator. We therefore need methods for constructing draws. Inverse-transform sampling uses cumulative probabilities; rejection sampling uses the shape of a density. Both can produce independent draws when their inputs are independently generated.

1.4.1 Inverse-transform sampling

The **cumulative distribution function** (CDF), $F(x) = P(X \le x)$, records how much probability lies to the left of x. Its inverse, the **quantile function**, starts with a cumulative probability u and returns the corresponding value of x. For a continuous, strictly increasing CDF, $F^{-1}(u)$ is the value satisfying $F(x) = u$.

Imagine choosing a height u uniformly between zero and one on a CDF plot. Move horizontally to the curve, then down to the x axis. The value you reach is a draw from the desired distribution. The left panel of Figure 1.5, below, shows five such constructions.

Formally, draw $U \sim \mathrm{Unif}(0,1)$ and set $X = F^{-1}(U)$. Because F is increasing, the events $F^{-1}(U) \le x$ and

$U \le F(x)$ are equivalent. Hence

$$P(X \le x) = P(F^{-1}(U) \le x) = P(U \le F(x)) = F(x).$$

The last equality holds because a uniform draw falls below any number $u \in [0,1]$ with probability u. Thus the generated X has the desired CDF.

Take the Exponential distribution with rate λ, which has $F(x) = 1 - e^{-\lambda x}$ for $x \ge 0$. To solve $u = 1 - e^{-\lambda x}$, rearrange to $1 - u = e^{-\lambda x}$, take logarithms to get $\log(1-u) = -\lambda x$, and divide by $-\lambda$. This gives $F^{-1}(u) = -\log(1-u)/\lambda$. So an Exponential draw is one line of code:

```
set.seed(84735)
r_exponential <- function(N, rate) -log(1 - runif(N)) / rate
draws_exp <- r_exponential(10000, rate = 2)
```

```
u_demo <- c(0.15, 0.4, 0.62, 0.8, 0.95)
x_demo <- -log(1 - u_demo) / 2
cdf_df <- tibble(x = seq(0, 3, length.out = 300), cdf = pexp(x, rate = 2))
paths <- tibble(u = u_demo, x = x_demo)

p_cdf <- ggplot(cdf_df, aes(x, cdf)) +
  geom_line(color = "gray50") +
  geom_segment(data = paths, aes(x = 0, xend = x, y = u, yend = u),
               color = "#552E6B", linetype = "dotted") +
  geom_segment(data = paths, aes(x = x, xend = x, y = u, yend = 0),
               color = "#552E6B", arrow = arrow(length = unit(0.15, "cm"))) +
  geom_point(data = paths, aes(0, u), color = "#552E6B") +
  labs(x = "x", y = "F(x)")

p_hist <- ggplot(tibble(x = draws_exp), aes(x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "#ECECEC", color =
    "white") +
  stat_function(fun = dexp, args = list(rate = 2), color = "#552E6B", linewidth = 1) +
  labs(x = "x", y = "Density")

wrap_plots(p_cdf, p_hist)
```

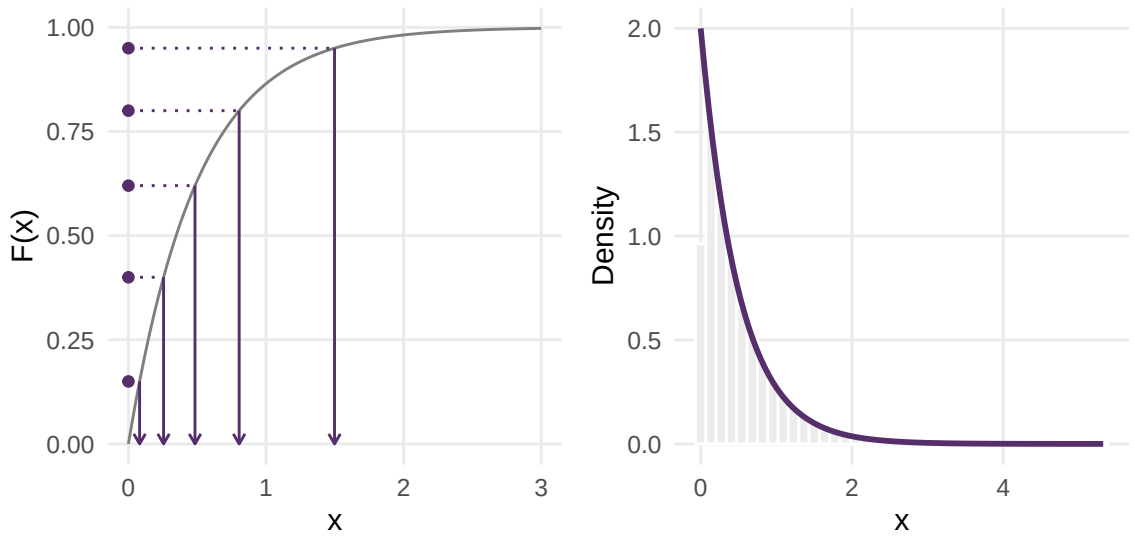


Figure 1.5: Inverse-transform sampling for the Exponential(2) distribution. Left: five values of u (points on the vertical axis) are mapped through the inverse CDF to values $x = F^{-1}(u)$ on the horizontal axis. Right: 10,000 draws produced this way, with the Exponential(2) density overlaid.

The histogram follows the Exponential density, and the simulated mean, 0.502, is close to $1/\lambda = 0.5$. This is one way to generate Exponential draws; it is not the algorithm used internally by R's `rexp()` see its documentation.

A numerical quantile function is sufficient; a closed-form inverse is not required. For example, `qnorm(runif(1000))` generates standard Normal draws even though the Normal quantile function has no elementary formula. For a discrete distribution, use the smallest x for which $F(x) \geq u$. This selects the appropriate step of its CDF.

The limitation is that we need F^{-1}. For a posterior density we typically know only the shape, $f(\theta \mid y) \propto f(y \mid \theta) f(\theta)$, and computing the CDF, let alone inverting it, requires exactly the integral we were trying to avoid.

1.4.2 Rejection sampling

Rejection sampling generates candidates from an accessible distribution and keeps some of them to obtain the distribution we want. Suppose we can evaluate the target density $f(x)$, or even just a function $h(x)$ that is proportional to it, $f(x) = h(x)/Z$ with an unknown constant Z. Suppose also that we have some other density $q(x)$, called the **proposal**, that we *can* draw from, and a constant M such that

$$h(x) \leq M\, q(x) \quad \text{for every } x.$$

The function $Mq(x)$ is an **envelope**. It lies at or above h everywhere. Rejection sampling draws a point uniformly under the envelope and keeps it if it also lies under h.

For a uniform proposal on $[0, 1]$, the envelope is a rectangle. A candidate's horizontal position is uniform; its vertical position is uniform between zero and the envelope's height. Where the target curve is high, a larger fraction of candidates lie below it and are kept. This makes accepted values more common in high-density regions. The left panel of Figure 1.6, below, shows the construction.

Rejection sampling

Given a target $h(x) \propto f(x)$, a proposal density q, and a constant M with $h(x) \leq Mq(x)$ for all x:

1. **Propose.** Draw $x^\star \sim q$.
2. **Accept or reject.** Draw $u \sim \text{Unif}(0, 1)$. If $u < h(x^\star)/(Mq(x^\star))$, accept $x^\star$; otherwise reject it

and return to step 1.

The accepted values are independent draws from f. The probability that a proposal is accepted is Z/M when q is a proper density, where $Z = \int h(x)\,dx$.

Why does this produce draws from f? A proposal at x is accepted with probability $h(x)/(Mq(x))$, so the density of proposing x *and* accepting it is

$$q(x) \cdot \frac{h(x)}{Mq(x)} = \frac{h(x)}{M}.$$

This is a joint density for the candidate value together with acceptance, not yet the density among accepted values. Its integral is the probability of acceptance,

$$P(\text{accept}) = \int \frac{h(x)}{M}\,dx = \frac{Z}{M}.$$

To obtain the conditional density among accepted values, divide by that probability:

$$f(x \mid \text{accept}) = \frac{h(x)/M}{Z/M} = \frac{h(x)}{Z} = f(x).$$

This explains why the unknown normalizing constant Z is not needed to run the algorithm. The acceptance step corrects the relative frequencies of the proposed values.

For a fixed proposal q, we want the smallest *valid* M. Increasing M enlarges the envelope without changing the area under h, so the acceptance rate Z/M falls. Choosing M too small is different; it invalidates the sampler wherever $h(x) > Mq(x)$.

We now return to using π for an on-time probability. Apply rejection sampling to the function $h(\pi) = \pi^{18}(1-\pi)^7$ on the interval $[0,1]$. A function proportional to a density is called its **kernel**. This one is the kernel of a Beta$(19, 8)$ density, which gives us a known distribution against which to check the sampler. The simplest proposal is the uniform density on $[0,1]$, $q(\pi) = 1$, and the smallest valid M is then the maximum of h, which is at $\pi = 18/25$. Every proposal is a uniform point in the unit interval and is accepted with probability $h(\pi^\star)/M$.

```
set.seed(84735)
h <- function(p) p^18 * (1 - p)^7
M <- h(18 / 25)

N_prop <- 20000
proposal <- runif(N_prop)
u <- runif(N_prop)
accept <- u < h(proposal) / M
draws_rej <- proposal[accept]
mean(accept)
#> [1] 0.2196
```

```
env_df <- tibble(proposal, u, accept) |> slice_head(n = 3000)
curve_h <- tibble(p = seq(0, 1, length.out = 400), scaled_h = h(p) / M)

p_env <- ggplot() +
  geom_point(data = env_df, aes(proposal, u, color = accept), size = 0.5, alpha = 0.6)
    +
  geom_line(data = curve_h, aes(p, scaled_h), color = "gray50") +
  geom_hline(yintercept = 1, linetype = "dashed") +
```

```
  scale_color_manual(values = c("TRUE" = "#552E6B", "FALSE" = "#ECECEC"), guide =
    "none") +
  labs(x = expression(pi), y = "u")

p_rej <- ggplot(tibble(p = draws_rej), aes(p)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "#ECECEC", color =
    "white") +
  stat_function(fun = dbeta, args = list(shape1 = 19, shape2 = 8), color = "#552E6B",
    linewidth = 1) +
  labs(x = expression(pi), y = "Density")

wrap_plots(p_env, p_rej)
```

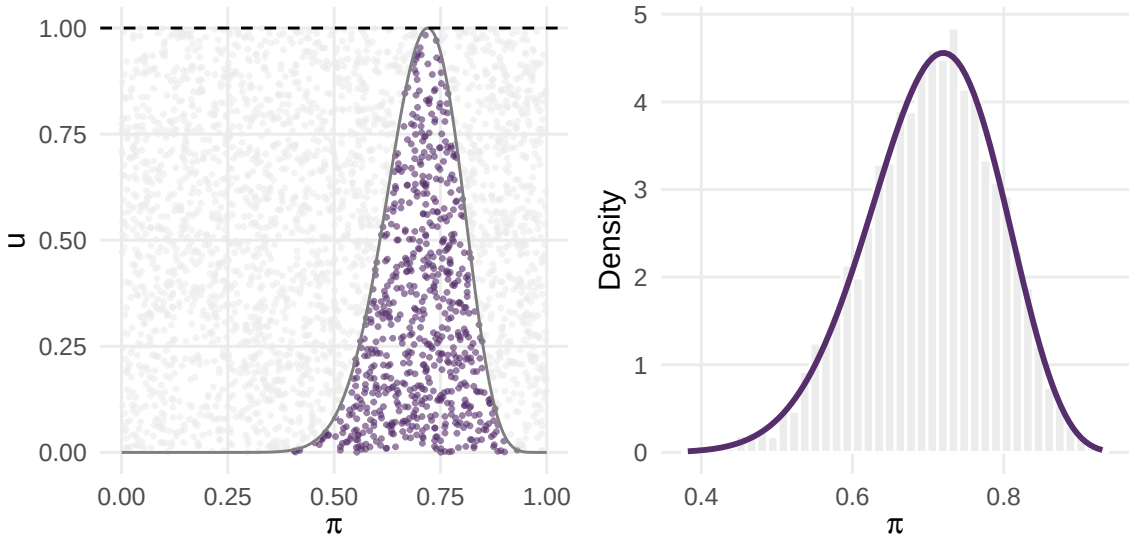


Figure 1.6: Rejection sampling for $h(\pi) = \pi^{18}(1-\pi)^7$ with a uniform proposal. Left: the first 3,000 proposals, placed at height u under the flat envelope at 1, so that the target is drawn as $h(\pi)/M$; points under the curve (purple) are accepted, the rest (gray) are rejected. Right: histogram of all accepted draws with the Beta(19, 8) density overlaid.

The left panel of Figure 1.6 shows how acceptance changes the distribution of candidates. The proposals fill the rectangle under the envelope uniformly; the accepted ones fill the region under the scaled target h/M uniformly; and a uniform scatter under a curve, projected onto the horizontal axis, has that curve as its density. The right panel provides a visual check that the 4,392 accepted draws follow the Beta$(19, 8)$ density.

The price is the rejections. Only 22% of the proposals survived, close to the theoretical rate $Z/M = B(19, 8)/M = 0.219$, where $B(19, 8)$ is the Beta function, the normalizing constant of h. The acceptance rate is the ratio of the area under h to the area under the envelope; a tighter envelope wastes fewer proposals. Here the flat envelope is a poor fit to a peaked target, and about 78% of the proposals are thrown away. Whether this is affordable depends on how many accepted draws we need and how expensive each proposal is. With several parameters, a poorly fitting envelope can become much more wasteful.

Products of many small likelihood terms can underflow, meaning the computer rounds them to zero. To avoid this, compute log densities directly and compare `log(u)` with `log_h(proposal) - log_M`. Taking `log(h(proposal))` after an underflow has occurred cannot recover the lost value. Chapter 2 develops calculations on the log scale.

The Metropolis algorithm in Chapter 5 also proposes a value and then accepts or rejects it. Its acceptance rule is different and depends on the current state. We will derive that rule separately.

Quiz yourself!

Suppose you had used $M = 2\,h(18/25)$ instead of $M = h(18/25)$, an envelope twice as high as necessary. What happens to the acceptance rate? What happens to the distribution of the accepted draws? And what if you had used $M = h(18/25)/2$?

1.5 Simulating a Bayesian model forward

Sara records 12 on-time arrivals among 15 trains. Conditional on π, we model the count as $Y \mid \pi \sim \text{Bin}(15, \pi)$. A Binomial distribution describes the number of successes in independent trials with a common success probability. Its probability mass at 12 is

$$P(Y = 12 \mid \pi) = \binom{15}{12} \pi^{12}(1-\pi)^3.$$

The coefficient counts the arrangements of 12 on-time and three late arrivals. With the observed count fixed, this expression is the **likelihood**, viewed as a function of π.

Bayes' rule updates the prior by multiplying it by the likelihood and then normalizing. The Beta$(7, 5)$ prior has kernel $\pi^6(1-\pi)^4$, so

$$f(\pi \mid Y = 12) \propto \underbrace{\pi^{12}(1-\pi)^3}_{\text{likelihood kernel}} \underbrace{\pi^6(1-\pi)^4}_{\text{prior kernel}} = \pi^{18}(1-\pi)^7.$$

This is the Beta$(19, 8)$ kernel used in the rejection-sampling example. The **posterior distribution** represents Sara's uncertainty after observing the data. Chapter 2 develops Bayesian updating further.

We can also obtain this posterior by simulation. First draw a possible on-time probability from the prior, then generate a count using that value. Repeating these steps gives pairs $(\pi^{(s)}, Y^{(s)})$, where s indexes simulations. Their joint distribution is $f(\pi)f(y \mid \pi)$. The distribution of the simulated counts alone is the **prior predictive distribution**; it describes data predicted before observing Sara's record.

Now retain only pairs whose simulated count is 12. Within those pairs, the distribution of π is the conditional distribution given $Y = 12$, which is precisely the posterior. Each entry of `pi_prior` below is used to generate one corresponding entry of `y_sim`; the logical condition selects the matching parameter draws.

```
set.seed(84735)
N_sim <- 100000
pi_prior <- rbeta(N_sim, shape1 = 7, shape2 = 5)          # 1. draw pi from the prior
y_sim    <- rbinom(N_sim, size = 15, prob = pi_prior)      # 2. simulate a week of
  trains
pi_post  <- pi_prior[y_sim == 12]                          # 3. keep the weeks that
  match
length(pi_post)
#> [1] 8358
```

```
joint <- tibble(pi = pi_prior, y = y_sim, keep = y_sim == 12)

p_joint <- joint |>
  slice_head(n = 5000) |>
  ggplot(aes(pi, y, color = keep)) +
  geom_point(position = position_jitter(height = 0.25, width = 0, seed = 1),
             size = 0.5, alpha = 0.5) +
  scale_color_manual(values = c("TRUE" = "#552E6B", "FALSE" = "#ECECEC"), guide =
    "none") +
```

```
  scale_y_continuous(breaks = seq(0, 15, 3)) +
  labs(x = expression(pi), y = "Simulated Y")

p_post <- ggplot(tibble(pi = pi_post), aes(pi)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "#ECECEC", color =
    "white") +
  stat_function(fun = dbeta, args = list(shape1 = 19, shape2 = 8), color = "#552E6B",
    linewidth = 1) +
  labs(x = expression(pi), y = "Density")

wrap_plots(p_joint, p_post)
```

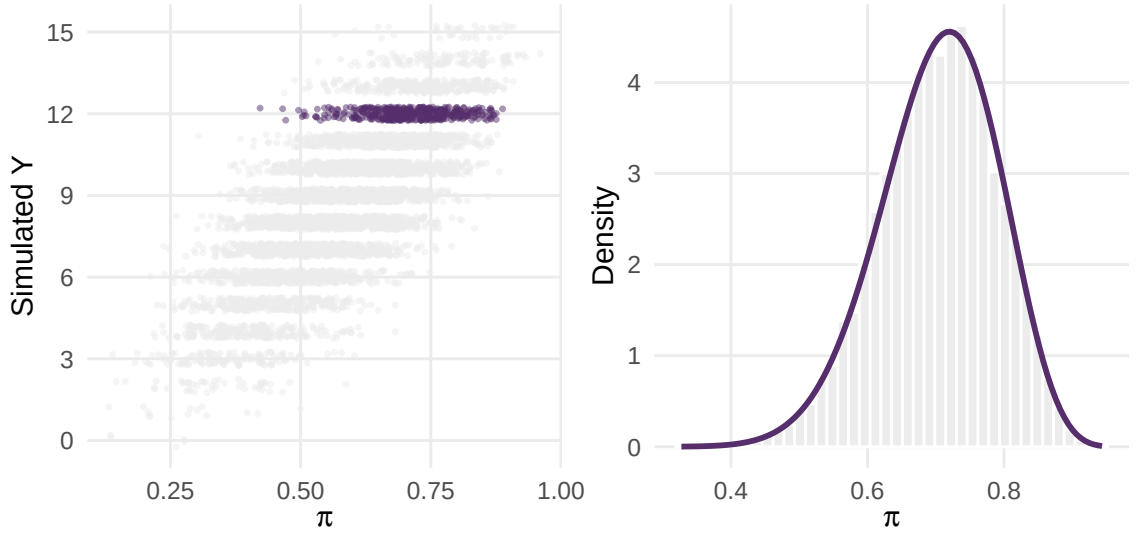


Figure 1.7: Forward simulation of the Beta-Binomial model. Left: 5,000 of the simulated (π, Y) pairs from the joint distribution of parameter and data; the pairs with $Y = 12$, the observed value, are purple. Right: histogram of all retained draws of π with the exact Beta(19, 8) posterior overlaid.

The left panel of Figure 1.7 shows the joint distribution. Small π produces small Y, large π produces large Y, and the purple row is the slice at $Y = 12$. The right panel shows that the π values in that slice are distributed as Beta(19, 8). Their summaries agree with the exact posterior as well:

```
c(mean = mean(pi_post), sd = sd(pi_post),
  lower = quantile(pi_post, 0.025, names = FALSE), upper = quantile(pi_post, 0.975,
    names = FALSE))
#>       mean         sd      lower      upper
#> 0.70253618 0.08615037 0.52149758 0.85547385
c(mean = 19 / 27, sd = sqrt(19 * 8 / (27^2 * 28)), lower = qbeta(0.025, 19, 8), upper
  = qbeta(0.975, 19, 8))
#>       mean         sd      lower      upper
#> 0.70370370 0.08629368 0.52212521 0.85674000
```

The retained draws describe uncertainty about π. Their standard deviation is a posterior standard deviation; the interval between their 2.5th and 97.5th percentiles is a 95% equal-tailed credible interval. The Monte Carlo standard error of the estimated posterior mean is a different quantity, estimated by `sd(pi_post) / sqrt(length(pi_post))`. More simulations reduce that computational error without changing the posterior itself.

The link to rejection sampling follows by considering the probability of keeping a proposed $\pi^\star$. It is $P(Y = 12 \mid \pi^\star)$, since that is the probability that the simulated count matches the observed one. This value is at most one because Y is discrete and we use its full probability mass, including the Binomial coefficient. A likelihood in general is not a probability distribution over the parameter, and a continuous-data likelihood is not bounded by one.

With proposal $q(\pi) = f(\pi)$ and unnormalized target $h(\pi) = f(12 \mid \pi) f(\pi)$, we therefore have $h(\pi) \leq q(\pi)$. The rejection constant $M = 1$ is valid, and the acceptance ratio is $h(\pi)/q(\pi) = f(12 \mid \pi)$.

Forward simulation is rejection sampling

For discrete data, drawing π from the prior, simulating data, and keeping the draws whose simulated data equal the observed data y is rejection sampling with proposal $q(\pi) = f(\pi)$, unnormalized target $h(\pi) = f(\pi) f(y \mid \pi)$ using the full data probability mass, and $M = 1$. Its acceptance rate is

$$P(\text{keep}) = \int f(y \mid \pi) f(\pi)\, d\pi = f(y),$$

the prior predictive probability of the observed count, also called its marginal likelihood. Chapter 2 discusses this quantity further.

This calculation also reveals the computational cost. Only a fraction $f(y)$ of simulations are retained. For the Beta-Binomial model, integrating the likelihood against the prior gives

$$f(y) = \binom{n}{y} \frac{B(7 + y, 5 + n - y)}{B(7, 5)},$$

where $B(a, b) = \int_0^1 p^{a-1}(1-p)^{b-1}\, dp$ is the Beta normalizing constant. The following function evaluates the expression on the log scale before converting it back to a probability. `lchoose()` and `lbeta()` return the logarithms of the Binomial coefficient and Beta function.

```
f_y <- function(y, n, a, b) {   # computed on the log scale to avoid overflow
  exp(lchoose(n, y) + lbeta(a + y, b + n - y) - lbeta(a, b))
}
f_y(12, n = 15, a = 7, b = 5)
#> [1] 0.08409611
```

The calculation predicts the acceptance rate; about 8% of the simulated records were retained in our run. Now suppose Sara keeps going. Consider longer records with 120 of 150 and 1,200 of 1,500 trains on time. The observed proportion stays at 0.8. The chance that a simulated record matches hers exactly shrinks roughly in proportion to the number of trains. We reuse the prior draws in `pi_prior`, and the week already simulated in `y_sim`, and simulate the two longer records from the same prior draws:

```
set.seed(84735)
y_sim_150  <- rbinom(N_sim, size = 150,  prob = pi_prior)
y_sim_1500 <- rbinom(N_sim, size = 1500, prob = pi_prior)
survival <- tibble(
  n = c(15, 150, 1500), y = c(12, 120, 1200),
  predicted = f_y(y, n, a = 7, b = 5),
  survivors = c(sum(y_sim == 12), sum(y_sim_150 == 120), sum(y_sim_1500 == 1200))
)
survival
#> # A tibble: 3 x 4
#>       n     y predicted survivors
#>   <dbl> <dbl>     <dbl>     <int>
#> 1    15    12  0.0841        8358
```

```
#> 2   150   120  0.00686        668
#> 3  1500  1200  0.000650        68
```

For the largest record, 68 of 100,000 simulations reproduce the observed count. These are still valid posterior draws, but fewer retained values mean more Monte Carlo error. Whether they are sufficient depends on the summary and precision required. A small number might give a rough mean while providing little information about a small tail probability.

Matching additional observations can lower acceptance further, but the joint matching probability is a product of marginal probabilities only under independence. For continuous observations, the probability of an exact match is zero. Accepting values within a tolerance changes the target and requires additional justification.

Quiz yourself!

Sara's colleague Joost keeps a longer record: 60 trains, 48 of them on time. Under the same Beta$(7, 5)$ prior, roughly how many of 100,000 forward simulations will match his record? Use the function `f_y()` to find out, and explain why the answer is smaller than for Sara's week.

1.6 When independent draws are not enough

Monte Carlo integration and sampling are separate tasks. If we can draw from the desired distribution, we can estimate many of its summaries by averaging. The difficulty is often obtaining those draws efficiently. Familiar distributions have built-in generators, and some others have accessible quantile functions or useful rejection envelopes. More complicated posteriors may have none of these conveniences.

1.6.1 One parameter: Using what we can evaluate

A posterior can often be evaluated up to a constant through

$$f(\theta \mid y) \propto f(y \mid \theta) f(\theta).$$

Here θ denotes the unknown parameter or parameters, and y the fixed observed data. The right-hand side gives the posterior's relative height at different values of θ, even when the integral needed to normalize it is difficult. This is enough for rejection sampling if we can find a suitable envelope. The Beta example showed the calculation in one dimension, where the target and envelope can be plotted directly.

1.6.2 Several parameters: Why a poor proposal becomes expensive

Consider a simple illustration with several parameters, each between zero and one. Suppose the target is uniform on the interval $[0.25, 0.75]$ for each parameter, but our proposal draws every coordinate independently from $[0, 1]$. A proposal is kept only if *all* its coordinates lie in the target intervals.

For one parameter, the acceptable interval occupies half the proposal range, so the acceptance probability is $1/2$. For two parameters, both independent conditions must hold, giving $(1/2)(1/2) = 1/4$. With d parameters, the acceptance rate is $(1/2)^d$.

At ten parameters, about one in 1,024 proposals is accepted; at twenty, about one in a million. Each individual condition is easy to satisfy, yet satisfying all of them at once is unlikely. This example requires only multiplication of independent probabilities, rather than geometry in higher dimensions.

The problem is the mismatch between proposal and target. In this deliberately simple example, drawing each coordinate directly from $[0.25, 0.75]$ solves it completely. For a real posterior, finding a proposal that matches both its concentration and the relationships between parameters can be much harder. Dimension alone does not force rejection sampling to fail, and dependent draws are not the only possible solution.

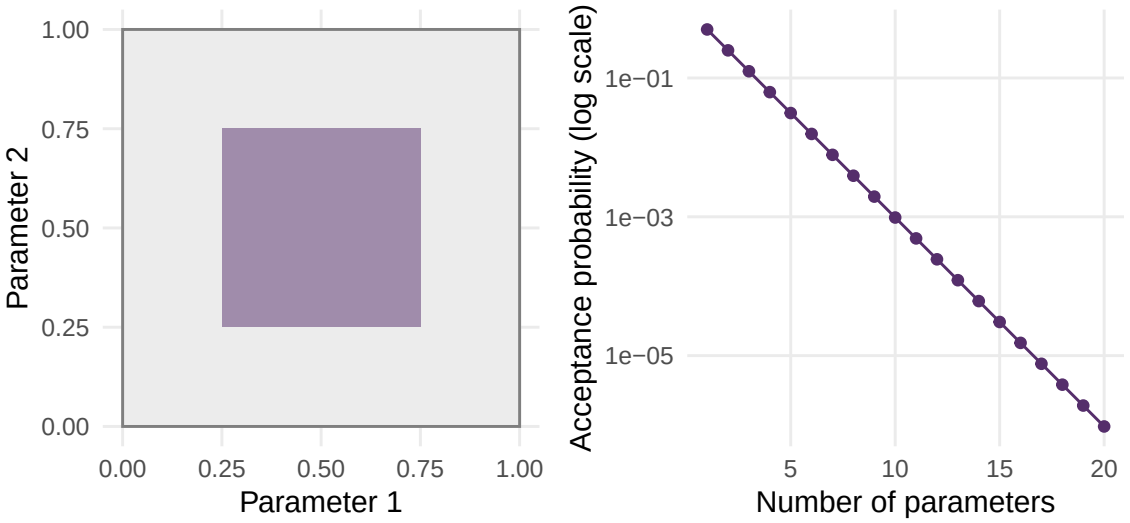


Figure 1.8: A broad proposal with several parameters. Left: For two parameters the acceptable central square occupies one quarter of the proposal square. Right: Requiring all d independent coordinates to fall in their middle halves gives acceptance probability (1/2) to the power d. This illustrates a particular poor proposal, not a universal rate for high-dimensional sampling.

Markov chain Monte Carlo offers another approach. Instead of generating every candidate independently, we construct a sequence in which the next state depends on the current state. With an appropriate transition rule, long-run averages can estimate expectations under the target distribution. A **Markov chain** has the specific property that, conditional on the current state, its next state does not depend on the earlier history. Dependence alone does not ensure either this property or the desired long-run distribution.

A chain can use its current location to propose nearby values, but this does not guarantee efficient exploration. It may move slowly or remain in one region for a long time. Chapter 3 develops the conditions needed for these methods, and Chapter 4 explains how to assess their performance.

1.7 Chapter summary

This chapter introduced simulation as a way to calculate properties of probability models.

- **Expectations** are probability-weighted sums, $\sum_x g(x)f(x)$, or integrals, $\int g(x)f(x)\,dx$. Means, variances, and event probabilities can be expressed this way when they exist.
- **Monte Carlo integration** draws independently from the required distribution and averages $g(X)$. The Law of Large Numbers establishes convergence when the absolute expectation is finite. With finite variance, the standard error is $\mathrm{sd}(g(X))/\sqrt{N}$. The rate does not explicitly depend on dimension, but the variance and difficulty of generating draws can.
- **Pseudo-random numbers** come from a deterministic generator. Reproducibility requires the seed, code, generator settings, and software environment.
- **Inverse-transform sampling** converts uniform draws into quantiles of the target distribution. A numerical quantile function can be sufficient.
- **Rejection sampling** accepts a proposal from q with probability $h(x)/(Mq(x))$. A valid envelope satisfies $h(x) \leq Mq(x)$ everywhere, and a tighter envelope improves acceptance for a fixed proposal.
- **Forward simulation** generates parameters and data from their joint distribution. For discrete data, retaining exact matches gives posterior draws. The acceptance rate is the prior predictive probability of the observed data.
- Poorly matched proposals can be expensive with many parameters. **Markov chain Monte Carlo** is one approach to this difficulty; its dependent draws require separate convergence and accuracy checks.

1.8 Quiz answers

Quiz 1. $P(0.6 < \pi < 0.8) = E[g(\pi)]$ with $g(\pi) = \mathbb{1}\{0.6 < \pi < 0.8\}$, the function that equals one inside the interval and zero outside it, and the expectation taken under the Beta$(7, 5)$ prior. The Monte Carlo estimate is the proportion of prior draws that land in the interval.

Quiz 2. The standard error is proportional to $1/\sqrt{N}$, so quadrupling N halves it. For a 95% interval of ± 0.001 we need $2\sqrt{\pi(4-\pi)/N} \leq 0.001$, which gives $N \geq 4\pi(4-\pi)/0.001^2 \approx 10.8$ million darts. Reducing the interval width by a factor of ten requires a hundred times as many draws with this estimator.

Quiz 3. With an envelope twice as high, each proposal is accepted with half the probability, so the acceptance rate halves from about 22% to about 11%. The distribution of the accepted draws does not change. The acceptance probability is still proportional to h, and the constant cancels in the argument that the accepted draws have density f. With $M = h(18/25)/2$, the envelope lies *below* h near the mode, and the ratio $h(\pi)/M$ exceeds one there, so every proposal near the mode is accepted; the peak is underrepresented relative to the tails, and the accepted draws have the wrong distribution. An envelope that is too low gives wrong answers; one that is too high only wastes time.

Quiz 4. The acceptance rate is the marginal likelihood `f_y(48, n = 60, a = 7, b = 5)` $= 0.018$, so roughly 1,800 of 100,000 simulations match Joost's record, against about 8,400 for Sara's week. With more trains there are more possible values of Y, and the prior probability of any one exact value is spread more thinly; the survival rate falls roughly in proportion to n.

1.9 Exercises

1.9.1 Conceptual exercises

Exercise 1.1 (Writing summaries as expectations) For each of the following quantities, write it as $E[g(X)]$ for a suitable function g, say what distribution X has, and write one line of R that estimates it from a vector `x` of draws.

a. $P(X > 2)$ for $X \sim \text{N}(0, 1)$.
b. The variance of a Beta$(2, 5)$ variable.
c. The posterior probability that a proportion π exceeds 0.5, given draws from the posterior.
d. The posterior predictive probability that the next train is on time, given posterior draws of π. Hint: Conditional on π, the success probability is π; average this over the posterior uncertainty.

Exercise 1.2 (Seeds and sequences) A student estimates the same posterior mean in two Monte Carlo runs with identical inputs. The function `simulate_posterior(a, b, n, y)` begins with `set.seed(1)`. She averages the two estimates to reduce Monte Carlo error.

a. Explain what the seed inside the function does to the two runs.
b. Is the averaged estimate more precise than a single run? Why or why not?
c. Rewrite the workflow so that the two runs are independent but the whole analysis is still reproducible.

Exercise 1.3 (Envelopes) A rejection sampler for a target $h(x)$ uses a proposal q and a constant M.

a. Explain in one sentence why $Mq(x) \geq h(x)$ must hold for every x, and what goes wrong if it fails on some interval.
b. The sampler accepts 4% of its proposals. Someone suggests using the accepted draws anyway, since "4% of a million is still 40,000 draws". Is this a valid objection to the sampler's *correctness*, to its

efficiency, or to neither?

c. If you could choose a proposal q that is proportional to h, what would M be, and what would the acceptance rate be?

Exercise 1.4 (Forward simulation as rejection sampling) Consider a model for counts, with a Gamma prior on the expected count λ: $\lambda \sim \text{Gamma}(3, 1)$ and $Y_i \mid \lambda \sim \text{Pois}(\lambda)$ for $i = 1, \dots, 6$, with observed data $y = (2, 0, 4, 1, 3, 2)$. The Gamma parameters are shape and rate, so its density is $f(\lambda) = \lambda^2 e^{-\lambda}/2$ for $\lambda > 0$. Conditional on λ, the six counts are independent and $P(Y_i = k \mid \lambda) = e^{-\lambda}\lambda^k/k!$. In R, use `rgamma(N, shape = 3, rate = 1)` and `rpois(N, lambda)` to simulate these distributions.

a. Describe the forward simulation that produces draws from the posterior of λ.

b. Identify the proposal, the target, and the acceptance probability of the equivalent rejection sampler.

c. Write down the acceptance rate as an integral. Would it be larger or smaller if the six observations were replaced by their sum, $\sum y_i = 12$, observed as a single Poisson count with rate 6λ? Explain.

1.9.2 Practical exercises

Exercise 1.5 (Two estimators of π) The darts estimate $\pi/4$ as the proportion of uniform points under the curve $y = \sqrt{1 - x^2}$. An alternative is to draw $X \sim \text{Unif}(0, 1)$ and average $g(X) = \sqrt{1 - X^2}$, which also has expectation $\pi/4$ (Hanada & Matsuura, 2022, Section 2.2).

a. Implement both estimators with $N = 10{,}000$ and report each with its Monte Carlo standard error.

b. Repeat each estimator 500 times and compare the standard deviations of the two sets of estimates. Which estimator is more precise, and by what factor?

c. Explain the difference in terms of the variance of the two functions being averaged.

Exercise 1.6 (A distribution without a mean) This optional exercise examines why the finite-expectation condition matters. The standard Cauchy distribution has CDF $F(x) = \frac{1}{2} + \frac{1}{\pi}\arctan(x)$.

a. Derive $F^{-1}(u)$ and write a function that generates Cauchy draws from uniform draws. Check it against `dcauchy()` with a histogram, restricting the plot to $[-10, 10]$.

b. Plot the running mean of 100,000 draws against the number of draws. Compare with the running mean of 100,000 Normal draws. What do you see, and what does it say about the Law of Large Numbers?

c. The Cauchy distribution reappears in Chapter 5 as a prior. Would a Monte Carlo estimate of the *median* of your draws behave better than the mean? Try it.

Exercise 1.7 (The inhibition function) Return to the stop-signal race model of the chapter. The stop-signal delay was fixed at 250 ms. In an experiment it varies from trial to trial.

a. Write a function `p_stop(ssd, N_races = 100000)` that returns the Monte Carlo estimate of the probability of stopping for a given delay, together with its standard error.

b. Evaluate it for delays from 0 to 500 ms in steps of 50 ms and plot the probability of successful stopping. The inhibition function is often plotted instead as the complementary probability of responding, so label your vertical axis explicitly.

c. Change the mean of the Normal component of the stop duration from 200 ms to 300 ms, keeping its standard deviation and Exponential component unchanged. Redraw the curve. Describe in words what a slower stop process does to the inhibition function.

Exercise 1.8 (Rejection sampling with a better envelope) The uniform envelope in Figure 1.6 wastes nearly four of every five proposals.

a. Use a Beta$(2, 2)$ proposal instead. Find the smallest valid M numerically with `optimize()` applied to the ratio $h(\pi)/q(\pi)$, and run the sampler.
b. Report the acceptance rate and compare the histogram of the accepted draws with the Beta$(19, 8)$ density.
c. Try a Beta$(15, 6)$ proposal. What acceptance rate do you get, and why is it so much better? What would happen with a Beta$(25, 4)$ proposal, whose left tail near zero is thinner than the target's? Examine the limiting ratio as $\pi \to 0$; a numerical maximum on a finite search interval is not a proof of a valid envelope.

1.9.3 Applied exercises

Exercise 1.9 (Believing in ghosts) The `pulse_of_the_nation` data in the `bayesrules` package record, for 1,000 American adults, whether they believe in ghosts (the `ghosts` variable). Let π be the population proportion who do, with prior $\pi \sim \text{Beta}(2, 4)$.

a. Load `data("pulse_of_the_nation", package = "bayesrules")` and inspect `table(pulse_of_the_nation$ghosts, useNA = "ifany")` before coding the response. Keep respondents with a recorded yes/no answer, and document which value denotes belief. Take the first 30 eligible respondents as your data and count how many believe in ghosts. Use forward simulation with 100,000 prior draws to approximate the posterior of π. Report the number of retained draws, the posterior mean, and a 95% interval, and compare them with the exact Beta posterior.
b. Now use all eligible respondents, reporting their actual number rather than assuming there are 1,000 complete responses. How many of 100,000 forward simulations match the observed count? Use `f_y()` from the chapter to predict this number before you run the simulation.
c. Sample the posterior for the full data by rejection sampling instead, with a uniform proposal and M equal to the maximum of the posterior kernel. Report the acceptance rate. Compare the retained sample sizes and Monte Carlo standard errors for the posterior mean. Explain the difference in efficiency and what the methods have in common. A low acceptance rate does not invalidate the retained draws. Compute the kernel and acceptance comparison on the log scale to avoid underflow.

Chapter 2

Bayesian Inference, Revisited

The Museum of Modern Art (MoMA) in New York holds work by artists from many generations. Suppose you want to estimate the proportion of artists in its collection who were born in 1965 or later, classified here as Generation X or younger. The `moma_sample` data in the `bayesrules` R package contain a random sample of 100 artists and a variable recording this classification. In *Bayes Rules!*, these data illustrate inference for an unknown proportion using a Beta prior, a Binomial likelihood, and a Beta posterior available in closed form (Johnson et al., 2022, Chapter 8).

This chapter examines the calculation behind that update. Bayes' rule combines a prior and a likelihood, then divides by a normalizing constant to obtain a posterior distribution. For some models, we can identify the posterior as a familiar distribution. For others, we must approximate it. We will use both approaches to estimate the proportion, summarize its uncertainty, and compare models for the data.

For further background on Bayesian models and posterior summaries, see *Bayes Rules!* (Johnson et al., 2022, Chapters 3–5 and 8). Run the following setup before working through the examples. As in Chapter 1, n denotes the number of observed data points and N the number of Monte Carlo draws.

```
library(tidyverse)
library(patchwork)
library(bayesrules)
```

Goals

- Explain the roles of the prior, likelihood, posterior, and normalizing constant in Bayes' rule.
- Write the unnormalized posterior as a `log_posterior()` function in R, a habit that every later chapter relies on.
- Recognize when the posterior is available in closed form (conjugacy, kernel recognition) and when it is not.
- Compute posterior means, credible intervals, and interval probabilities using formulas and posterior draws.
- Define the marginal likelihood, the Bayes factor, and posterior model probabilities, and compute a Bayes factor for a point hypothesis with the Savage–Dickey density ratio.
- Implement grid approximation in one and two dimensions and explain how its computational cost grows with the number of parameters.

2.1 Prior, likelihood, posterior

Let θ denote an unknown parameter, or a vector of parameters, and let y denote the observed data. The **prior density** $f(\theta)$ describes uncertainty before observing these data. The sampling model specifies the distribution of possible data given θ. With y fixed at its observed value, its probability mass or density $f(y \mid \theta)$ is the **likelihood**, viewed as a function of θ. The **posterior density** $f(\theta \mid y)$ describes uncertainty

after observing y. Bayes' rule says that the posterior density is

$$f(\theta \mid y) = \frac{f(y \mid \theta)\, f(\theta)}{f(y)}, \qquad f(y) = \int f(y \mid \theta)\, f(\theta)\, d\theta. \tag{2.1}$$

The numerator multiplies the likelihood by the prior. It assigns relatively large values to parameter values supported by both. The symbol $\propto$ means "proportional to", so we can write

$$f(\theta \mid y) \propto f(y \mid \theta) f(\theta).$$

The right-hand side is the **unnormalized posterior**. Its integral need not be one. Dividing it by its integral, $f(y)$ in Equation 2.1, produces a density with total area one.

The data y are held fixed throughout this calculation, so $f(y)$ is a single number rather than a function of θ. It is called the **normalizing constant** or **marginal likelihood**. For discrete data, it is the prior predictive probability of observing y. For continuous data, it is a prior predictive density at y, which can exceed one.

When we evaluate the posterior at two parameter values, this constant cancels in their ratio. This is useful when the likelihood and prior can be evaluated but the integral over θ is difficult. We can then compare posterior heights without calculating the constant. Later in the chapter we will see why model comparison sometimes requires that constant, or an equivalent calculation.

2.1.1 The modern art example

Let's set up the running example. Of the 100 sampled artists, 14 are Gen X or younger:

```
moma_sample |>
  count(genx)
#>    genx  n
#> 1 FALSE 86
#> 2  TRUE 14
y <- sum(moma_sample$genx)
n <- nrow(moma_sample)
```

We use a Binomial sampling model, $Y \mid \pi \sim \text{Bin}(n, \pi)$, treating the classifications as independent observations with a common probability π. We assign π a Beta$(4, 6)$ prior, with mean $4/(4+6) = 0.4$. This prior describes uncertainty about the proportion in the collection; it does not assert that the observed sample proportion will be exactly 0.4. Combining the Binomial likelihood and Beta prior gives

$$f(\pi \mid y) \propto \binom{n}{y} \pi^y (1-\pi)^{n-y} \times \frac{\pi^3 (1-\pi)^5}{B(4,6)}.$$

The factor $\binom{n}{y}$ is the Binomial coefficient, and $B(4,6)$ normalizes the Beta prior. Both are constant with respect to π. They affect the overall scale of the numerator but not which values of π have higher posterior density. We will visualize the prior, likelihood, and posterior in Figure 2.1 after identifying the posterior distribution.

2.1.2 The log-scale habit

For numerical calculations, we write a function that returns the *logarithm* of the unnormalized posterior:

$$\log f(\pi \mid y) = \log f(y \mid \pi) + \log f(\pi) + \text{constant}.$$

On the log scale the product becomes a sum, and R's `d*` functions compute each term with `log = TRUE`:

```
log_posterior <- function(pi) {
  dbinom(y, size = n, prob = pi, log = TRUE) +   # log likelihood
```

```
    dbeta(pi, 4, 6, log = TRUE)                    # log prior
}
log_posterior(0.15)
#> [1] -2.49048
log_posterior(0.5)
#> [1] -30.31011
```

Throughout this book, `log_posterior()` returns the **unnormalized log posterior**, calculated as the log likelihood plus the log prior.

The two values returned above are negative for this particular calculation. Log densities need not always be negative, because densities can exceed one. To compare posterior heights, subtract the two log values and exponentiate:

$$\frac{f(0.15 \mid y)}{f(0.5 \mid y)} = \exp[\texttt{log_posterior}(0.15) - \texttt{log_posterior}(0.5)] \,.$$

The resulting ratio is approximately 1,200,000,000,000. It compares densities at two points; it is not a ratio of probabilities assigned to those individual points.

The log scale helps prevent numerical underflow. If we record each artist's classification separately, the joint likelihood is a product of Bernoulli probabilities. For a hypothetical sample of 10,000 artists, with 1,400 classified as Gen X or younger, this product becomes too small for the computer to represent:

```
y_big <- rep(c(1, 0), times = c(1400, 8600))          # 10,000 artists, 14% Gen X
prod(dbinom(y_big, size = 1, prob = 0.2))              # raw scale
#> [1] 0
sum(dbinom(y_big, size = 1, prob = 0.2, log = TRUE))   # log scale
#> [1] -4172.248
```

On the raw scale the likelihood underflows to exactly zero, so a ratio of two such rounded likelihoods could become `0/0`. On the log scale it is a sum of ten thousand moderate numbers, and the result remains representable. The same construction applies to other models. Add the log likelihood terms and the log prior terms, leaving out the posterior normalizing constant.

Bayes' rule, computationally

The posterior is $f(\theta \mid y) = f(y \mid \theta) f(\theta) / f(y)$. In the models studied here, we can evaluate the likelihood and prior at a specified parameter value. Integrating their product may be much harder.

- `log_posterior(theta)` returns the log likelihood plus the log prior.
- Differences between its values give log ratios of posterior densities.
- Its integral on the original scale equals $f(y)$ **only when the full likelihood and normalized prior have been retained**. Dropping additional constants preserves the posterior's shape but changes that integral.

Posterior sampling can use relative heights; model comparison requires careful treatment of constants.

Quiz yourself!

Two students calculate `log_posterior()` for the modern art data. One uses `dbinom(y, n, pi, log = TRUE)`, which includes the binomial coefficient $\log \binom{n}{y}$; the other writes the Bernoulli likelihood as `y * log(pi) + (n - y) * log(1 - pi)`, which does not. Their functions return different numbers. Do they describe different posteriors?

2.2 Conjugate families and kernel recognition

A prior family is **conjugate** to a sampling model when updating with that model leaves the posterior in the same family. This lets us identify a normalized posterior without numerically integrating it.

To see this, collect the terms in the unnormalized log posterior that depend on the parameter. An additive term independent of the parameter can be represented by C. Matching the remaining expression to the log density of a known family is called **kernel recognition**.

For the modern art data, the log likelihood and log prior combine as

$$\begin{aligned}\log f(\pi \mid y) &= y \log \pi + (n-y)\log(1-\pi) + 3\log\pi + 5\log(1-\pi) + C \\ &= (y+4-1)\log\pi + (n-y+6-1)\log(1-\pi) + C.\end{aligned}$$

The Beta(a,b) log density has the form $(a-1)\log\pi + (b-1)\log(1-\pi) - \log B(a,b)$. Comparing coefficients gives

$$\pi \mid y \sim \text{Beta}(y+4, n-y+6) = \text{Beta}(18, 92).$$

Thus the normalized posterior is available through R's Beta functions. The Beta function $B(y+4, n-y+6)$ normalizes the kernel $\pi^{y+3}(1-\pi)^{n-y+5}$. The marginal likelihood of the original model also includes the constants from the likelihood and prior; we calculate it in the section on model comparison.

The three conjugate pairs in Table 2.1 can each be obtained by collecting terms in this way. The Gamma distribution uses shape and **rate** parameters; the second parameter of a Normal distribution in mathematical notation is its **variance**. In the Normal row, θ and τ^2 are the prior mean and variance, $\bar{y}$ is the sample mean, and σ^2 is the known sampling variance. R's `dnorm()` uses a standard deviation for its `sd` argument. Exercise 2.2 develops the Poisson and Normal cases.

Table 2.1: Three conjugate models. For repeated observations in the Poisson and Normal rows, observations are independent conditional on their parameter. The posterior parameters follow by adding the log likelihood and log prior and matching the resulting kernel.

Likelihood	Prior	Posterior
$Y \mid \pi \sim \text{Bin}(n, \pi)$	$\pi \sim \text{Beta}(\alpha, \beta)$	$\text{Beta}(\alpha + y,\ \beta + n - y)$
$Y_i \mid \lambda \sim \text{Pois}(\lambda),\ i = 1, \ldots, n$	$\lambda \sim \text{Gamma}(s, r)$	$\text{Gamma}(s + \sum_i y_i,\ r + n)$
$Y_i \mid \mu \sim \text{N}(\mu, \sigma^2)$, σ known	$\mu \sim \text{N}(\theta, \tau^2)$	$\text{N}\left(\dfrac{\theta/\tau^2 + n\bar{y}/\sigma^2}{1/\tau^2 + n/\sigma^2},\ \dfrac{1}{1/\tau^2 + n/\sigma^2}\right)$

Appendix D works through each derivation step by step, including the completion of the square in the Normal case, and illustrates how the posterior parameters combine prior information with the data.

Figure 2.1 displays the Beta update. The code evaluates each curve on the same grid. To compare their shapes, the likelihood is rescaled to have area one as a function of π; this plotting choice does not turn the likelihood into a prior or a posterior. The posterior combines the likelihood with the prior and is shifted slightly toward the prior mean.

```
pi_plot <- seq(0, 1, length.out = 501)
delta <- pi_plot[2] - pi_plot[1]
lik <- dbinom(y, n, pi_plot)

tibble(pi = pi_plot,
       prior = dbeta(pi_plot, 4, 6),
       likelihood = lik / sum(lik * delta),
       posterior = dbeta(pi_plot, y + 4, n - y + 6)) |>
  pivot_longer(-pi, names_to = "curve", values_to = "density") |>
  mutate(curve = factor(curve, levels = c("prior", "likelihood", "posterior"))) |>
```

```r
ggplot(aes(pi, density, color = curve, linetype = curve)) +
geom_line(linewidth = 0.9) +
scale_color_manual(values = c(prior = "gray55", likelihood = "gray30", posterior =
  "#552E6B"), name = NULL) +
scale_linetype_manual(values = c(prior = "dashed", likelihood = "dotted", posterior
  = "solid"), name = NULL) +
labs(x = expression(pi), y = "Density")
```

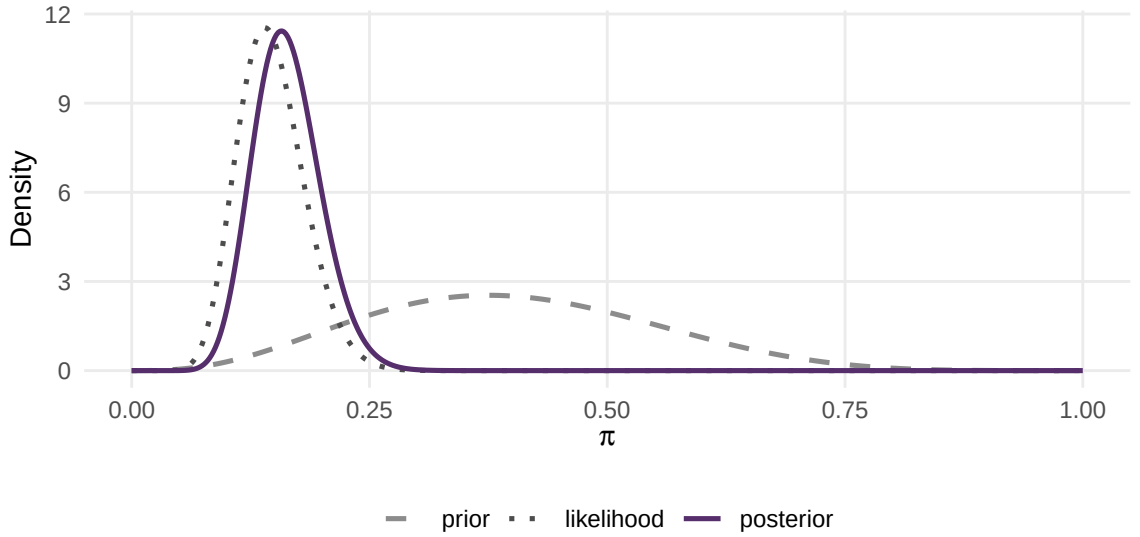


Figure 2.1: The Beta(4, 6) prior, the Binomial likelihood of 14 Gen X artists out of 100 (scaled to integrate to one), and the Beta(18, 92) posterior for the proportion π of Gen X or younger artists.

Kernel recognition

To identify a conjugate posterior:

1. Add the log likelihood and log prior.
2. Collect terms involving the parameter and group all other terms into C.
3. Match the result to a known log density and identify its parameters, as in the Beta derivation and Table 2.1.

If no familiar kernel is apparent, this recognition method has not supplied a closed-form posterior. Numerical integration or sampling may still be possible.

A different prior can produce a posterior outside the conjugate family. Suppose we place a Normal prior on the **log-odds**,

$$z = \log \frac{\pi}{1-\pi}, \qquad z \sim \mathrm{N}(-1.5, 1^2).$$

The odds $\pi/(1-\pi)$ compare success with failure. For $\pi = 0.2$, the odds are $0.2/0.8 = 0.25$ and the log-odds are $\log(0.25) \approx -1.39$. The log-odds transformation takes a probability in $(0, 1)$ to a value anywhere on the real line. Its inverse is $\pi = 1/(1 + e^{-z})$, implemented by `plogis()` in R; `qlogis()` maps in the other direction. The resulting distribution of π is called a logit-Normal distribution.

To express this prior as a density for π, we must account for how intervals change under the transformation. A short interval of width $d\pi$ corresponds to a log-odds interval of width approximately $|dz/d\pi|d\pi$. Equal probabilities in these corresponding intervals require

$$f_\pi(\pi) = f_z\Big(\log \frac{\pi}{1-\pi}\Big) \left|\frac{dz}{d\pi}\right|, \qquad \frac{dz}{d\pi} = \frac{1}{\pi(1-\pi)}.$$

The derivative factor is the **Jacobian** for this one-dimensional transformation. On the log scale, it contributes $-\log\pi - \log(1-\pi)$. The following function includes it and returns $-\infty$ outside $(0, 1)$, corresponding to zero posterior density there.

```
log_posterior_logitnormal <- function(pi) {
  lp <- rep(-Inf, length(pi))
  inside <- pi > 0 & pi < 1
  p <- pi[inside]
  lp[inside] <- dbinom(y, size = n, prob = p, log = TRUE) +
    dnorm(qlogis(p), mean = -1.5, sd = 1, log = TRUE) -
    log(p) - log1p(-p)
  lp
}
```

The function evaluates a log posterior even though the result does not have the Beta log-density form. In the grid section, we use these evaluations to approximate its normalized posterior. Chapter 6 (Metropolis–Hastings and Adaptive Proposals) returns to transformations in more detail.

2.3 Summarizing a posterior

A posterior distribution supports several kinds of summary. Means and interval probabilities are expectations, as in Chapter 1. Quantiles locate values below which specified amounts of posterior probability lie. The mode identifies the highest point of the density.

Table 2.2: Posterior summaries. Means and interval probabilities can be estimated by averaging functions of the draws. Credible interval endpoints are estimated by sample quantiles.

Summary	Definition	From posterior draws in `theta`
Mean	$E[\theta \mid y] = \int \theta f(\theta \mid y)\, d\theta$	`mean(theta)`
Variance	$E[(\theta - E[\theta \mid y])^2 \mid y]$	`var(theta)`
Interval probability	$P(a < \theta < b \mid y) = E[\mathbb{1}\{a < \theta < b\} \mid y]$	`mean(theta > a & theta < b)`
Central 95% credible interval	2.5th and 97.5th posterior percentiles	`quantile(theta, c(.025, .975))`
Mode	A value maximizing $f(\theta \mid y)$	Optimize a density or log density

The first four rows of Table 2.2 specify the mean, variance, interval probability, and central interval calculations used below. For the modern art model, we can compute these summaries directly from the Beta posterior and compare them with estimates from independent draws. The interval in the code below contains 95% of the posterior probability for π; it describes uncertainty about the population proportion.

```
a <- y + 4
b <- n - y + 6

exact <- c(mean = a / (a + b),
           sd = sqrt(a * b / ((a + b)^2 * (a + b + 1))),
           lower = qbeta(0.025, a, b),
           upper = qbeta(0.975, a, b),
           p_below_0.2 = pbeta(0.2, a, b))
exact
#>        mean          sd       lower       upper p_below_0.2
#>  0.16363636  0.03511365  0.10090844  0.23792856  0.84898556
```

Now estimate the same summaries from $N = 10{,}000$ independent posterior draws. R's `rbeta()` generates them directly because the posterior is a Beta distribution.

```
set.seed(84735)
N <- 10000
draws <- tibble(pi = rbeta(N, a, b))

draws |>
  summarize(mean = mean(pi), sd = sd(pi),
            lower = quantile(pi, 0.025), upper = quantile(pi, 0.975),
            p_below_0.2 = mean(pi < 0.2))
#> # A tibble: 1 x 5
#>    mean     sd lower upper p_below_0.2
#>   <dbl>  <dbl> <dbl> <dbl>       <dbl>
#> 1 0.164 0.0353 0.100 0.239       0.845
```

The simulated summaries are close to the exact values, but they vary from run to run. The Monte Carlo standard error of the mean is estimated by `sd(draws$pi) / sqrt(N)`. For the estimated probability $\hat{p} =$ `mean(draws$pi < 0.2)`, use $\sqrt{\hat{p}(1-\hat{p})/N}$:

```
p_hat <- mean(draws$pi < 0.2)
c(mean_mcse = sd(draws$pi) / sqrt(N),
  probability_mcse = sqrt(p_hat * (1 - p_hat) / N))
#>        mean_mcse probability_mcse
#>     0.0003532832     0.0036161846
```

These errors concern the computation of posterior summaries. Increasing N reduces them without making the underlying posterior distribution narrower. The $1/\sqrt{N}$ standard-error formula applies to independent averages with finite variance. Quantiles have their own sampling errors; their standard errors are not obtained by substituting a quantile into the formula for a mean.

2.3.1 Credible intervals and the mode

A **central 95% credible interval** leaves 2.5% of the posterior probability in each tail. Another choice is the **highest posterior density** (HPD) region, which contains 95% probability using values of highest density. For a continuous, unimodal density such as this Beta posterior, the HPD region is a shortest 95% interval. For a multimodal density, an HPD region can consist of separate intervals.

For the unimodal case, we can approximate a shortest interval from the draws. Sort them, consider every consecutive group containing at least 95% of the draws, and retain the narrowest group. The function below uses `ceiling()` to determine the required number of draws. Because it searches a finite sample, the result has Monte Carlo error.

```
hpd <- function(x, level = 0.95) {
  x <- sort(x)
  n_draws <- length(x)
  m <- ceiling(level * n_draws)
  stopifnot(level > 0, level < 1, m >= 2, m <= n_draws)
  starts <- seq_len(n_draws - m + 1)
  ends <- starts + m - 1
  i <- which.min(x[ends] - x[starts])
  c(lower = x[starts[i]], upper = x[ends[i]])
}
central_interval <- setNames(quantile(draws$pi, c(0.025, 0.975)),
                             c("lower", "upper"))
```

```
hpd_interval <- hpd(draws$pi)
rbind(central = central_interval, shortest = hpd_interval)
#>                lower     upper
#> central  0.10005397 0.2386363
#> shortest 0.09721451 0.2337647
```

To display the population intervals without sampling noise, the next figure obtains the shortest interval directly from the Beta quantile function. If q is the probability below its lower endpoint, then the upper endpoint has cumulative probability $q + 0.95$. We choose q between zero and 0.05 to minimize their distance.

```
q_lower <- optimize(function(q) qbeta(q + 0.95, a, b) - qbeta(q, a, b),
                    interval = c(0, 0.05))$minimum
intervals <- tibble(
  interval = c("Central: Equal tail probabilities", "HPD: Shortest interval"),
  lower = c(qbeta(0.025, a, b), qbeta(q_lower, a, b)),
  upper = c(qbeta(0.975, a, b), qbeta(q_lower + 0.95, a, b))
)
intervals |> mutate(width = upper - lower)
#> # A tibble: 2 x 4
#>   interval                           lower upper width
#>   <chr>                              <dbl> <dbl> <dbl>
#> 1 Central: Equal tail probabilities 0.101  0.238 0.137
#> 2 HPD: Shortest interval            0.0973 0.233 0.136
```

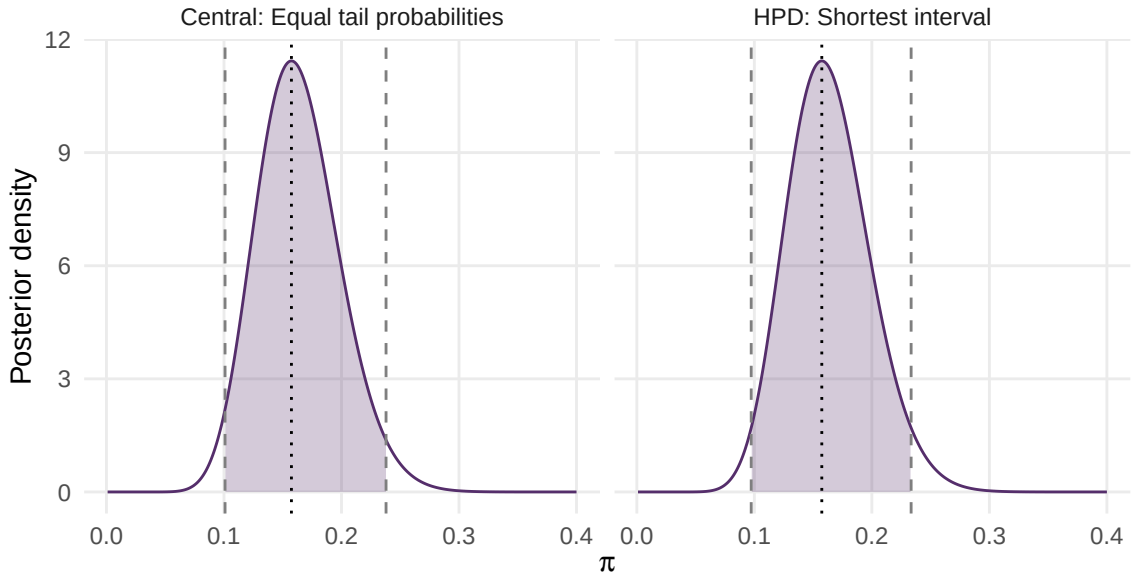


Figure 2.2: Two 95% credible intervals for the same Beta(18,92) posterior. Shading marks the probability inside each interval. The central interval leaves equal tail probabilities; the HPD interval is slightly shorter and leaves unequal tails. The dotted vertical line marks the posterior mode, a point rather than an interval.

Both intervals in Figure 2.2 describe uncertainty about the same proportion. They differ because they use different rules to allocate the 5% probability left outside. The **posterior mode**, also called the maximum a posteriori estimate, is the value at the peak. It is a maximization, rather than an expectation or a quantile. For this model, it can be found directly:

```r
mode_pi <- optimize(log_posterior, interval = c(0.001, 0.999), maximum = TRUE)$maximum
c(mode = mode_pi, mean = a / (a + b))
#>      mode      mean
#> 0.1574135 0.1636364
```

For a continuous posterior, repeated draws almost never have identical values. Counting the most frequent draw therefore does not provide a useful way to estimate the mode.

Quiz yourself!

The museum's benchmark is a Gen X share between 0.15 and 0.25. Write $P(0.15 < \pi < 0.25 \mid y)$ as an expectation. How would you estimate it from the 10,000 draws in `draws`, and roughly how large is the Monte Carlo error of that estimate?

2.4 Comparing models: marginal likelihoods and Bayes factors

So far we have calculated probabilities for ranges of π within one model, such as $\pi < 0.2$ or $0.15 < \pi < 0.25$. For illustration, suppose the museum's board asks whether a model fixing the proportion at exactly 20% predicts the data better than a model allowing the proportion to vary. Under our continuous Beta posterior, $P(\pi = 0.2 \mid y) = 0$, just as it is for every other single value. Assessing a model that assigns all its parameter probability to 0.2 therefore requires a separate model comparison.

Interval and point hypotheses

An **interval hypothesis** says that the parameter lies in a range, such as $\pi < 0.2$ or $0.15 < \pi < 0.25$. Its posterior probability is an expectation and is computed as in the previous section. A **point hypothesis** specifies a single value, such as $\pi = 0.2$. A separate point-null model assigns its parameter probability to that value; its posterior model probability comes from comparing it with an alternative model.

We compare two models for the data: H_0, in which π is fixed at 0.2, and H_1, in which π is unknown and has the Beta$(4, 6)$ prior. Each model makes a prediction about the data, and the prediction is its **marginal likelihood**, the very normalizing constant we have been avoiding:

$$f(y \mid H_0) = f(y \mid \pi = 0.2), \qquad f(y \mid H_1) = \int_0^1 f(y \mid \pi)\, f(\pi \mid H_1)\, d\pi.$$

Under H_0 there is nothing to integrate, because π is fixed. Under H_1, the marginal likelihood averages the likelihood over the prior. It is the prior predictive probability of the observed count, allowing for the uncertainty about π specified by that model. The **Bayes factor** is the ratio of the two,

$$\mathrm{BF}_{01} = \frac{f(y \mid H_0)}{f(y \mid H_1)}, \tag{2.2}$$

where the subscript 01 specifies the direction of comparison. $\mathrm{BF}_{01} > 1$ favors H_0 relative to H_1; $\mathrm{BF}_{01} < 1$ favors H_1. The reverse comparison is $\mathrm{BF}_{10} = 1/\mathrm{BF}_{01}$.

The Bayes factor also describes how the data update **model odds**. Odds compare probabilities by division; for example, probabilities 0.75 and 0.25 give odds of $3 : 1$. Applying Bayes' rule to the models gives

$$\frac{P(H_0 \mid y)}{P(H_1 \mid y)} = \frac{f(y \mid H_0)}{f(y \mid H_1)} \times \frac{P(H_0)}{P(H_1)}, \tag{2.3}$$

Thus posterior odds equal prior odds multiplied by the Bayes factor. If the prior odds are $1 : 1$ and $\mathrm{BF}_{01} = 4$, the posterior odds are $4 : 1$, giving $P(H_0 \mid y) = 4/(4+1) = 0.8$. With prior odds $1 : 4$, the same Bayes factor instead gives posterior odds $1 : 1$ and $P(H_0 \mid y) = 0.5$.

For two models with prior model probabilities that sum to one, write $O_{01} = P(H_0)/P(H_1)$. Then

$$P(H_0 \mid y) = \frac{\mathrm{BF}_{01} O_{01}}{1 + \mathrm{BF}_{01} O_{01}}.$$

These prior probabilities concern which model is considered plausible; they are distinct from the Beta prior on π *within* H_1. Chapter 8 (Bayesian Variable Selection) and Chapter 9 (Bayesian Model Averaging) extend this reasoning to larger sets of models.

For the Beta-Binomial model both marginal likelihoods are available in closed form. Under H_1 the integral is the same Beta-function integral that kernel recognition handled for us,

$$f(y \mid H_1) = \binom{n}{y} \frac{B(y + \alpha,\ n - y + \beta)}{B(\alpha, \beta)},$$

so we can compute the Bayes factor directly. We also compute the integral numerically with `integrate()`, to make the phrase "average the likelihood over the prior" concrete.

```
pi0 <- 0.2
ml_H0 <- dbinom(y, size = n, prob = pi0)
log_ml_H0 <- dbinom(y, size = n, prob = pi0, log = TRUE)
log_ml_H1 <- lchoose(n, y) + lbeta(y + 4, n - y + 6) - lbeta(4, 6)
ml_H1 <- exp(log_ml_H1)
ml_H1_numeric <- integrate(function(pi) dbinom(y, n, pi) * dbeta(pi, 4, 6), 0,
  1)$value

c(ml_H0 = ml_H0, ml_H1 = ml_H1, ml_H1_numeric = ml_H1_numeric)
#>         ml_H0         ml_H1 ml_H1_numeric
#>   0.033531484   0.007417289   0.007417289
BF_01 <- exp(log_ml_H0 - log_ml_H1)
BF_01
#> [1] 4.520719
```

The data are about 4.5 times more likely under H_0 than under H_1. With equal prior model probabilities, the posterior probability of the fixed-proportion model is $P(H_0 \mid y) = 0.82$.

Interpreting a Bayes factor

A Bayes factor is a continuous measure of evidence and its interpretation depends on the context. The ranges below provide a conventional descriptive vocabulary. Values close to a boundary, such as 2.9 and 3.1, represent similar evidence and should not lead to sharply different conclusions. Report the numerical Bayes factor, the compared models, and their priors.

BF_{10}	Evidence
> 10	strong for H_1
3 to 10	moderate for H_1
1 to 3	anecdotal for H_1
1	none
1/3 to 1	anecdotal for H_0
1/10 to 1/3	moderate for H_0
$< 1/10$	strong for H_0

Note that $\mathrm{BF}_{01} = 1/\mathrm{BF}_{10}$; a BF_{01} of 4.5 is a BF_{10} of about 0.22.

The categories follow Jeffreys (1961) as tabulated by Lee & Wagenmakers (2013).

2.4.1 The Savage–Dickey density ratio

In the modern art example, conjugacy makes both marginal likelihoods available. For more complicated models, integrating the likelihood over the prior can be difficult. In certain nested point-null comparisons, the **Savage–Dickey density ratio** instead calculates the Bayes factor from prior and posterior densities under the alternative. It requires matching sampling models at the null value and compatible priors, as specified in the box below (Dickey & Lientz, 1970; Wagenmakers et al., 2010).

For the one-parameter comparison here, the identity follows directly from Bayes' rule. Under H_1, Bayes' rule evaluated at the null value π_0 reads

$$f(\pi_0 \mid y, H_1) = \frac{f(y \mid \pi_0)\, f(\pi_0 \mid H_1)}{f(y \mid H_1)}.$$

Because H_0 is nested in H_1, the likelihood at π_0 under H_1 is the likelihood under H_0: $f(y \mid \pi_0) = f(y \mid H_0)$. Rearranging,

$$\mathrm{BF}_{01} = \frac{f(y \mid H_0)}{f(y \mid H_1)} = \frac{f(\pi_0 \mid y, H_1)}{f(\pi_0 \mid H_1)}. \tag{2.4}$$

The Bayes factor for the point hypothesis is the **posterior density at the null value divided by the prior density at the null value**, both under H_1. The ratio avoids separately computing the two marginal likelihoods. If the data make π_0 more plausible than it was, the posterior density there exceeds the prior density and the Bayes factor favors H_0; if the data pull the posterior away from π_0, the Bayes factor favors H_1.

```
prior_at_pi0 <- dbeta(pi0, 4, 6)
posterior_at_pi0 <- dbeta(pi0, y + 4, n - y + 6)
posterior_at_pi0 / prior_at_pi0
#> [1] 4.520719
```

The ratio agrees with the marginal-likelihood calculation. Figure 2.3 shows the two heights being compared. Both are densities under H_1; neither is the probability that π is exactly 0.2.

The Savage–Dickey density ratio

For the one-parameter comparison above, H_0 fixes $\theta = \theta_0$ and uses the same sampling model as H_1 at that value. With a proper prior under H_1 whose density is continuous and positive at θ_0,

$$\mathrm{BF}_{01} = \frac{f(\theta_0 \mid y, H_1)}{f(\theta_0 \mid H_1)}.$$

The Bayes factor is the ratio of posterior to prior density at the null value, as in Equation 2.4. If the models also contain other parameters, their prior under H_0 must match the conditional prior under H_1 given $\theta = \theta_0$. The ratio cannot be used for an arbitrary pair of nested models without checking this condition (Wagenmakers et al., 2010).

2.4.2 The prior matters

The alternative model includes a prior, so changing that prior changes its predictions and can change the Bayes factor. Compare the original $\mathrm{Beta}(4, 6)$ prior with $\mathrm{Beta}(0.5, 0.5)$, which places substantial mass near zero and one, and $\mathrm{Beta}(20, 80)$, which concentrates around 0.2. Each comparison uses the same point-null model and the same data.

```
priors <- tribble(
  ~prior,            ~alpha, ~beta,
  "Beta(0.5, 0.5)",     0.5,   0.5,
  "Beta(4, 6)",           4,     6,
```

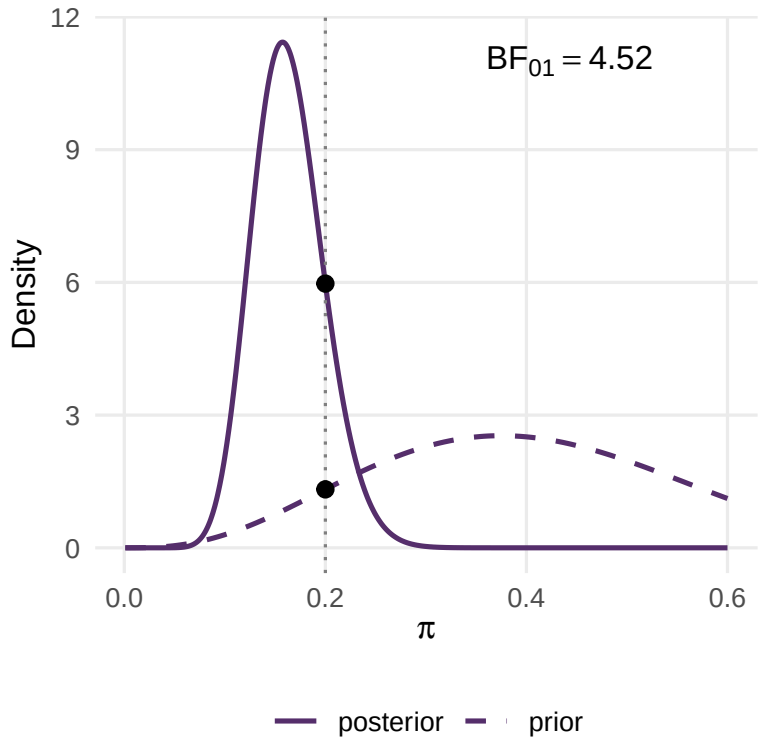


Figure 2.3: Savage–Dickey for the modern art data. At the null value 0.2 (vertical dotted line), the posterior height is divided by the prior height. The resulting BF_{01} is printed on the plot. The calculation compares two model predictions; it does not assign point probability within the continuous posterior.

```
  "Beta(20, 80)",        20,    80
)

sd_table <- priors |>
  mutate(prior_density = dbeta(pi0, alpha, beta),
         posterior_density = dbeta(pi0, alpha + y, beta + n - y),
         BF_01 = exp(dbeta(pi0, alpha + y, beta + n - y, log = TRUE) -
                     dbeta(pi0, alpha, beta, log = TRUE)))
sd_table
#> # A tibble: 3 x 6
#>   prior          alpha  beta prior_density posterior_density BF_01
#>   <chr>          <dbl> <dbl>         <dbl>             <dbl> <dbl>
#> 1 Beta(0.5, 0.5)   0.5   0.5         0.796              2.94 3.69
#> 2 Beta(4, 6)       4     6           1.32               5.97 4.52
#> 3 Beta(20, 80)    20    80           9.93               7.36 0.741
```

```
panels <- sd_table |>
  mutate(panel = paste0("'", prior, "'~~BF[\"01\"] == ", round(BF_01, 2)),
         panel = factor(panel, levels = panel))

curves <- panels |>
  select(panel, alpha, beta) |>
  crossing(pi = seq(0.001, 0.999, length.out = 400)) |>
  mutate(prior = dbeta(pi, alpha, beta),
         posterior = dbeta(pi, alpha + y, beta + n - y)) |>
```

```
  pivot_longer(c(prior, posterior), names_to = "curve", values_to = "density")

dots <- panels |>
  select(panel, prior_density, posterior_density) |>
  pivot_longer(c(prior_density, posterior_density), names_to = "curve", values_to =
    "density") |>
  mutate(curve = if_else(curve == "prior_density", "prior", "posterior"))

ggplot(curves, aes(pi, density, linetype = curve)) +
  geom_line(color = "#552E6B") +
  geom_point(data = dots, aes(x = pi0, y = density), size = 2) +
  facet_wrap(~ panel, labeller = label_parsed) +
  coord_cartesian(xlim = c(0, 0.6), ylim = c(0, 16)) +
  scale_linetype_manual(values = c(prior = "dashed", posterior = "solid"), name =
    NULL) +
  labs(x = expression(pi), y = "Density")
```

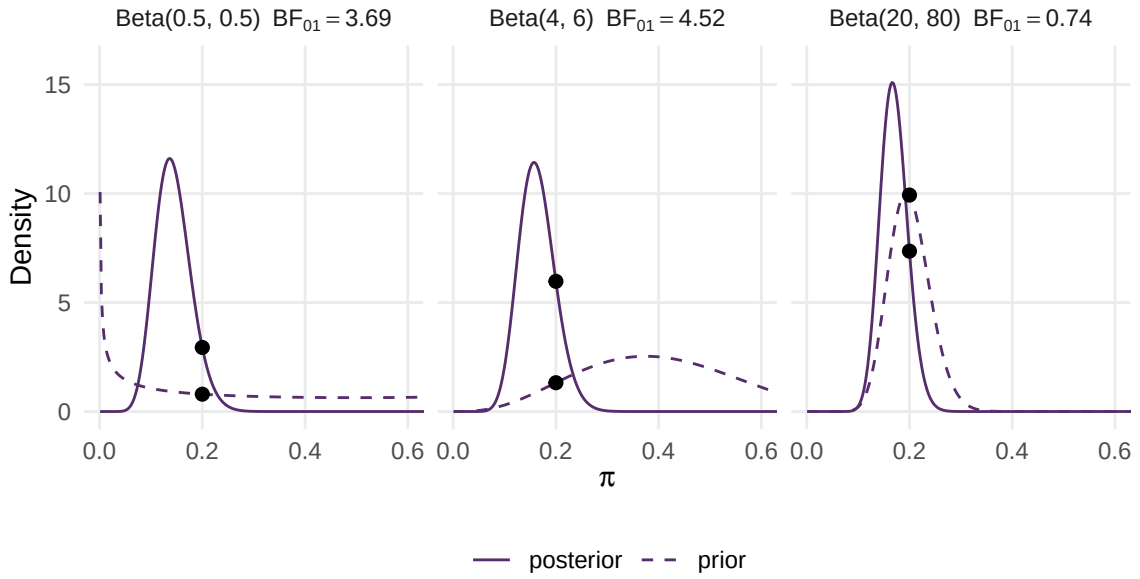


Figure 2.4: Prior (dashed) and posterior (solid) densities for π under three priors. The Savage–Dickey Bayes factor for $\pi = 0.2$ is the ratio of the heights of the two dots. Under the diffuse prior the data raise the density at 0.2 a lot; under the concentrated prior they lower it slightly.

With Beta$(0.5, 0.5)$ under H_1, $\mathrm{BF}_{01} = 3.69$; with Beta$(20, 80)$, $\mathrm{BF}_{01} = 0.74$. The observed sample proportion is 0.14. A model concentrated around 0.2 predicts these data differently from one spreading its mass across much of $[0, 1]$. The values in Figure 2.4 therefore compare the fixed-proportion model with three distinct alternatives. Report the prior under H_1 with the Bayes factor, and examine whether conclusions depend strongly on plausible changes to that prior.

2.4.3 Savage–Dickey from draws

The examples so far evaluate a Beta posterior density directly. When we have posterior draws instead, we can estimate that density at the null value and divide by the known prior density. This adds a density-estimation step. The following calculation illustrates it for a case where the exact answer is available; it is an optional numerical extension of the density-ratio identity.

```r
set.seed(84735)
post_draws <- rbeta(1e5, y + 4, n - y + 6)
dens <- density(post_draws)
posterior_at_pi0_hat <- approx(dens$x, dens$y, xout = pi0)$y
posterior_at_pi0_hat / dbeta(pi0, 4, 6)
#> [1] 4.59112
```

Compare the estimate with the exact value 4.52. Unlike averaging an indicator, a kernel density estimate depends on a smoothing bandwidth. Estimating a density in a sparsely sampled tail can be unstable, and more draws do not remove the need to assess the smoothing choice. Use this calculation as a numerical check rather than assuming that any density estimate yields an accurate Bayes factor.

Quiz yourself!

A colleague reads that $\text{BF}_{01} = 4.5$ for the board's target of $\pi = 0.2$ and concludes, "The posterior probability that $\pi = 0.2$ is $4.5/5.5 \approx 0.82$." What is right about this and what is wrong?

2.5 Grid approximation and its limits

For the logit-Normal prior, we can evaluate the unnormalized log posterior but have not identified a familiar posterior family. A **grid approximation** divides a bounded parameter range into small cells and represents each cell by its midpoint. Multiplying the density at a midpoint by the cell width approximates the probability in that cell. We can normalize those approximate masses and then calculate summaries by summation.

This is a numerical approximation to a continuous distribution, rather than a claim that the parameter can take only the grid values. The grid must be fine enough to resolve the density and wide enough to include nearly all posterior probability.

Grid approximation

To approximate a one-dimensional posterior on a bounded interval:

1. **Grid.** Divide the interval into G equal cells of width Δ and use their midpoints $\theta_1, \dots, \theta_G$.
2. **Evaluate.** For each grid index $j = 1, \dots, G$, compute $\ell_j = \log f(y \mid \theta_j) + \log f(\theta_j)$, using `log_posterior(theta_j)`.
3. **Normalize.** Let $c = \max_{j=1,\dots,G} \ell_j$, set $w_j = \exp(\ell_j - c)$, and calculate $p_j = w_j / \sum_{k=1}^{G} w_k$.
4. **Summarize.** Approximate $E[g(\theta) \mid y]$ by $\sum_{j=1}^{G} g(\theta_j) p_j$, or sample midpoints with probabilities p_j.

Here j and k index grid cells, and g is the function whose expectation we want. The weight p_j approximates the posterior probability in cell j; p_j/Δ approximates the density at its midpoint. Equal cell widths cancel when the weights are normalized. With unequal widths, each unnormalized weight must include its cell width.

Subtracting c before exponentiating changes all weights by the same factor, which cancels on normalization:

$$\frac{e^{\ell_j - c}}{\sum_k e^{\ell_k - c}} = \frac{e^{-c} e^{\ell_j}}{e^{-c} \sum_k e^{\ell_k}} = \frac{e^{\ell_j}}{\sum_k e^{\ell_k}}.$$

For example, the following raw exponentials all round to zero. After subtracting the maximum, the relative weights remain representable:

```r
lp_demo <- c(-1001, -1000, -1002)
exp(lp_demo)
```

```
#> [1] 0 0 0
w_demo <- exp(lp_demo - max(lp_demo))
cbind(log_value = lp_demo, shifted_weight = w_demo,
      probability = w_demo / sum(w_demo))
#>      log_value shifted_weight probability
#> [1,]     -1001      0.3678794  0.24472847
#> [2,]     -1000      1.0000000  0.66524096
#> [3,]     -1002      0.1353353  0.09003057
```

At least one shifted weight is exactly one. Very small weights can still underflow, but they no longer cause *every* weight to become zero.

Here is the recipe as a function, followed by a check on the Beta prior, where we know the answer, and then the logit-Normal prior, where we do not.

```
grid_posterior <- function(log_posterior, grid, ...) {
  lp <- log_posterior(grid, ...)
  w <- exp(lp - max(lp))
  tibble(theta = grid,
         prob = w / sum(w),
         density = prob / (grid[2] - grid[1]))
}

G <- 200
pi_grid <- (seq_len(G) - 0.5) / G  # midpoints of G cells covering [0, 1]
grid_beta  <- grid_posterior(log_posterior, pi_grid)
grid_logit <- grid_posterior(log_posterior_logitnormal, pi_grid)
```

```
p_beta <- ggplot(grid_beta, aes(theta, density)) +
  stat_function(fun = dbeta, args = list(shape1 = y + 4, shape2 = n - y + 6),
                color = "#552E6B", linewidth = 1) +
  geom_point(size = 0.7) +
  coord_cartesian(xlim = c(0, 0.5)) +
  labs(x = expression(pi), y = "Density", title = "Beta(4, 6) prior")

p_logit <- ggplot(grid_logit, aes(theta, density)) +
  geom_point(size = 0.7) +
  coord_cartesian(xlim = c(0, 0.5)) +
  labs(x = expression(pi), y = "Density", title = "Normal prior on the log-odds")

patchwork::wrap_plots(p_beta, p_logit)
```

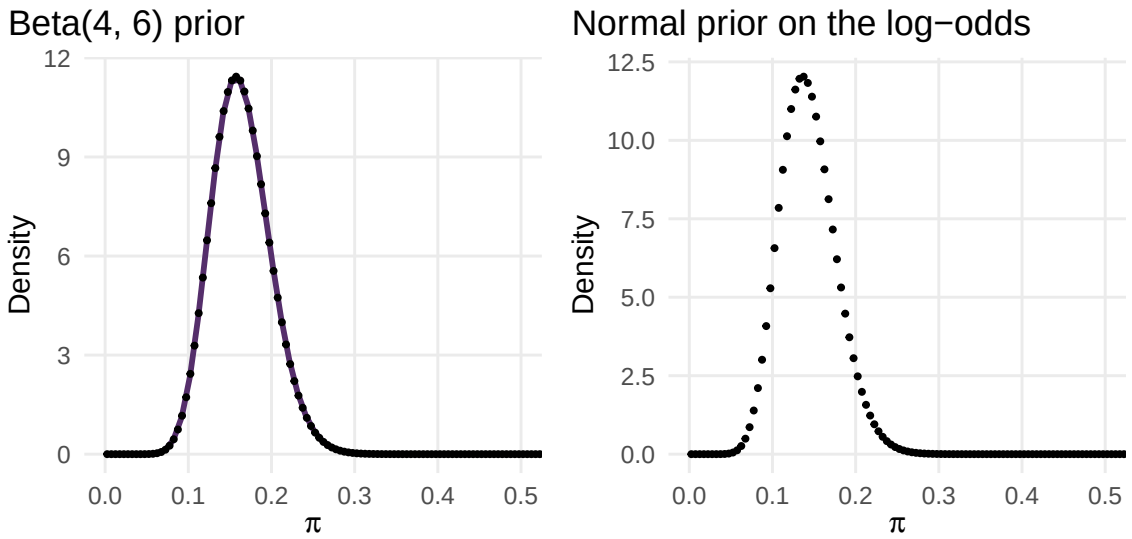


Figure 2.5: Grid approximation with 200 points. Left: for the Beta(4, 6) prior the grid (dots) reproduces the exact Beta(18, 92) posterior (line). Right: the grid approximates the posterior under the Normal prior on the log-odds. Its accuracy must be assessed by refining the grid.

Posterior summaries follow from step 4 of the box. Here are the posterior mean and the probability that $\pi < 0.2$ under both priors:

```
bind_rows(`Beta(4, 6)` = grid_beta, `logit-Normal` = grid_logit, .id = "prior") |>
  group_by(prior) |>
  summarize(mean = sum(theta * prob),
            p_below_0.2 = sum(prob[theta < 0.2]))
#> # A tibble: 2 x 3
#>   prior         mean p_below_0.2
#>   <chr>        <dbl>       <dbl>
#> 1 Beta(4, 6)   0.164       0.849
#> 2 logit-Normal 0.143       0.945
```

The Beta row approximates the exact posterior mean of 0.164 with 200 evaluations of `log_posterior()`. Its probability estimate differs from the exact 0.849 by 1.8×10^{-4}. These are finite-cell approximations to integrals. Increasing the grid resolution provides a check on their accuracy. If a probability cutoff falls inside a cell, assigning that entire cell to one side introduces an additional approximation. When `log_posterior()` retains the full likelihood and normalized prior, the grid also approximates the marginal likelihood. On the log scale, the approximation is $\log f(y) \approx c + \log(\Delta \sum_j w_j)$. Exercise 2.7 asks you to add this calculation. If the grid omits non-negligible posterior tails, normalizing over its cells instead approximates a truncated posterior.

2.5.1 Two parameters

Grid approximation extends to two parameters by gridding each and evaluating the log posterior at every combination. Consider a further question about the modern art data. Does the Gen X proportion differ between artists recorded as female and male? Restricting to the 99 artists who are recorded as female or male, we model $Y_i \mid \pi_i \sim \text{Bern}(\pi_i)$ with

$$\log \frac{\pi_i}{1-\pi_i} = \beta_0 + \beta_1 \cdot \text{female}_i, \qquad \beta_0 \sim \text{N}(0, 2^2), \quad \beta_1 \sim \text{N}(0, 1^2).$$

Here i indexes artists, $Y_i = 1$ denotes Gen X or younger, and $Y_i = 0$ denotes an older generation. The Bernoulli distribution describes one such binary observation. The predictor female_i equals one for an artist

recorded as female and zero for an artist recorded as male. This is a logistic regression with one binary predictor, and β_1 is the log-odds ratio of being Gen X for female versus male artists. The model gives male artists probability `plogis`(β_0) and female artists probability `plogis`$(\beta_0 + \beta_1)$. Thus $\beta_1 > 0$ means a larger modeled Gen X probability for female artists. The coefficient describes an association within the collection. The recorded gender categories determine which artists enter this illustration.

The independent Normal priors are not conjugate to this likelihood. We will approximate the joint posterior with a grid. Each row of `grid_2d` contains one combination of β_0 and β_1, and `map2_dbl()` evaluates the log posterior at each combination.

```
art <- moma_sample |>
  filter(gender %in% c("female", "male")) |>
  mutate(genx = as.numeric(genx), female = as.numeric(gender == "female"))

log_posterior_2d <- function(beta0, beta1, data = art, sd_beta1 = 1) {
  eta <- beta0 + beta1 * data$female
  sum(dbinom(data$genx, size = 1, prob = plogis(eta), log = TRUE)) +
    dnorm(beta0, 0, 2, log = TRUE) + dnorm(beta1, 0, sd_beta1, log = TRUE)
}

G <- 200
beta0_grid <- -6 + (seq_len(G) - 0.5) * 8 / G   # cells covering [-6, 2]
beta1_grid <- -6 + (seq_len(G) - 0.5) * 12 / G  # cells covering [-6, 6]
grid_2d <- expand_grid(beta0 = beta0_grid, beta1 = beta1_grid) |>
  mutate(lp = map2_dbl(beta0, beta1, log_posterior_2d),
         prob = exp(lp - max(lp)) / sum(exp(lp - max(lp))))
```

```
marginal_beta1 <- grid_2d |>
  group_by(beta1) |>
  summarize(prob = sum(prob))

p_joint <- ggplot(grid_2d, aes(beta0, beta1, fill = prob)) +
  geom_raster() +
  scale_fill_gradient(low = "white", high = "#552E6B", guide = "none") +
  coord_cartesian(xlim = c(-4, -0.5), ylim = c(-1.5, 4)) +
  labs(x = expression(beta[0]), y = expression(beta[1]))

delta_beta1 <- diff(marginal_beta1$beta1)[1]
p_marg <- ggplot(marginal_beta1, aes(beta1, prob)) +
  geom_col(fill = "#552E6B", width = delta_beta1) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  coord_cartesian(xlim = c(-1.5, 4)) +
  labs(x = expression(beta[1]), y = "Posterior probability")

patchwork::wrap_plots(p_joint, p_marg)
```

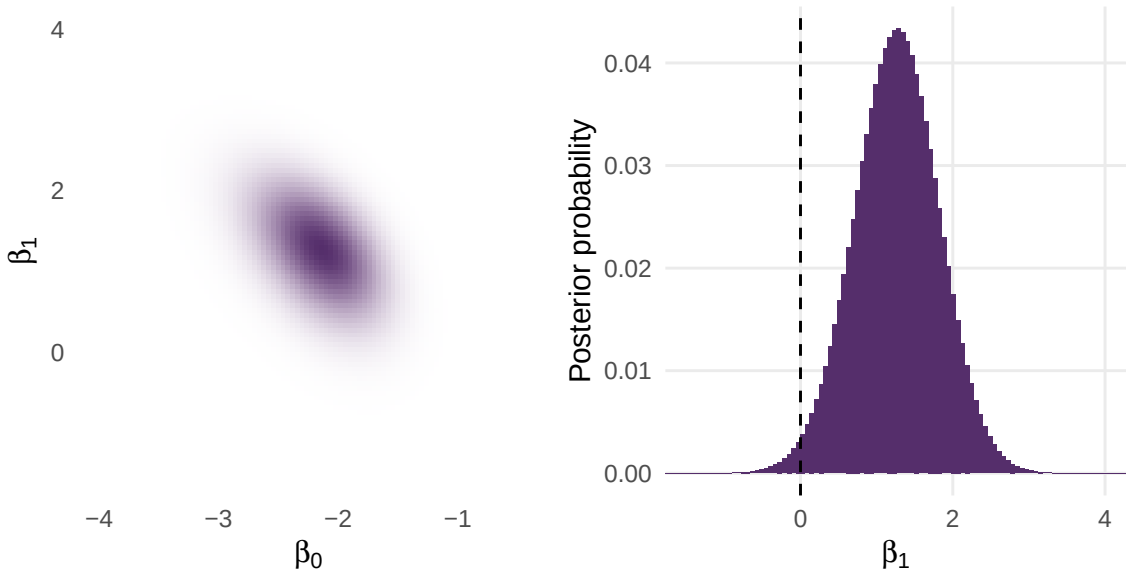


Figure 2.6: Left: the joint posterior of the logistic regression intercept β_0 and the gender effect β_1 on a 200 by 200 grid; darker is more probable. Right: the marginal posterior of β_1, obtained by summing the grid over β_0. Both panels show a central portion of the wider evaluation grid.

To obtain a marginal posterior, sum over the parameter that is no longer of interest. In `marginal_beta1`, the weight for a given β_1 adds the joint probabilities across all β_0 cells. This is the grid counterpart of integrating out β_0. The marginal posterior of β_1 is the sum of the joint grid over β_0, its mean is 1.26 on the log-odds scale, and the posterior probability that female artists are more likely to be Gen X, $P(\beta_1 > 0 \mid y)$, is 0.987. Exercise 2.9 asks you to turn this grid into a Savage–Dickey Bayes factor for $\beta_1 = 0$.

2.5.2 More parameters

Figure 2.7 shows how grid combinations multiply. With four values for one parameter, we evaluate four points. Adding a second parameter gives $4 \times 4 = 16$ pairs; a third gives $4 \times 4 \times 4 = 64$ triples. We evaluate every combination even if many have low posterior density.

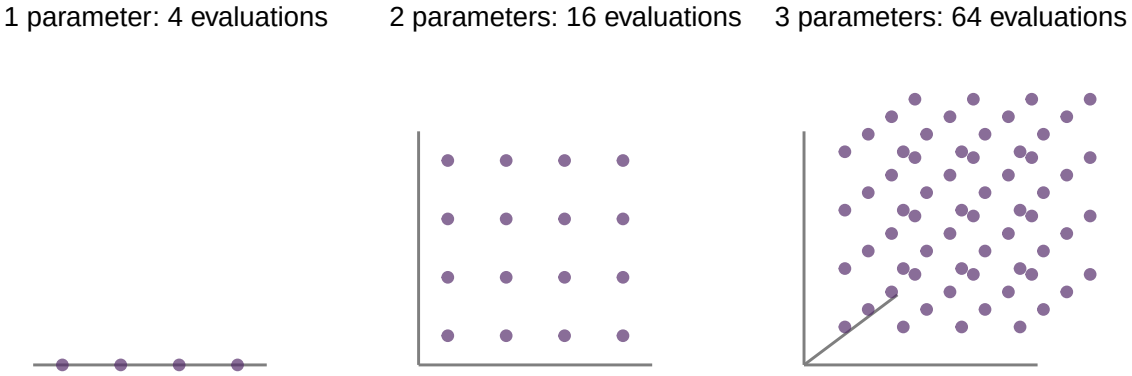


Figure 2.7: Four midpoint values per parameter produce 4 points in one dimension, 16 pairs in two dimensions, and 64 triples in three dimensions. The three-dimensional grid is shown in projection. These points illustrate evaluation locations, not posterior draws.

Look at what the two-parameter grid cost: $G^2 = 40,000$ evaluations of the log posterior, each a sum over 99 observations. Now count for a model with d parameters. A grid of G points per parameter has G^d points in total, and the number grows exponentially in d.

```r
expand_grid(d = 1:10, G = c(10, 50, 100)) |>
  mutate(points = G^d, G = factor(G, labels = paste("G =", c(10, 50, 100)))) |>
  ggplot(aes(d, points, color = G)) +
  geom_hline(yintercept = 1e9, linetype = "dashed") +
  geom_line() + geom_point() +
  scale_y_log10(labels = scales::label_log()) +
  scale_x_continuous(breaks = 1:10) +
  scale_color_manual(values = c("gray60", "gray30", "#552E6B"), name = NULL) +
  labs(x = "Number of parameters d", y = "Grid points")
```

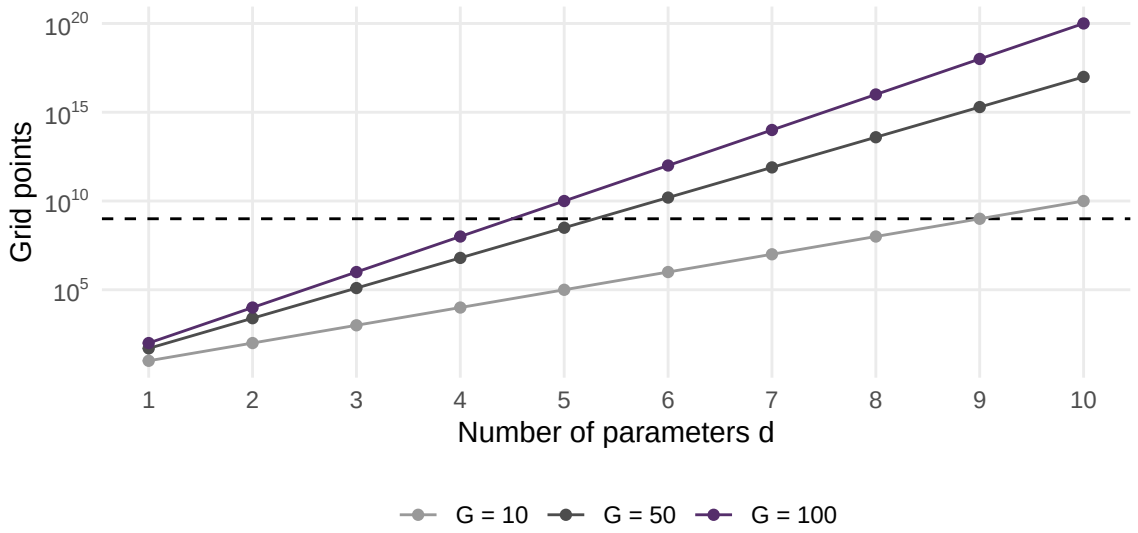


Figure 2.8: The number of grid points, G^d, as a function of the number of parameters d, for three grid resolutions. The vertical axis is logarithmic. The dashed line marks one billion evaluations.

With $G = 100$ and $d = 10$, a modest regression with an intercept, eight predictors, and a residual variance, the grid has 10^{20} points. If the computer evaluates a million log posteriors per second, that is 3.2 million years. Dropping to $G = 10$ brings it down to 10^{10}, a few hours, but ten points per parameter may miss important changes in a concentrated posterior density. Choosing a coarser grid saves work only if it still gives an adequate approximation.

A broad grid can also spend many evaluations in low-density regions. For the logistic regression, Figure 2.6 shows that the posterior occupies only part of the chosen rectangle. Narrowing the grid can save evaluations, but an overly narrow range discards posterior mass. Increasing the resolution addresses spacing; widening the range addresses truncation. Both require checks.

The growth from G to G^d points is one form of the **curse of dimensionality**. It concerns full grids at a fixed resolution per parameter. Adaptive integration and problem-specific structure can help, but a full grid becomes impractical for many models with several parameters.

Quiz yourself!

You want a grid with 50 points per parameter for a model with four parameters, and each evaluation of the log posterior takes one millisecond. How many evaluations are there, and how long will they take?

2.6 Sampling instead of gridding

In many models, we can evaluate an unnormalized posterior on the log scale even when we cannot integrate it. Grid approximation uses those evaluations to construct a discrete approximation, but its cost grows rapidly with the number of parameters. Sampling methods seek to obtain useful draws without evaluating every combination on a grid.

Markov chain Monte Carlo constructs dependent draws using a transition rule designed to have the posterior as its long-run distribution. In the Metropolis methods, the normalizing constant cancels in ratios of posterior heights, so a function such as `log_posterior()` supplies the needed evaluations. The transition rule and its conditions matter; an arbitrary sequence of accepted candidates need not represent the posterior. Chapter 3 develops those conditions, and Chapter 4 explains how to assess the resulting simulations.

2.7 Chapter summary

The chapter developed the connection between Bayesian updating and its numerical implementation.

- **Bayes' rule** normalizes the likelihood times the prior. The marginal likelihood $f(y)$ is constant with respect to the parameter. `log_posterior()` evaluates the unnormalized log posterior; omitted constants must be tracked when calculating a marginal likelihood.
- **Kernel recognition** identifies some posteriors by collecting parameter-dependent terms. Conjugate families supply familiar normalized posteriors; other models may require numerical methods.
- **Posterior summaries** include expectations, quantiles, and modes. Posterior draws estimate them in different ways. A credible interval describes parameter uncertainty; Monte Carlo error describes numerical uncertainty in its estimated summaries.
- **Bayes factors** compare marginal likelihoods and multiply prior model odds to obtain posterior model odds. Prior probabilities of models and priors on parameters within a model play different roles.
- The **Savage–Dickey density ratio** compares posterior and prior density at a null value when its conditions hold. The densities are not point probabilities. Results depend on the specified alternative prior.
- **Grid approximation** assigns weights to parameter cells and normalizes by summation. Subtracting the maximum log value stabilizes the calculation. Refining the spacing and checking the range address different approximation errors.
- A full grid with G points for each of d parameters uses G^d evaluations. This motivates sampling methods that avoid enumerating every combination.

2.8 Quiz answers

Quiz 1. No. The two functions differ by $\log \binom{n}{y}$, which does not depend on π, and their normalized posteriors are identical. Normalizing either kernel gives the same posterior, because the extra constant cancels between the numerator and its integral. It *would* matter if the students wanted the marginal likelihood itself, because $f(y)$ is the integral of the full likelihood times the prior, coefficient included.

Quiz 2. $P(0.15 < \pi < 0.25 \mid y) = E[\mathbb{1}(0.15 < \pi < 0.25) \mid y]$, the expected value of an indicator. From draws it is `mean(draws$pi > 0.15 & draws$pi < 0.25)`, which for the Beta(18, 92) posterior is about 0.61. The Monte Carlo standard error of a proportion p from N independent draws is $\sqrt{p(1-p)/N}$, estimated here by 0.0049.

Quiz 3. The arithmetic is right if two conditions hold: the prior probabilities of the two models are equal, so that $P(H_0 \mid y) = \mathrm{BF}_{01}/(1 + \mathrm{BF}_{01})$, and "the probability that $\pi = 0.2$" is understood as the posterior probability of the *model* H_0. What is wrong is reading it as a statement about the continuous posterior of π under H_1, where $P(\pi = 0.2 \mid y) = 0$ regardless of the data. The value 0.82 is a posterior model probability. It assumes a prior probability of one half for H_0, which is additional to the Bayes-factor calculation.

Quiz 4. $50^4 = 6{,}250{,}000$ evaluations. At one millisecond each, that is 6,250 seconds, about 1.7 hours. Adding a fifth parameter multiplies both numbers by 50.

2.9 Exercises

2.9.1 Conceptual exercises

Exercise 2.1 (Where did the constant go?) Explain, in words and in one formula, why we can compute posterior means, credible intervals, and posterior probabilities of interval hypotheses without ever evaluating $f(y)$, provided we can draw from the posterior. Then explain why the absolute scale matters for a marginal likelihood, and why an ordinary posterior sample average does not directly give it. How does the Savage–Dickey identity allow certain Bayes factors to be calculated without separately evaluating marginal likelihoods?

Exercise 2.2 (Kernel recognition) Derive the second and third rows of Table 2.1 by kernel recognition.

a. For $Y_1, \dots, Y_n \mid \lambda \sim \text{Pois}(\lambda)$ and $\lambda \sim \text{Gamma}(s, r)$, write the prior kernel and the joint likelihood kernel, multiply them, and identify the posterior.
b. For $Y_1, \dots, Y_n \mid \mu \sim \text{N}(\mu, \sigma^2)$ with σ known and $\mu \sim \text{N}(\theta, \tau^2)$, do the same. Hint: expand the squares in the exponent, collect the terms in μ^2 and in μ, and complete the square.
c. Now replace the prior in (a) by $\log \lambda \sim \text{N}(0, 1)$. Show that step 3 of the kernel recognition box fails, and write the `log_posterior(lambda)` function for this model anyway.

Exercise 2.3 (Point hypotheses, revisited) A fellow student has just learned about Bayes factors and has questions. Answer each in two or three sentences.

a. What is a point hypothesis, and why can its posterior probability not be read off the posterior distribution of the parameter?
b. What is an interval hypothesis, and how is its posterior probability an expectation?
c. What is a marginal likelihood, and why is it sometimes called the normalizing constant and sometimes the evidence?
d. What is the Savage–Dickey density ratio, when can it be used, and what does it avoid computing?
e. Why does a Bayes factor of 4.5 in favor of H_0 not by itself mean that $P(H_0 \mid y) = 0.82$?

Exercise 2.4 (The prior matters, a lot) A well-being score is modeled as $Y_i \mid \mu \sim \text{N}(\mu, 0.5^2)$, independently for $i = 1, \dots, 25$. The observed mean is $\bar{y} = 5.735$. Compare $H_0 : \mu = 6$ with $H_1 : \mu \sim \text{N}(6.5, \tau^2)$. Use the Normal posterior formula in Table 2.1 to explore how the Bayes factor changes as τ grows.

a. Without computing, reason about what happens to the prior density at $\mu = 6$ and to the posterior density at $\mu = 6$ as $\tau \to \infty$. What does this imply for BF_{01}?
b. Verify your reasoning with `dnorm()` for $\tau \in \{0.4, 1, 5, 25, 100\}$.
c. This limit illustrates Bartlett's paradox: With fixed data, spreading the alternative prior increasingly widely can favor the point null. Explain this using the prior predictive distribution, and discuss why increasing the prior variance is not a neutral choice for model comparison.

2.9.2 Practical exercises

Exercise 2.5 (Hypothesis tests, three ways) A colleague's model for a mean μ had a $\mathrm{N}(10, 10^2)$ prior and, after seeing the data, a $\mathrm{N}(5, 3^2)$ posterior.

a. Compute the posterior probability of the interval hypothesis $\mu \geq 5.2$ exactly with `pnorm()` and from 10,000 draws with `rnorm()`.
b. Compute the Bayes factor in favor of the point hypothesis $H_0 : \mu = 5.2$ with the Savage–Dickey density ratio.
c. Estimate the same Bayes factor from the 10,000 draws using `density()`, and compare.
d. The interval probability in (a) and the Bayes factor in (b) answer different questions. State both questions in words.

Exercise 2.6 (Posterior summaries) Use the $\mathrm{Beta}(18, 92)$ posterior of the modern art example.

a. Consider the posterior mean of the log-odds, $E[\log \frac{\pi}{1-\pi} \mid y]$. Compute it numerically with `integrate()` and from 10,000 draws with `rbeta()`. Report the Monte Carlo standard error of the second estimate. An independent check is the identity $E[\log(\pi/(1-\pi))] = \mathrm{digamma}(a) - \mathrm{digamma}(b)$ for a $\mathrm{Beta}(a, b)$ variable, evaluated in R with `digamma()`.
b. Compute the posterior probability that π lies within 0.05 of the board's target of 0.2, both ways.
c. Compute a 95% HPD interval from the draws with the `hpd()` function of this chapter and compare it with the central interval. Which is shorter, and by how much?

Exercise 2.7 (The grid, in detail) Use `grid_posterior()` and `log_posterior_logitnormal()` from this chapter.

a. Use midpoint grids `pi_grid <- (seq_len(G) - 0.5) / G`. Compute the posterior mean and central 95% interval of π under the logit-Normal prior for $G = 10$, 20, 50, 200, and 1,000. For the grid quantiles, use the first midpoint at which the cumulative probability reaches 0.025 or 0.975. At what G do the summaries stop changing in the second decimal?
b. Modify `grid_posterior()` so that it also returns $\log f(y) \approx c + \log(\Delta \sum_j w_j)$. Retain the full Binomial probability mass and the normalized prior in `log_posterior()`. Check against `log_ml_H1` for the $\mathrm{Beta}(4, 6)$ model.
c. Use the modified function to compute the Bayes factor comparing the $\mathrm{Beta}(4, 6)$ prior with the logit-Normal prior for the same data. Which prior predicted the data better?

2.9.3 Applied exercises

Exercise 2.8 (Ghosts) The `pulse_of_the_nation` data in the `bayesrules` package record, among many other things, whether each of 1,000 survey respondents believes in ghosts (`ghosts`). Suppose that before seeing the data you place a $\mathrm{Beta}(2, 3)$ prior on the proportion π of believers, and that a colleague claims that "about a third of people believe in ghosts."

a. Load `data("pulse_of_the_nation", package = "bayesrules")` and tabulate `ghosts`, including missing values. Identify the recorded categories for yes and no, retain valid yes/no responses, and report the resulting y and n. Write the posterior for π.
b. Compute the Bayes factor in favor of $H_0 : \pi = 1/3$ using the two closed-form marginal likelihoods. Calculate their logarithms with `dbinom(..., log = TRUE)`, `lchoose()`, and `lbeta()` before exponentiating their difference.
c. Compute the same Bayes factor with the Savage–Dickey density ratio and confirm that the two agree.
d. Repeat (c) with a $\mathrm{Beta}(1, 1)$ prior and with a $\mathrm{Beta}(30, 60)$ prior. Interpret the three Bayes factors

with the table in this chapter, and explain why they differ.

e. Explain why a large sample does not by itself guarantee evidence for the point null. How does the observed proportion relative to 1/3 affect the comparison? Distinguish evidence favoring a null model from failing to reject a null in a significance test.

Exercise 2.9 (Is the gender effect zero?) Use the two-parameter logistic regression grid and its marginal `marginal_beta1`. Under H_0, fix $\beta_1 = 0$ and retain the $\mathrm{N}(0, 2^2)$ prior on β_0. Under H_1, use the independent priors specified in the chapter. This matches the prior on the remaining parameter as required by the Savage–Dickey identity.

a. The marginal grid probabilities of β_1 approximate a density once divided by the grid spacing Δ. Estimate the marginal posterior density of β_1 at 0 by linear interpolation of `prob / delta` between the two grid points closest to 0.

b. Divide by the prior density `dnorm(0, 0, 1)` to obtain the Savage–Dickey Bayes factor BF_{01} for $H_0 \colon \beta_1 = 0$. Interpret it.

c. With equal prior probabilities for the two models, what is the posterior probability that the gender effect is zero? Compare it with $P(\beta_1 > 0 \mid y)$ from the text and explain why the two numbers answer different questions.

d. Repeat (a) to (b) with a $\mathrm{N}(0, 3^2)$ prior on β_1, widening the grid as needed. Recompute the grid with `sd_beta1 = 3`; check both its range and resolution. How does the Bayes factor change? Relate the result to the prior-predictive argument in Exercise 2.4, without assuming that a wider prior must always favor H_0.

Part II

Markov Chains

Chapter 3

Markov Chains

Suppose you keep a mood diary, recording *happy*, *neutral*, or *sad* each evening. A simple model might allow tomorrow's mood to depend on today's. For example, a happy day could be followed by another happy day more often than by a sad one. To use a **Markov chain** for this model, we make a specific assumption. Once today's mood is known, knowing earlier moods provides no additional information about tomorrow's mood. Whether this assumption describes a real diary well is a separate modeling question.

Markov chains also serve a computational purpose. Instead of modeling a person's mood, we can construct a sequence of parameter values whose long-run behavior represents a posterior distribution. This chapter develops the connection, beginning with a small set of mood states and ending with a chain that samples a grid approximation to a posterior.

We will distinguish the value of a chain at one time from the distribution of that value, and distinguish convergence of distributions from convergence of averages along a chain. These distinctions explain how dependent simulation can support Bayesian inference. Run the following setup before working through the examples.

```
library(tidyverse)
library(patchwork)
```

Goals

- Define and simulate a Markov chain with a finite set of possible states, specifying both its initial distribution and its transition probabilities.
- Compute and interpret state probabilities using a transition matrix and its powers.
- Determine stationary distributions using equations, eigenvectors, matrix powers, and simulated frequencies, and explain the conditions under which the last two methods apply.
- Analyze irreducibility and aperiodicity using examples and apply the finite-state convergence theorem.
- Distinguish a distribution at a specified time from a time average, and interpret the finite-state ergodic theorem.
- Verify detailed balance, explain why it implies stationarity, and check irreducibility and aperiodicity separately.
- Interpret a continuous transition rule through a worked Normal example.
- Construct and evaluate a grid-based sampling rule that preserves a chosen target distribution.

3.1 Chains on a finite state space

A **state space** is the set of values a chain can take. For the mood model it is $\{\text{happy}, \text{neutral}, \text{sad}\}$. This is a **finite state space** because it contains only three possible values. Write $X^{(t)}$ for the state at time t, with $t = 0, 1, 2, \ldots$.

The **Markov property** says that, conditional on the current state, the distribution of the next state does not depend on earlier states:

$$P\big(X^{(t+1)} = j \mid X^{(t)} = i, X^{(t-1)}, \dots, X^{(0)}\big) = P\big(X^{(t+1)} = j \mid X^{(t)} = i\big).$$

Here i is the **current** state and j is the **next** state. We consider **time-homogeneous** chains, whose transition probabilities do not change with t, and write these probabilities as P_{ij}. Numbering happy, neutral, and sad as states 1, 2, and 3 gives, for example,

$$P_{12} = P(\text{neutral tomorrow} \mid \text{happy today}).$$

Specifying a finite-state Markov chain

Specify the possible states, an **initial distribution** $\mu^{(0)}$ over them, and a transition matrix $\mathbf{P}$. Entry P_{ij} is the probability of moving from state i to state j in one step.

A fixed initial state is a special case. Starting happy with certainty gives $\mu^{(0)} = (1, 0, 0)$. Alternatively, we can draw the initial state using any specified probability vector over the three moods.

Arrange the transition probabilities with one row for each current state and one column for each next state. The following matrix defines our illustrative mood model. In R, `byrow = TRUE` fills the entries row by row, and `dimnames` attaches the mood labels to both rows and columns:

```
moods <- c("happy", "neutral", "sad")
P <- matrix(c(0.70, 0.25, 0.05,
              0.20, 0.50, 0.30,
              0.20, 0.40, 0.40),
            nrow = 3, byrow = TRUE, dimnames = list(moods, moods))
P
#>         happy neutral  sad
#> happy     0.7    0.25 0.05
#> neutral   0.2    0.50 0.30
#> sad       0.2    0.40 0.40
```

Read it by rows. After a happy day, the next day is happy with probability 0.70, neutral with probability 0.25, and sad with probability 0.05. After a sad day, the mood stays sad with probability 0.40, recovers to neutral with probability 0.40, and jumps straight to happy with probability 0.20. Every row is a probability distribution over tomorrow, so every row sums to one:

```
rowSums(P)
#>   happy neutral     sad
#>       1       1       1
```

The column sums need not equal one. A matrix with non-negative entries and row sums of one is called a **stochastic matrix**. Together with an initial distribution, it specifies a time-homogeneous Markov chain.

A note on notation. In this chapter the states are moods, so we call the chain $X^{(t)}$ and the states i and j. From Section 3.6 onward, the states are candidate values of a parameter, and we write the chain as $\theta^{(t)}$, as in Appendix B. The states then describe a computational sequence rather than an observed sequence of moods. We use bold π for a vector of stationary probabilities and plain π for an unknown proportion.

3.1.1 Simulating the chain

At each transition, select the row for the current mood and draw the next mood using that row's probabilities. `sample()` takes the state labels, the number of draws, and the probabilities. For example, `sample(moods, 1, prob = P["happy", ])` generates a possible next state after a happy day.

A simulated chain of length N contains the states $X^{(0)}, \ldots, X^{(N-1)}$, including the initial state. The function below makes `n_iter` transitions and returns one state more than that, because the initial state is included. With `n_iter = 60`, it returns 61 states at times 0 through 60. The first stored element, `x[1]`, represents $X^{(0)}$.

```
simulate_chain <- function(P, init, n_iter) {
  states <- rownames(P)
  x <- character(n_iter + 1)
  x[1] <- init
  for (t in seq_len(n_iter)) {
    x[t + 1] <- sample(states, size = 1, prob = P[x[t], ])
  }
  x
}
```

```
set.seed(84735)
diary <- simulate_chain(P, init = "happy", n_iter = 60)
head(diary, 14)
#>  [1] "happy"   "neutral" "sad"     "sad"     "happy"   "neutral" "neutral"
#>  [8] "sad"     "sad"     "sad"     "neutral" "happy"   "happy"   "happy"
```

Consecutive entries can repeat. For example, the probability of remaining happy for one more day is $P_{11} = 0.70$. Figure 3.1 shows the resulting path. Each point is one realized state, not a probability distribution.

```
tibble(day = 0:60, mood = factor(diary, levels = rev(moods))) |>
  ggplot(aes(day, mood, group = 1)) +
  geom_step(color = "#552E6B") +
  geom_point(color = "#552E6B", size = 1) +
  labs(x = "Day", y = NULL)
```

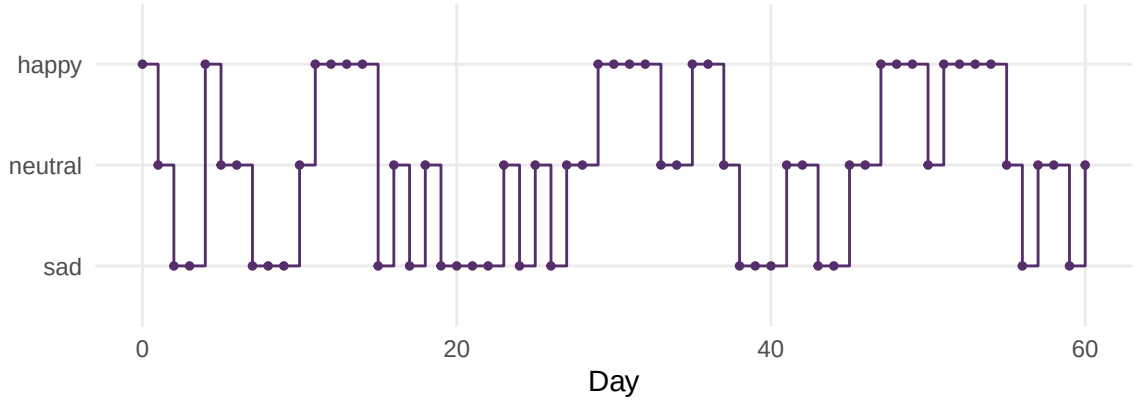


Figure 3.1: A mood diary simulated for 60 transitions, with its happy initial state shown at day zero. Consecutive states can be identical. The path records one sequence of outcomes.

3.1.2 Where will the chain be in t days?

The transition matrix answers questions about tomorrow. To ask about the day after tomorrow, we sum over what can happen in between. Starting from state i, the chain reaches state j in two steps by passing

through some state k on day one, so

$$P\big(X^{(t+2)} = j \mid X^{(t)} = i\big) = \sum_{k} P_{ik} P_{kj},$$

The index k runs over the possible intermediate states. Each product describes a two-step route, and summing the products accounts for all those routes. This is the (i, j) entry of $\mathbf{P}^2$. More generally, entry (i, j) of $\mathbf{P}^t$ is the probability of reaching j after t transitions from i.

In R, `%*%` performs matrix multiplication, whereas `P^2` squares each entry separately. The function below begins with the identity matrix, representing zero transitions, and multiplies by `P` once per transition. With `t = 0`, `seq_len(t)` is empty and the identity is returned.

```
mat_power <- function(P, t) {
  out <- diag(nrow(P))
  dimnames(out) <- dimnames(P)
  for (k in seq_len(t)) out <- out %*% P
  out
}
mat_power(P, 2)
#>         happy neutral  sad
#> happy    0.55    0.32 0.13
#> neutral  0.30    0.42 0.28
#> sad      0.30    0.41 0.29
```

Quiz yourself!

Today is a happy day. Compute by hand, from $\mathbf{P}$, the probability that the day after tomorrow is sad. Check your answer against `mat_power(P, 2)`.

The distribution at a given time is a different object from one realized state. The vector

$$\mu^{(t)} = \big(P(X^{(t)} = \text{happy}), P(X^{(t)} = \text{neutral}), P(X^{(t)} = \text{sad})\big)$$

lists the probabilities of the three possible moods at time t. Its entries sum to one. Starting happy with certainty gives $\mu^{(0)} = (1, 0, 0)$; one transition gives $(0.70, 0.25, 0.05)$.

For a less certain start, suppose $\mu^{(0)} = (0.2, 0.5, 0.3)$. The probability of a happy day after one transition is

$$0.2 \times 0.70 + 0.5 \times 0.20 + 0.3 \times 0.20 = 0.30.$$

The other entries are calculated in the same way. Matrix multiplication performs all three weighted sums:

```
mu0 <- c(happy = 0.2, neutral = 0.5, sad = 0.3)
mu0 %*% P
#>      happy neutral  sad
#> [1,]   0.3    0.42 0.28
```

The result is $(0.30, 0.42, 0.28)$. It describes probabilities across possible realizations of the chain, not three moods experienced simultaneously by one person. We use row vectors, so the update is

$$\mu^{(t+1)} = \mu^{(t)} \mathbf{P}, \qquad \mu^{(t)} = \mu^{(0)} \mathbf{P}^t. \tag{3.1}$$

Each row of $\mathbf{P}^t$ corresponds to a different fixed starting state. Compare the rows after five and twenty transitions:

```
round(mat_power(P, 5), 4)
#>         happy neutral    sad
```

```
#> happy   0.4187  0.3707 0.2105
#> neutral 0.3875  0.3825 0.2300
#> sad     0.3875  0.3825 0.2300
round(mat_power(P, 20), 4)
#>         happy neutral    sad
#> happy     0.4  0.3778 0.2222
#> neutral   0.4  0.3778 0.2222
#> sad       0.4  0.3778 0.2222
```

After twenty transitions, the rows agree to four decimal places. The probability of a happy state is approximately 0.4 for any of the three starts. This is approximate agreement of state distributions; the individual paths still fluctuate.

3.2 Stationary distributions

Stationary distribution

A probability vector $\pi = (\pi_1, \dots, \pi_S)$ over the S states is a **stationary distribution** of the chain with transition matrix $\mathbf{P}$ if

$$\pi\mathbf{P} = \pi, \qquad \text{that is,} \qquad \sum_i \pi_i P_{ij} = \pi_j \quad \text{for every state } j.$$

If $X^{(t)}$ is distributed according to π, then so is $X^{(t+1)}$, and so is every later state.

The definition says that π is a fixed point of the update $\mu \mapsto \mu\mathbf{P}$. If you start the chain by drawing $X^{(0)}$ from π, the distribution of the state never changes, even though the state itself keeps moving. For a population of diary keepers following this model, the expected proportions in the three moods would then remain constant. Realized proportions in a finite population can still fluctuate. The word stationary describes the distribution, not an unchanging individual mood. We first solve its defining equations, then compare matrix powers and simulated frequencies.

Solve the equation. Write h, u, and s for the stationary probabilities of happy, neutral, and sad. The first stationary equation and $h + u + s = 1$ give

$$h = 0.70h + 0.20u + 0.20s = 0.70h + 0.20(1 - h), \qquad 0.50h = 0.20,$$

so $h = 0.40$. The neutral equation is

$$u = 0.25(0.40) + 0.50u + 0.40(0.60 - u) = 0.34 + 0.10u.$$

Thus $u = 0.34/0.90 = 17/45$, and $s = 0.60 - 17/45 = 2/9$. The stationary vector is $(0.40, 17/45, 2/9)$.

For a larger matrix, eigenvectors provide a numerical way to solve the same equation. The condition $\pi\mathbf{P} = \pi$ says that π is a *left* eigenvector of $\mathbf{P}$ with eigenvalue 1. Equivalently, transposing both sides, $\pi^\top$ is an ordinary (right) eigenvector of $\mathbf{P}^\top$, which is what `eigen()` computes. If you have encountered principal component analysis (PCA), the operation may be familiar. PCA finds eigenvectors of a covariance matrix to identify directions of variation. Here the matrix is $\mathbf{P}^\top$, and an eigenvalue of one identifies a vector unchanged by the probability update. We rescale that eigenvector so its entries sum to one; no knowledge of PCA is needed for this calculation.

The function below is used for chains with a unique stationary distribution. If there are several stationary distributions, the eigenvalue-one condition alone does not select a particular mixture.

```r
stationary <- function(P) {
  e <- eigen(t(P))
  k <- which.min(abs(e$values - 1))     # the eigenvalue equal to 1
  v <- Re(e$vectors[, k])
  setNames(v / sum(v), rownames(P))
}
pi_mood <- stationary(P)
pi_mood
#>     happy   neutral       sad
#> 0.4000000 0.3777778 0.2222222
```

For this chain, long-run frequencies approach about 40% of days are happy, 38% neutral, and 22% sad. We can check the defining equation directly:

```r
pi_mood %*% P
#>      happy   neutral       sad
#> [1,]   0.4 0.3777778 0.2222222
```

Raise P to a large power. For this chain, each row approaches the stationary vector as the power increases. The rows agree approximately, rather than becoming exactly equal after a finite number of transitions. The conditions supporting this approximation are stated in the next section.

```r
mat_power(P, 50)[1, ]
#>     happy   neutral       sad
#> 0.4000000 0.3777778 0.2222222
```

Run the chain and count. The third way is the one that matters for the rest of the book. Simulate a long diary, and count how often each mood occurs:

```r
set.seed(84735)
long_diary <- simulate_chain(P, init = "sad", n_iter = 20000)
freq_mood <- table(factor(long_diary[-1], levels = moods)) / (length(long_diary) - 1)
freq_mood
#>
#>   happy neutral     sad
#> 0.39535 0.37980 0.22485
```

The simulated frequencies approximate π (Figure 3.2). We omit the initial state when counting the 20,000 generated states. Unlike the eigenvector and matrix-power calculations, these estimates vary from simulation to simulation. The ergodic theorem below explains why averaging along an appropriate chain can estimate properties of its stationary distribution.

```r
tibble(mood = factor(rep(moods, 3), levels = moods),
       method = rep(c("Eigenvector", "Matrix power", "Simulated frequencies"), each =
         3),
       probability = c(pi_mood, mat_power(P, 50)[1, ], as.numeric(freq_mood))) |>
  ggplot(aes(mood, probability, fill = method)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = c("#552E6B", "#9E7BB5", "#ECECEC"), name = NULL) +
  labs(x = NULL, y = "Probability")
```

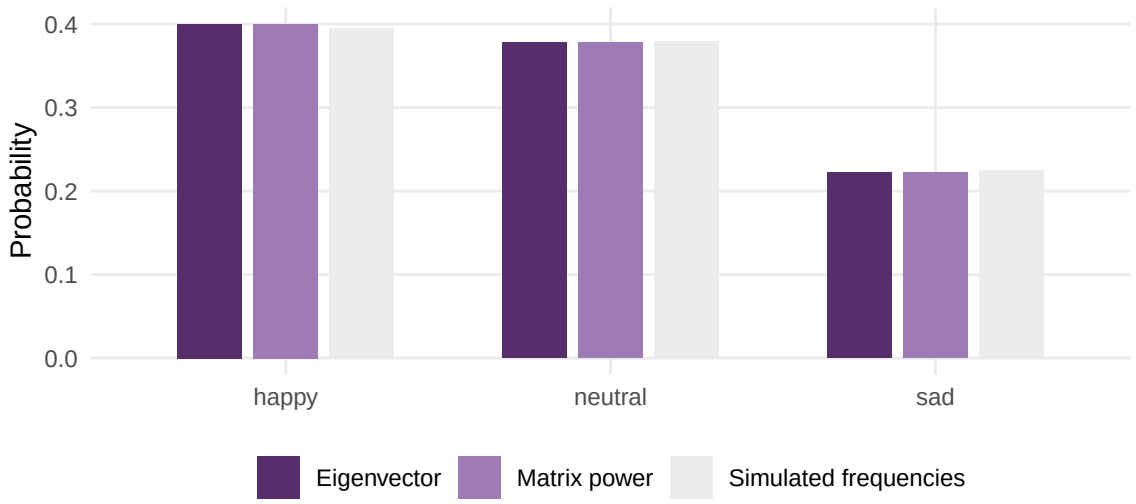


Figure 3.2: Stationary probabilities from an eigenvector calculation and a large matrix power, compared with frequency estimates from 20,000 generated states. The simulated frequencies have Monte Carlo error.

How quickly does the starting distribution matter less? Figure 3.3 follows all three mood probabilities from each possible initial state, using Equation 3.1. These are calculated probabilities, not three simulated diary paths.

```
forgetting <- expand_grid(start = moods, day = 0:12) |>
  mutate(distribution = map2(start, day, \(s, t) {
    mu0 <- setNames(as.numeric(moods == s), moods)
    drop(mu0 %*% mat_power(P, t))
  })) |>
  unnest_wider(distribution) |>
  pivot_longer(all_of(moods), names_to = "mood", values_to = "probability") |>
  mutate(mood = factor(mood, levels = moods, labels = c("Happy", "Neutral", "Sad")))
stationary_lines <- tibble(mood = factor(c("Happy", "Neutral", "Sad"),
                                         levels = c("Happy", "Neutral", "Sad")),
                           probability = as.numeric(pi_mood))
ggplot(forgetting, aes(day, probability, color = start, linetype = start)) +
  geom_hline(data = stationary_lines, aes(yintercept = probability),
             linetype = "dotted", inherit.aes = FALSE) +
  geom_line(linewidth = 0.7) +
  facet_wrap(~ mood) +
  scale_color_manual(values = c("happy" = "#552E6B", "neutral" = "#9E7BB5", "sad" =
    "gray50"), name = "Initial mood") +
  scale_linetype_manual(values = c("happy" = "solid", "neutral" = "dashed", "sad" =
    "dotdash"), name = "Initial mood") +
  scale_x_continuous(breaks = c(0, 6, 12)) +
  labs(x = "Day t", y = "State probability")
```

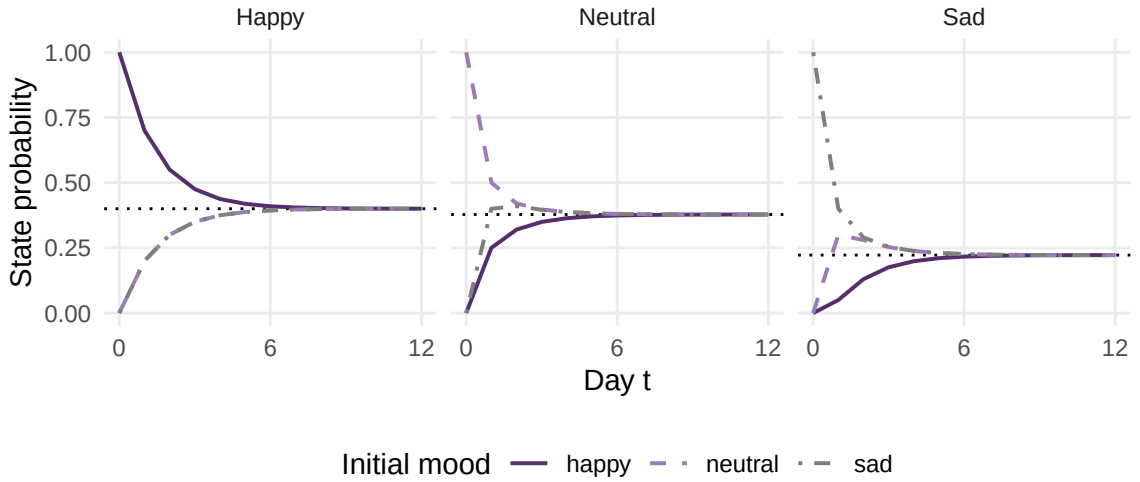


Figure 3.3: State probabilities from each of the three fixed initial moods. Thin black dotted lines mark the stationary probabilities. In the Happy panel, the neutral and sad starts produce exactly the same curve, so only two distinct colored curves are visible. The other panels show how these starts differ for the remaining moods.

The overlapping curves in the Happy panel have a numerical explanation. A neutral or sad day is followed by a happy day with the same probability, 0.20. If $p_t = P(X^{(t)} = \text{happy})$, then

$$p_{t+1} = 0.70p_t + 0.20(1 - p_t) = 0.20 + 0.50p_t.$$

Neutral and sad starts both have $p_0 = 0$, so their happy-state probabilities agree at every time. Subtract the stationary happy probability, 0.40, from the update:

$$p_{t+1} - 0.40 = 0.50(p_t - 0.40), \qquad p_t = 0.40 + 0.50^t(p_0 - 0.40).$$

For a happy start, the sequence begins $1, 0.70, 0.55, 0.475$. The difference from 0.40 halves at each transition. This explains the rate in this particular panel.

Eigenvalues connect this repeated multiplication to the matrix calculation. The eigenvalues of $\mathbf{P}$ are 1, 0.5, and 0.1. The eigenvalue 1 corresponds to the stationary vector. In this example, the other components shrink at rates 0.5^t and 0.1^t. A component multiplied by 0.99 at each transition would decline much more slowly; after 100 transitions it would retain about 37% of its initial size, because $0.99^{100} \approx 0.37$. Eigenvalues near one in absolute value can therefore indicate slow decay of initial differences.

Discarding initial states is one way to reduce the effect of an arbitrary start when estimating posterior summaries. It does not make later states independent. In Chapter 4, warm-up also includes the adaptation of sampler settings.

3.3 Irreducibility, aperiodicity, and convergence

The mood chain has a unique stationary distribution, and its state distribution approaches that distribution from every start. Every finite-state chain has at least one stationary distribution, but uniqueness and convergence require further conditions. The following examples separate those questions.

A chain with two closed groups of states. Consider a model with two types of diary keepers. Type A can be happy or neutral; type B can be sad or gloomy. In this model a person's type stays fixed. Combining the four moods into one transition matrix gives two groups between which an individual chain cannot move:

```
P_split <- matrix(c(0.7, 0.3, 0.0, 0.0,
                    0.4, 0.6, 0.0, 0.0,
                    0.0, 0.0, 0.5, 0.5,
                    0.0, 0.0, 0.2, 0.8),
                  nrow = 4, byrow = TRUE,
                  dimnames = list(c("happy", "neutral", "sad", "gloomy"),
                                  c("happy", "neutral", "sad", "gloomy")))
mat_power(P_split, 50)
#>             happy   neutral       sad    gloomy
#> happy   0.5714286 0.4285714 0.0000000 0.0000000
#> neutral 0.5714286 0.4285714 0.0000000 0.0000000
#> sad     0.0000000 0.0000000 0.2857143 0.7142857
#> gloomy  0.0000000 0.0000000 0.2857143 0.7142857
```

Rows for type A approach $(4/7, 3/7, 0, 0)$, while rows for type B approach $(0, 0, 2/7, 5/7)$. Each vector is stationary, and any weighted average of the two is stationary as well. Starting in one group does not tell us the distribution of states in the other group.

This chain is **reducible**. A chain is **irreducible** if every state can be reached from every other in some finite number of transitions with positive probability. The mood chain with three states is irreducible, because every entry of its matrix is positive.

A chain that never settles. Now suppose a diary keeper's moods alternate: a happy day is always followed by a sad one, and a sad day by a happy one.

```
P_swing <- matrix(c(0, 1,
                    1, 0),
                  nrow = 2, byrow = TRUE,
                  dimnames = list(c("happy", "sad"), c("happy", "sad")))
mat_power(P_swing, 7)
#>       happy sad
#> happy     0   1
#> sad       1   0
mat_power(P_swing, 8)
#>       happy sad
#> happy     1   0
#> sad       0   1
```

This chain is irreducible and has the unique stationary distribution $(0.5, 0.5)$. Check it by multiplication:

$$(0.5, 0.5)\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = (0.5, 0.5).$$

The stationary equation can hold even when the distribution from a fixed starting state oscillates. But $\mathbf{P}^t$ never converges: it is $\mathbf{P}$ for odd t and the identity for even t. Started on a happy day, the chain is happy on every even day with certainty, forever (Figure 3.4). From that fixed initial state, the distribution of $X^{(t)}$ does not approach π. Starting with $\pi = (0.5, 0.5)$ would be different: That distribution is already stationary.

This chain is **periodic** with period 2: it can only return to a state after an even number of steps.

The period of a state is the greatest common divisor of its possible positive return times: The largest integer dividing every such time. Return times 2, 4, 6, and so on have common divisor 2. If returns are possible in both 2 and 3 transitions, their greatest common divisor is 1. A chain is **aperiodic** when every state has period 1.

A useful sufficient condition in an irreducible chain is a self-loop. If some $P_{ii} > 0$, the chain can return to i in one step, so the period of i is 1, and in an irreducible chain all states share the same period. The mood chain has positive entries on its diagonal, so it is aperiodic.

```r
p_happy <- function(P, start, t) {
  mu0 <- setNames(as.numeric(rownames(P) == start), rownames(P))
  (mu0 %*% mat_power(P, t))[1, "happy"]
}
bind_rows(
  tibble(chain = "Mood chain", day = 0:12,
         p = map_dbl(day, \(t) p_happy(P, "happy", t)), pi = pi_mood["happy"]),
  tibble(chain = "Alternating chain", day = 0:12,
         p = map_dbl(day, \(t) p_happy(P_swing, "happy", t)), pi = 0.5)
) |>
  mutate(chain = factor(chain, levels = c("Mood chain", "Alternating chain"))) |>
  ggplot(aes(day, p)) +
  geom_hline(aes(yintercept = pi), linetype = "dashed") +
  geom_line(color = "#552E6B") + geom_point(color = "#552E6B") +
  facet_wrap(~ chain) +
  scale_x_continuous(breaks = seq(0, 12, by = 2)) +
  labs(x = "Day t", y = expression(P(X^(t) == "happy")))
```

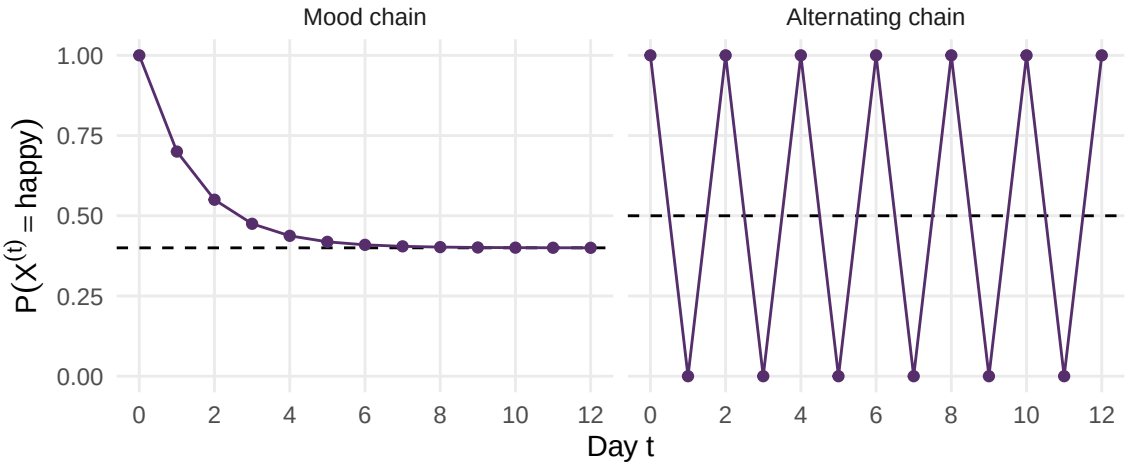


Figure 3.4: $P(X^{(t)} = \text{happy})$ for a chain started on a happy day: the mood chain (left) converges to π_{happy}, the alternating chain (right) swings between 1 and 0 forever, even though $\pi = (0.5, 0.5)$ is its stationary distribution.

The convergence theorem

Let $\mathbf{P}$ be the transition matrix of a Markov chain on a finite state space.

- If the chain is **irreducible**, it has exactly one stationary distribution π, and every state has $\pi_i > 0$.
- If the chain is also **aperiodic**, then $\mu^{(0)}\mathbf{P}^t \to \pi$ as $t \to \infty$ for every starting distribution $\mu^{(0)}$. Equivalently, every row of $\mathbf{P}^t$ converges to π.

These finite-state results are developed in Norris (1997) and Robert & Casella (2004, Chapter 6). The examples illustrate why both conditions matter. The split chain has more than one stationary distribution; the alternating chain has one, but does not converge to it from a fixed state. In the sampling construction later in this chapter, we check stationarity, irreducibility, and aperiodicity separately.

Quiz yourself!

A chain on three states has transition matrix

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

Is it irreducible? What are the possible return times, and is it aperiodic? Check whether $(1/3, 1/3, 1/3)$ is stationary by multiplying it by $\mathbf{P}$. Starting from $\mu^{(0)} = (1, 0, 0)$, does the state distribution converge to that vector? Use the theorem to explain your conclusions.

3.4 The ergodic theorem

Convergence of $\mu^{(t)}$ to π is a statement about the distribution of the state on a single day t. For Monte Carlo estimation, we usually average over many states from one chain, as in the calculation of `freq_mood`. The **ergodic theorem** establishes the convergence of these time averages. It is the Markov chain version of the law of large numbers from Chapter 1.

The ergodic theorem (informal)

Let $X^{(0)}, X^{(1)}, \ldots$ be an irreducible Markov chain on a finite state space with stationary distribution π, and let h be any function of the state. Then, whatever the starting state,

$$\frac{1}{N}\sum_{t=1}^{N} h\big(X^{(t)}\big) \longrightarrow \sum_{i} h(i)\,\pi_i \qquad \text{as } N \to \infty.$$

N is the number of sampled states included in the average. Long-run averages along one chain converge to expectations under the stationary distribution.

Take $h(i) = 1$ if i is happy and 0 otherwise. The theorem says that the fraction of happy days in a long diary converges to π_{happy}, which is what Figure 3.2 showed. Figure 3.5 shows the convergence in progress: three diaries of 2,000 transitions, started in the three moods, and for each the running fraction of happy days over the 2,001 days including day zero. Here the running fractions include the initial state, so they begin at 1, 0, and 0. Including one initial value does not change their limiting values, although finite-simulation estimates differ.

```
set.seed(84735)
running <- map(moods, \(s) {
  x <- simulate_chain(P, init = s, n_iter = 2000)
  tibble(start = s, day = 0:2000, running_mean = cummean(x == "happy"))
}) |> list_rbind()

ggplot(running, aes(day, running_mean, color = start)) +
  geom_hline(yintercept = pi_mood["happy"], linetype = "dashed") +
  geom_line() +
  scale_color_manual(values = c("#552E6B", "#9E7BB5", "gray50"), name = "Started") +
  coord_cartesian(ylim = c(0, 1)) +
  labs(x = "Day", y = "Fraction of happy days")
```

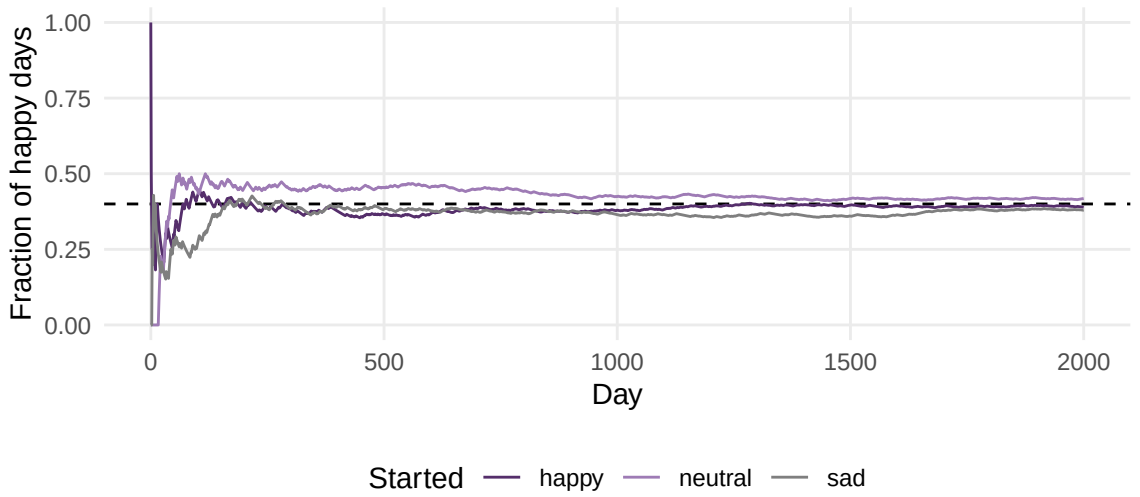


Figure 3.5: Running fraction of happy days in three simulated diaries of 2,000 transitions, one started in each mood. The dashed line is π_{happy}. The finite-run estimates fluctuate; the finite-state ergodic theorem establishes their common limiting value.

The alternating chain illustrates the difference between the two kinds of convergence. Starting happy, its state distribution alternates forever between $(1, 0)$ and $(0, 1)$. Its time average of the happy-state indicator nevertheless approaches 0.5. Aperiodicity is needed for the finite-state distribution-convergence result, but not for this finite-state ergodic theorem.

Dependence also does not invalidate the time-average result. It does affect numerical precision. To isolate that effect, compare 365 observations from the mood chain with 365 independent draws from its stationary distribution. We draw the chain's initial state from π so that all 365 states have that marginal distribution and the comparison is not confounded by a fixed starting mood.

```
set.seed(84735)
one_year <- function() {
  initial_mood <- sample(moods, size = 1, prob = pi_mood)
  chain <- simulate_chain(P, init = initial_mood, n_iter = 364)
  iid   <- sample(moods, size = 365, replace = TRUE, prob = pi_mood)
  c(chain = mean(chain == "happy"), iid = mean(iid == "happy"))
}
years <- map(1:500, \(i) one_year()) |> bind_rows()
years |> summarize(across(everything(), list(mean = mean, sd = sd)))
#> # A tibble: 1 x 4
#>   chain_mean chain_sd iid_mean iid_sd
#>        <dbl>    <dbl>    <dbl>  <dbl>
#> 1      0.401   0.0422    0.401 0.0247
```

Both estimates have expectation π_{happy}. In this experiment, the standard deviation across the dependent-chain estimates is about 1.7 times the standard deviation across the independent estimates. The persistence of moods makes the chain estimates more variable. The ergodic theorem establishes their limiting behavior, but does not supply a standard error for a finite run. Chapter 4 develops effective sample size and Monte Carlo standard errors for dependent draws.

3.5 Detailed balance

So far, we have specified a transition matrix and investigated its stationary distribution. For posterior simulation, we work in the other direction. We specify the target distribution and choose transitions that preserve it. Many different transition matrices can have the same stationary distribution. **Detailed balance** provides a convenient way to construct one of them.

Suppose the current state has distribution π. The product $\pi_i P_{ij}$ is the joint probability of being in i now and in j next. It can also be interpreted as an expected fraction of transitions in a large population of diary keepers. We call it the probability **flow** from i to j (Hanada & Matsuura, 2022, Section 3.4). Stationarity says that the total flow into each state equals the total flow out of it, so that no state gains or loses probability:

$$\underbrace{\sum_i \pi_i P_{ij}}_{\text{flow into } j} = \pi_j = \underbrace{\sum_k \pi_j P_{jk}}_{\text{flow out of } j}.$$

This is a **balance** condition: it constrains the sums. **Detailed** balance asks for more, namely that the flow between each *pair* of states is the same in both directions.

Detailed balance

A transition matrix $\mathbf{P}$ satisfies **detailed balance** with respect to a probability vector π if

$$\pi_i P_{ij} = \pi_j P_{ji} \qquad \text{for all states } i, j.$$

If detailed balance holds, then π is a stationary distribution of the chain.

To show why detailed balance implies stationarity, fix any next state j and sum over the current states i:

$$\sum_i \pi_i P_{ij} = \sum_i \pi_j P_{ji} = \pi_j \sum_i P_{ji} = \pi_j.$$

The first equality is detailed balance, applied to every term. The second pulls π_j out of the sum, because it does not depend on i. The third uses that row j of $\mathbf{P}$ sums to one. The left-hand side is the jth entry of $\pi\mathbf{P}$, so we have shown $\pi\mathbf{P} = \pi$. Thus pairwise balance ensures that the total probability arriving in each state equals its stationary probability. The proof uses the row-sum condition as well as detailed balance.

Detailed balance is *sufficient* for stationarity, not necessary. The mood chain is an example. Its stationary distribution certainly balances, but look at the flows in each direction:

```
flow <- P
for (i in seq_len(nrow(P))) {
  flow[i, ] <- pi_mood[i] * P[i, ]
}
round(flow, 3)
#>         happy neutral   sad
#> happy   0.280   0.100 0.020
#> neutral 0.076   0.189 0.113
#> sad     0.044   0.089 0.089
round(flow - t(flow), 3)       # zero everywhere if detailed balance holds
#>          happy neutral    sad
#> happy    0.000   0.024 -0.024
#> neutral -0.024   0.000  0.024
#> sad      0.024  -0.024  0.000
```

The loop multiplies every entry in row i by π_i. For example, the happy-to-neutral flow is $0.40 \times 0.25 = 0.10$, whereas the neutral-to-happy flow is $(17/45) \times 0.20 \approx 0.0756$. Transposing the matrix reverses each pair of states, so `flow - t(flow)` compares the two directions.

Quiz yourself!

What does a positive entry in `flow - t(flow)` mean? Why are its diagonal entries zero even when detailed balance fails? Would zero column sums of this difference be enough to establish detailed balance?

More diary keepers go from happy to neutral each day than from neutral to happy, more go from neutral to sad than back, and more go from sad to happy than the other way. The flows do not match pairwise; instead there is a steady circulation happy → neutral → sad → happy, of about 0.024 of the population per day, and every state still receives as much as it gives. A chain satisfying detailed balance is **reversible** when started in its stationary distribution. Reversing the order of a stationary sequence does not change its joint probability law. The mood model has a stationary distribution but does not satisfy detailed balance, so reversibility is not required for stationarity.

3.5.1 What detailed balance guarantees

Detailed balance implies that the specified target distribution is stationary. It does **not** imply irreducibility or aperiodicity.

- The identity matrix leaves every state where it is. It satisfies detailed balance for any target vector, because all flows between different states are zero. Yet it is reducible and cannot explore the target.
- The alternating two-state chain satisfies detailed balance for $(0.5, 0.5)$, because the flow in either direction is 0.5. It is irreducible but periodic.

For a finite-state sampler, the checks therefore serve different purposes:

Check	What it establishes
Detailed balance with the target	The target is stationary
Irreducibility	There is a unique stationary distribution; time averages converge to its expectations
Irreducibility and aperiodicity	State distributions approach the stationary distribution from any start

3.5.2 Choosing transitions for a target

Consider a target with two states and probabilities $\pi = (0.25, 0.75)$. We can choose $P_{12} = 0.30$ and determine the reverse probability from detailed balance:

$$0.25 \times 0.30 = 0.75 \times P_{21}, \qquad P_{21} = 0.10.$$

Complete each row by assigning its remaining probability to staying in the same state:

$$\mathbf{P}_* = \begin{pmatrix} 0.70 & 0.30 \\ 0.10 & 0.90 \end{pmatrix}.$$

Both cross-state flows are 0.075. Direct multiplication confirms that $(0.25, 0.75)\mathbf{P}_* = (0.25, 0.75)$. The chain is also irreducible, because both cross-state transitions have positive probability, and aperiodic, because it can stay in a state.

The choice is not unique. Smaller cross-state transition probabilities can satisfy the same equation but produce a slower-moving chain. In general, for positive target probabilities and a pair of permitted moves, detailed balance can be written

$$\frac{P_{ij}}{P_{ji}} = \frac{\pi_j}{\pi_i}.$$

If the target is known up to a common normalizing constant, that constant cancels in the ratio on the right. We can therefore design transitions using relative target weights. The grid construction below applies this idea to a posterior.

Quiz yourself!

A chain has only two states, with $P_{12} = a$ and $P_{21} = b$, both positive. Find its stationary distribution, and show that every such two-state chain satisfies detailed balance. Why is the three-state mood chain different?

3.6 From transition matrices to transition kernels

Most parameters in this book take values in intervals or in a continuous space with several coordinates. We can no longer list a transition probability for each possible next value in a finite matrix. Instead, a **transition kernel** specifies the distribution of the next state given the current state.

3.6.1 A Normal transition example

Suppose the transition rule is

$$\theta^{(t+1)} = 0.6\theta^{(t)} + 0.8Z^{(t+1)}, \qquad Z^{(t+1)} \sim \mathrm{N}(0, 1),$$

where each new $Z^{(t+1)}$ is independent of the earlier states and random draws. Given a current value θ, the next state has a Normal distribution with mean 0.6θ and standard deviation 0.8:

$$\theta^{(t+1)} \mid \theta^{(t)} = \theta \sim \mathrm{N}(0.6\theta, 0.8^2).$$

For example, from $\theta = 1$, the next-state distribution is $\mathrm{N}(0.6, 0.8^2)$. We can generate possible next values or calculate an interval probability:

```
set.seed(84735)
current <- 1
rnorm(5, mean = 0.6 * current, sd = 0.8)
#> [1] 1.1337862 0.5009278 1.2246941 0.7895346 1.2372088
pnorm(1, mean = 0.6 * current, sd = 0.8) -
  pnorm(0, mean = 0.6 * current, sd = 0.8)
#> [1] 0.4648351
```

The five draws are independent possible next states from the **same** current value; they are not five consecutive states of a chain. The last calculation gives a probability of about 0.465 that the next state lies between zero and one. To generate a chain, use each new state as the current value for the next transition.

```
kernel_curves <- crossing(current = c(-1, 0, 1), next_value = seq(-3, 3, length.out =
  501)) |>
  mutate(density = dnorm(next_value, mean = 0.6 * current, sd = 0.8),
         current = factor(current))
ggplot(kernel_curves, aes(next_value, density, color = current, linetype = current)) +
  geom_line(linewidth = 0.8) +
  scale_color_manual(values = c("#9E7BB5", "#552E6B", "gray40"), name = "Current
    state") +
  labs(x = "Next state", y = "Conditional density", linetype = "Current state")
```

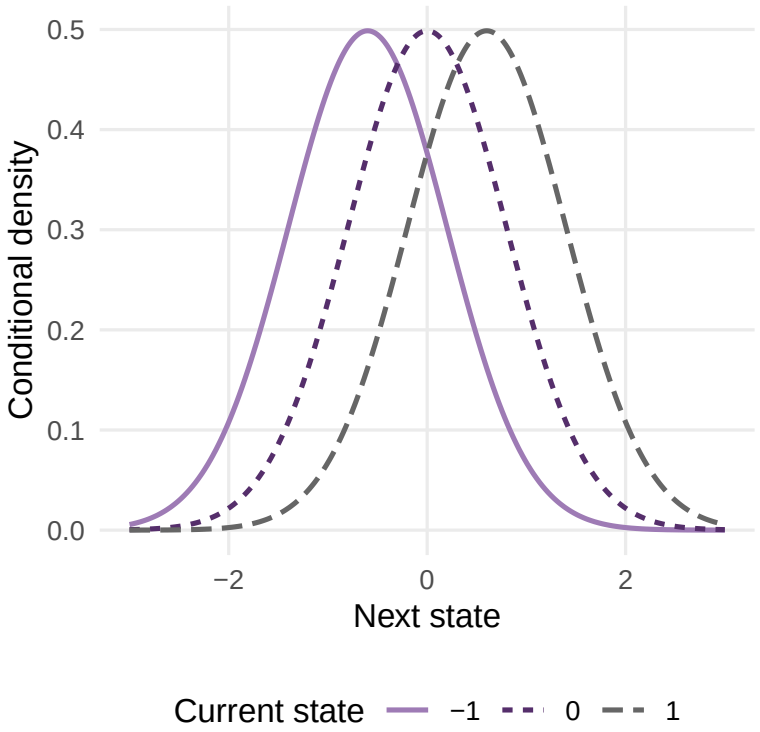


Figure 3.6: Conditional densities for the Normal transition rule, from current states -1, 0, and 1. Each curve integrates to one and describes a possible next value. Changing the current state shifts the mean to 0.6 times that state. These conditional densities are different from a stationary density.

A useful property of Normal variables

If X and Z are independent Normal variables, then $aX + bZ$ is also Normal. Its mean and variance are

$$\mathbb{E}[aX + bZ] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Z],$$

$$\mathrm{Var}(aX + bZ) = a^2\,\mathrm{Var}(X) + b^2\,\mathrm{Var}(Z).$$

Independence is essential here: without it, the variance also includes a covariance term.

To check whether the standard Normal distribution is stationary, suppose the current state has distribution $\theta^{(t)} \sim \mathrm{N}(0, 1)$. The new draw $Z^{(t+1)} \sim \mathrm{N}(0, 1)$ is independent of that state. By the property above, the next state,

$$\theta^{(t+1)} = 0.6\theta^{(t)} + 0.8Z^{(t+1)},$$

is also Normal. Its mean is zero, and its variance is

$$0.6^2 \times 1 + 0.8^2 \times 1 = 0.36 + 0.64 = 1.$$

Thus $\theta^{(t+1)} \sim \mathrm{N}(0, 1)$: one transition preserves the standard Normal distribution, establishing stationarity. The individual values still change; it is their distribution that stays the same.

3.6.2 When a transition has a density

When the next-state distribution has a density, write it as $K(\theta, \theta')$, with θ the current state and θ' the next state. Fixing θ gives the continuous counterpart of one row of a transition matrix. For the Normal example,

$$K(\theta, \theta') = \frac{1}{0.8\sqrt{2\pi}} \exp\left[-\frac{(\theta' - 0.6\theta)^2}{2(0.8)^2}\right].$$

A density value is not a transition probability. The probability of reaching an interval is an area:

$$P(a < \theta^{(t+1)} < b \mid \theta^{(t)} = \theta) = \int_a^b K(\theta, \theta')\, d\theta'.$$

For kernels that have such a density, the finite-state equations take the following forms:

Finite states	Transitions with a density
P_{ij} is a probability of the next state	$K(\theta, \theta')$ is a density for the next state
$\sum_j P_{ij} = 1$	$\int K(\theta, \theta')\, d\theta' = 1$
$\sum_i \pi_i P_{ij} = \pi_j$	$\int f(\theta) K(\theta, \theta')\, d\theta = f(\theta')$
$\pi_i P_{ij} = \pi_j P_{ji}$	$f(\theta) K(\theta, \theta') = f(\theta') K(\theta', \theta)$

Here f is the candidate stationary density. Detailed balance implies stationarity by integration, just as it did by summation:

$$\begin{aligned}\int f(\theta) K(\theta, \theta')\, d\theta &= \int f(\theta') K(\theta', \theta)\, d\theta \\ &= f(\theta') \int K(\theta', \theta)\, d\theta = f(\theta').\end{aligned}$$

The final integral is one because, with θ' fixed, $K(\theta', \theta)$ integrates over all possible next states θ.

3.7 A first hand-built chain

We now use a Markov chain as a computational tool. Its states will be candidate parameter values, rather than observed moods. We use detailed balance to preserve a target posterior approximation, then check irreducibility and aperiodicity. Keeping the state space finite lets us verify the complete transition matrix.

Consider a survey asking what proportion π of students had a sad day yesterday. You ask $n = 100$ students and 14 say yes, so $Y \mid \pi \sim \text{Bin}(100, \pi)$ with $Y = 14$. For a prior, suppose you expected sad days to be fairly common, say around 40% but with real uncertainty, and chose $\pi \sim \text{Beta}(4, 6)$. The Beta-Binomial model then gives the posterior $\pi \mid Y = 14 \sim \text{Beta}(18, 92)$. The known Beta posterior provides a benchmark for the numerical calculation. This survey model is separate from the transition model for an individual's daily moods.

Use $G = 100$ grid midpoints between zero and one, as in Chapter 2. Evaluate the unnormalized posterior on the log scale. For plotting and matrix checks, also form relative weights by subtracting their maximum before exponentiating:

```
G <- 100
grid <- (seq_len(G) - 0.5) / G
log_target <- dbeta(grid, 4, 6, log = TRUE) +
  dbinom(14, size = 100, prob = grid, log = TRUE)
target <- exp(log_target - max(log_target))
```

Index g labels a **position** on the grid, whereas `grid[g]` is the corresponding **parameter value**. For example, index 51 corresponds to $\pi = 0.505$. These density evaluations provide approximate cell weights; the equal cell widths cancel when the weights are normalized. Normalizing them gives `target / sum(target)`, but the transition rule will use only differences of `log_target` values.

3.7.1 The design

A small example makes the rule concrete. Suppose three grid positions have relative target weights $(1, 2, 1)$, or normalized probabilities $(0.25, 0.50, 0.25)$. At each transition, propose one position to the left or right, each with probability $1/2$.

- From position 1, a left proposal is outside the grid, so the chain stays at 1. A right proposal reaches the larger weight at position 2 and is always accepted. Thus $P_{11} = 0.5$ and $P_{12} = 0.5$.
- From position 2, either neighbor has half the current weight. Accept either proposal with probability 1/2. The probability of reaching each neighbor is therefore $(1/2)(1/2) = 0.25$. The remaining probability, 0.50, belongs to staying at position 2.
- Position 3 is the mirror image of position 1.

The resulting matrix is

$$\mathbf{P}_{\text{small}} = \begin{pmatrix} 0.50 & 0.50 & 0 \\ 0.25 & 0.50 & 0.25 \\ 0 & 0.50 & 0.50 \end{pmatrix}.$$

For positions 1 and 2, the two stationary flows are $0.25 \times 0.50 = 0.125$ and $0.50 \times 0.25 = 0.125$. The same equality holds for positions 2 and 3. There are no direct moves between positions 1 and 3. Detailed balance therefore holds for the target vector.

The same construction works with G grid positions and positive weights w_g. From the current index g:

1. **Propose.** Draw $g^\star = g - 1$ or $g^\star = g + 1$ with equal probability. If the proposal is outside $1, \ldots, G$, record g again.
2. **Accept or repeat.** For a valid proposal, move with probability $\min\{1, w_{g^\star}/w_g\}$. If it is not accepted, record g again.

For neighboring indices g and j, the proposal probability is 1/2 in both directions. Multiplying by the target weight gives

$$w_g P_{gj} = \tfrac{1}{2} w_g \min\{1, w_j/w_g\} = \tfrac{1}{2} \min\{w_g, w_j\} = w_j P_{jg}.$$

Normalizing the weights would divide both sides by the same sum, so detailed balance holds for the normalized target. Non-neighboring pairs have zero transition probability in both directions, and self-transitions satisfy the equality automatically.

Two additional checks complete the finite-state argument. Positive weights make every neighbor move possible, so sequences of neighbor moves connect all grid positions: The chain is irreducible. An outward proposal at either endpoint leaves the state unchanged with positive probability. Since the chain is irreducible and has a self-loop, it is aperiodic. The state distribution therefore approaches the normalized grid target from any start.

3.7.2 Checking the design

For the three-position example, we can verify the stationary equation directly:

```
P_small <- matrix(c(0.50, 0.50, 0,
                    0.25, 0.50, 0.25,
                    0, 0.50, 0.50), nrow = 3, byrow = TRUE)
c(0.25, 0.50, 0.25) %*% P_small
#>      [,1] [,2] [,3]
#> [1,] 0.25  0.5 0.25
```

For the posterior grid, the following function builds the full transition matrix. Each off-diagonal neighbor probability is the proposal probability, 1/2, multiplied by the acceptance probability. The remaining row probability is assigned to staying at the current index. The function takes a vector of log weights, so an acceptance probability is computed as `exp(min(0, log_ratio))`.

```
build_walk <- function(log_target) {
  G <- length(log_target)
  P <- matrix(0, nrow = G, ncol = G)
  for (g in seq_len(G)) {
    if (g > 1) {
      P[g, g - 1] <- 0.5 * exp(min(0, log_target[g - 1] - log_target[g]))
    }
```

```
    if (g < G) {
      P[g, g + 1] <- 0.5 * exp(min(0, log_target[g + 1] - log_target[g]))
    }
    P[g, g] <- 1 - sum(P[g, ])
  }
  P
}
P_walk <- build_walk(log_target)
```

Check the row sums, the stationary equation, and the stationary vector calculated independently by `stationary()`:

```
grid_prob <- target / sum(target)
range(rowSums(P_walk))
#> [1] 1 1
max(abs(grid_prob %*% P_walk - grid_prob))
#> [1] 1.387779e-17
max(abs(stationary(P_walk) - grid_prob))
#> [1] 1.026956e-15
```

The first result should be a range whose endpoints are both one. The other two results should be close to zero, up to numerical rounding. These calculations check the transition rule on the finite grid; they do not remove the approximation involved in representing a continuous posterior by grid cells.

3.7.3 Running the chain

The matrix is useful for verification, but simulation can apply the transition rule directly. The function below first records the current index as the next one. It changes that entry only if a valid proposal is accepted. Repeated values are therefore included automatically.

```
walk_grid <- function(log_target, init, n_iter) {
  G <- length(log_target)
  g <- integer(n_iter + 1)
  g[1] <- init
  for (t in seq_len(n_iter)) {
    g[t + 1] <- g[t]
    proposal <- g[t] + sample(c(-1L, 1L), size = 1)
    if (proposal >= 1 && proposal <= G) {
      log_alpha <- min(0, log_target[proposal] - log_target[g[t]])
      if (log(runif(1)) < log_alpha) g[t + 1] <- proposal
    }
  }
  g
}
```

When the proposed log weight is at least the current one, `log_alpha` is zero and the proposal is always accepted. Otherwise, the comparison accepts with the appropriate weight ratio. No normalizing constant is needed. The samplers of later chapters follow the same calling pattern, `sampler(log_target, init, n_iter, ...)`, but return a tibble of draws rather than a bare vector.

Start at index 51, corresponding to $\pi = 0.505$, and make `n_iter = 50000` transitions. The returned vector includes the initial state, so it contains 50,001 indices.

```
set.seed(84735)
walk <- walk_grid(log_target, init = 51, n_iter = 50000)
```

```
draws <- tibble(step = 0:50000, pi = grid[walk])
```

```
p_trace <- draws |>
  filter(step <= 500) |>
  ggplot(aes(step, pi)) +
  geom_line(color = "#552E6B", linewidth = 0.4) +
  labs(x = "Step", y = expression(pi))

p_hist <- draws |>
  filter(step > 500) |>
  count(pi, name = "count") |>
  mutate(frequency = count / sum(count)) |>
  ggplot(aes(pi, frequency)) +
  geom_col(fill = "#ECECEC", color = "white", width = 0.01) +
  geom_line(data = tibble(pi = grid, frequency = grid_prob), color = "#552E6B") +
  geom_point(data = tibble(pi = grid, frequency = grid_prob), color = "#552E6B", size
    = 0.8) +
  coord_cartesian(xlim = c(0.05, 0.35)) +
  labs(x = expression(pi), y = "Probability")

wrap_plots(p_trace, p_hist)
```

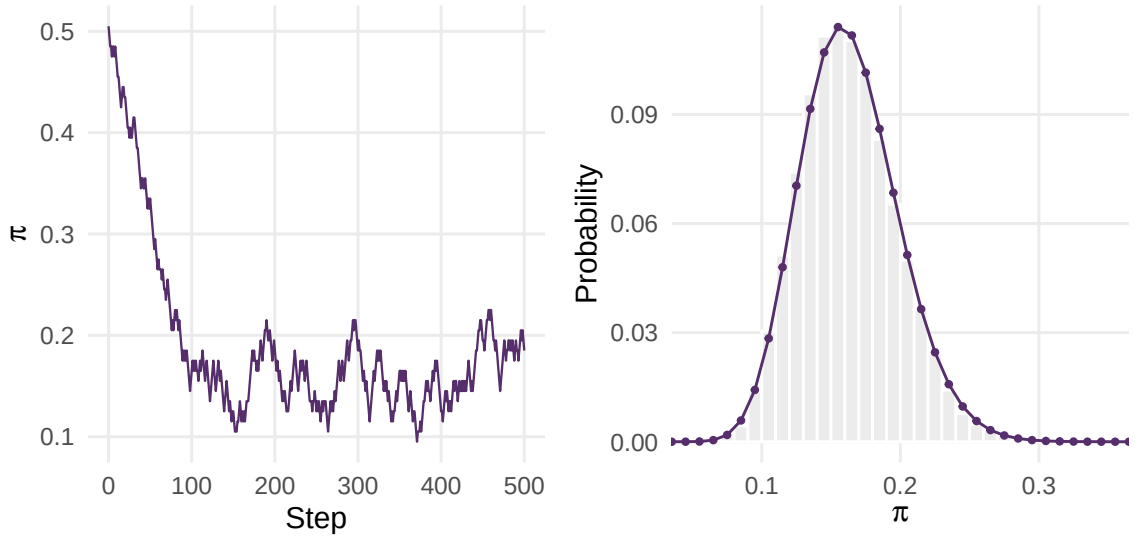


Figure 3.7: Left: steps 0 to 500 of the hand-built chain, started at $\pi = 0.505$. Repeated values appear as horizontal stretches. Right: frequencies after the first 500 transitions (bars), compared with the normalized grid target (line and dots).

Compare the empirical frequencies in Figure 3.7 with the normalized grid target. The following code also compares posterior summaries after omitting the first 500 transitions with the continuous Beta benchmark. The initial omission is illustrative; Chapter 4 develops ways to assess simulations more systematically.

```
draws |>
  filter(step > 500) |>
  summarize(mean = mean(pi), lower = quantile(pi, 0.025), upper = quantile(pi, 0.975))
#> # A tibble: 1 x 3
```

```
#>    mean lower upper
#>   <dbl> <dbl> <dbl>
#> 1 0.162 0.105 0.235
c(mean = 18 / 110, lower = qbeta(0.025, 18, 92), upper = qbeta(0.975, 18, 92))
#>      mean     lower     upper
#> 0.1636364 0.1009084 0.2379286
```

Differences from the continuous Beta values can arise from the finite grid, Monte Carlo variation, and any remaining effect of the starting state. Neighbor moves also produce persistent successive values, so counting $N = 49{,}500$ retained states does not describe their precision as if they were independent. Chapter 4 develops the corresponding Monte Carlo error calculations.

The construction used relative log weights, neighbor proposals, and an acceptance step. The normalization was needed for the matrix check and the plotted benchmark, but not inside `walk_grid()`. For a different prior and likelihood, the same transition rule can be applied to new finite log weights. Its target would still be a grid approximation. Chapter 5 develops the corresponding method on a continuous parameter space.

Quiz yourself!

Suppose you replaced the coin flip in step 1 by a biased one that proposes $g + 1$ with probability 0.7 and $g - 1$ with probability 0.3, keeping the acceptance rule the same. Does detailed balance still hold? What would the chain's stationary distribution look like relative to the target?

3.8 Chapter summary

A Markov chain can describe a sequence of observations or serve as a computational sequence for sampling a target distribution.

- A finite-state chain is specified by its states, **initial distribution**, and **transition matrix**. Given the current state, the next-state distribution does not depend on the earlier history. Entry (i, j) of $\mathbf{P}^t$ gives a t-transition probability.
- The row vector $\mu^{(t)} = \mu^{(0)}\mathbf{P}^t$ describes the distribution at time t. It is distinct from the realized state of an individual chain.
- A **stationary distribution** satisfies $\pi\mathbf{P} = \pi$. A finite irreducible chain has a unique stationary distribution. With aperiodicity as well, state distributions approach it from any start.
- The finite-state **ergodic theorem** establishes convergence of time averages under irreducibility, even for periodic chains. It does not by itself quantify error in a finite simulation.
- **Detailed balance** equates the target-weighted flows for each pair of states and implies stationarity. It implies neither irreducibility nor aperiodicity; those require separate checks.
- A **transition kernel** specifies a next-state distribution. When that distribution has a density, sums in the finite-state balance equations become integrals. A rule that can repeat the current value also requires a point mass.
- The grid construction preserves its normalized target using only relative weights. Its output approximates continuous posterior summaries with both discretization and simulation error.

3.9 Quiz answers

Quiz 1. Sum over the intermediate day: $P_{\text{happy,happy}}P_{\text{happy,sad}} + P_{\text{happy,neutral}}P_{\text{neutral,sad}} + P_{\text{happy,sad}}P_{\text{sad,sad}} = 0.70 \times 0.05 + 0.25 \times 0.30 + 0.05 \times 0.40 = 0.035 + 0.075 + 0.020 = 0.13$, which is the (happy, sad) entry of `mat_power(P, 2)`.

Quiz 2. The chain cycles $1 \to 2 \to 3 \to 1$ with certainty, so every state is reachable from every other and the chain is irreducible. Returns to a state take exactly 3, 6, 9, … steps, so the period is 3 and the chain is

not aperiodic. It has a unique stationary distribution, $\pi = (1/3, 1/3, 1/3)$, but $\mu^{(0)}\mathbf{P}^t$ does not converge to it: started in state 1, the chain is in state 1 with certainty at $t = 0, 3, 6, \ldots$. The long-run fraction of time in each state does converge to 1/3.

Quiz 3. Entry (i, j) compares the stationary flow from i to j with the reverse flow. A positive difference means the former is larger. Diagonal entries subtract a value from itself and are always zero, so they cannot establish detailed balance. Zero column sums express overall flow balance at each state, not equality for every pair; the mood chain demonstrates this distinction.

Quiz 4. Stationarity requires $\pi_1(1-a)+\pi_2 b = \pi_1$, that is $\pi_1 a = \pi_2 b$, together with $\pi_1 + \pi_2 = 1$, which gives $\pi = (b, a)/(a + b)$. The equation $\pi_1 a = \pi_2 b$ is exactly detailed balance for the only pair of distinct states, so every two-state chain with positive a and b is reversible. With three states, balance only constrains the sum of the flows into each state, and a circulation $1 \to 2 \to 3 \to 1$ can balance every state without balancing any pair; the mood chain has such a circulation.

Quiz 5. No. With the biased coin, $P_{g,g+1} = 0.7\min\{1, \text{target}[g+1]/\text{target}[g]\}$ but $P_{g+1,g} = 0.3\min\{1, \text{target}[g]/\text{target}[g+1]\}$, so $\text{target}[g]P_{g,g+1} = 0.7\min\{\cdot\}$ while $\text{target}[g+1]P_{g+1,g} = 0.3\min\{\cdot\}$ with the same minimum; they differ by a factor 7/3. The chain is pushed to the right, so its stationary distribution is shifted toward larger π relative to the target. The proposal must be symmetric for this acceptance rule to work; Chapter 6 (Metropolis–Hastings and Adaptive Proposals) shows how to fix the rule when it is not.

3.10 Exercises

3.10.1 Conceptual exercises

Exercise 3.1 (Stationary or not?) For each of the following transition matrices, state whether the chain is irreducible, whether it is aperiodic, and whether the given vector is a stationary distribution. Justify each answer in one sentence.

a. $\mathbf{P} = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}$, $\pi = (0.5, 0.5)$.

b. $\mathbf{P} = \begin{pmatrix} 1 & 0 \\ 0.3 & 0.7 \end{pmatrix}$, $\pi = (1, 0)$.

c. $\mathbf{P} = \begin{pmatrix} 0 & 0.5 & 0.5 \\ 0.5 & 0 & 0.5 \\ 0.5 & 0.5 & 0 \end{pmatrix}$, $\pi = (1/3, 1/3, 1/3)$.

d. $\mathbf{P} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\pi = (0.3, 0.7)$.

Exercise 3.2 (Markov or not?) For each sequence below, say whether it is a Markov chain on the state space indicated, and if it is not, whether it becomes one after enlarging the state.

a. Your running total of correct answers on a 40-item test, one item at a time, with each item answered correctly with probability 0.8 independently of the others. State space: the integers $0, \ldots, 40$.

b. Your mood each day, when tomorrow's mood depends on today's *and* yesterday's. State space: $\{\text{happy}, \text{neutral}, \text{sad}\}$.

c. The number of distinct participants who have shown up so far, as a study recruits one participant at a time from a pool of 50, each time picking uniformly from the whole pool (so repeats are possible). State space: $0, \ldots, 50$.

d. The current streak of consecutive happy days in the mood chain of this chapter. State space: $0, 1, 2, \ldots$. For a zero streak, compare the probabilities of a happy next day from the neutral and sad states before deciding whether additional information is needed.

Exercise 3.3 (Balance without detailed balance) Consider the chain on three states with $P_{12} = P_{23} = P_{31} = 0.9$ and $P_{11} = P_{22} = P_{33} = 0.1$, all other entries zero.

a. Show that $\pi = (1/3, 1/3, 1/3)$ is stationary by checking $\pi\mathbf{P} = \pi$.
b. Show that detailed balance fails for the pair $(1, 2)$.
c. Describe in words what a film of this chain run backwards would look like, and why it would be recognizably different from the forward film.

Exercise 3.4 (Rewriting the proof) Assume the transition kernel has an ordinary conditional density, as in the Normal example. The detailed balance proof for finite states summed over the *first* index. Redo it for a continuous state space by starting from $f(\theta')K(\theta', \theta) = f(\theta)K(\theta, \theta')$ and integrating over θ' instead of θ. Which stationarity equation do you get, and is it the same one?

3.10.2 Practical exercises

Exercise 3.5 (How long to forget?) Use `mat_power()` and the mood transition matrix `P` of this chapter.

a. Find the smallest t for which every entry in each row of $\mathbf{P}^t$ differs from the corresponding entry of its first row by less than 0.001. Use the same numerical criterion in part b.
b. Change the first row of $\mathbf{P}$ to $P_{\text{happy,happy}} = 0.95$, $P_{\text{happy,neutral}} = 0.04$, and $P_{\text{happy,sad}} = 0.01$, so that the row still sums to one, and repeat. Explain the difference in terms of how sticky the happy state has become.
c. For both matrices, compute the largest absolute value among the eigenvalues other than 1 with `eigen()` and relate it to your answers in a. and b.

Exercise 3.6 (Averages along a periodic chain) Simulate the alternating chain `P_swing` from a happy initial state for 1,000 transitions, retaining the initial state at time zero.

a. Plot $P(X^{(t)} = \text{happy})$ from `mat_power()` for $t = 0, \dots, 10$ and the running fraction of happy days from your simulation on separate panels.
b. One converges and one does not. Which is which, and which theorem of this chapter explains each?
c. Now let $P_{\text{happy,happy}} = 0.01$ and $P_{\text{happy,sad}} = 0.99$, keeping the second row equal to $(1, 0)$. Repeat a. Check the new stationary distribution; do not assume it remains exactly $(0.5, 0.5)$. What changed and why?

Exercise 3.7 (A wider step on the grid) Modify `walk_grid()` so that each transition proposes one of $g - 2, g - 1, g + 1, g + 2$ with equal probability.

a. Show, by the argument of the chapter, that detailed balance still holds.
b. Modify `build_walk()` to match, and check numerically that `grid_prob` is stationary for the new matrix. Both functions take `log_target`. Keep the probability of each of the four proposed offsets equal to 1/4, including offsets that leave the grid; such proposals leave the state unchanged.
c. Run both versions for 50,000 transitions from `init = 51` and compare the first time each reaches a grid point below $\pi = 0.25$. Repeat with several seeds. Explain why a shorter first-passage time alone does not establish convergence of the full state distribution.

Exercise 3.8 (A chain for a bimodal target) Define a log target on the same grid with `log_target2 <- log(dnorm(grid, 0.25, 0.04) + dnorm(grid, 0.75, 0.04))`. It represents two separated modes

with equal target probability.

a. Build the matrix with `build_walk(log_target2)` and check that its stationary distribution is proportional to `exp(log_target2)`. Normalize by subtracting the maximum log weight before exponentiating.
b. Run `walk_grid(log_target2, init = 25, n_iter = 20000)` and plot the trace. How often does the chain cross from one bump to the other?
c. Compute the fraction of steps spent below $\pi = 0.5$ and compare it with the target's mass there. Explain the discrepancy using the distinction between a stationary distribution existing and a chain having reached it.

3.10.3 Applied exercises

Exercise 3.9 (Estimating a transition matrix) The `weather_australia` data in the `bayesrules` package record, for sampled days in three Australian locations, whether it rained today (`raintoday`) and the next day (`raintomorrow`). Load them with `data("weather_australia", package = "bayesrules")`, inspect the category labels, and omit pairs with either rain indicator missing before tabulating transitions. Treat each pair as one observed transition of a two-state chain with states *rain* and *no rain*.

a. For Wollongong, tabulate the transitions and estimate the transition matrix $\mathbf{P}$ from the row proportions.
b. Compute the stationary distribution of your estimated chain with `stationary()` and compare π_{rain} with the overall proportion of rainy days among `raintoday`. Should they agree exactly? Why or why not?
c. Each row contains a probability of rain tomorrow given today's weather and its complement. For this exercise, use an independent Beta$(1, 1)$ prior for each of these two rain probabilities and update it with the corresponding row counts, as in Chapter 2. This is a time-homogeneous two-state working model for the sampled transitions. Draw 1,000 transition matrices from the two posteriors, compute the stationary distribution of each, and report a 95% posterior interval for π_{rain}.
d. Repeat for Uluru. Which location's chain forgets yesterday's weather faster, and how can you tell from the transition matrices alone?

Chapter 4

Diagnosing Markov Chains

A Markov chain can have the right stationary distribution and still give a poor estimate after a thousand iterations. It may spend much of that time near its starting point, move slowly through the distribution, or explore only one of several regions. The convergence results of Chapter 3 describe what happens as a run grows indefinitely. In practice, we run a chain for a limited number of iterations and must decide whether the resulting estimates are accurate enough.

Suppose two chains each produce 20,000 values from a standard Normal target. Both aim at a distribution with mean zero and standard deviation one. If we repeated both simulations, would their averages vary equally much from run to run? And if one average is 0.04, is that a reason to worry? We will use this simple target to investigate what a chain's behavior tells us about the accuracy of its estimates.

We will then apply the same reasoning to a Bayesian analysis: estimating the proportion of Dutch-language songs on a radio station. There, we will need to judge the precision of a posterior mean and a credible interval. The central question throughout is **how much confidence we can place in a finite simulation**.

Run the following setup before working through the examples. We use `tidyverse` to organize and summarize the draws, `posterior` to calculate diagnostics, and `bayesplot` and `patchwork` to draw and combine the plots. The `rstan` package fits the radio model later in the chapter.

```
library(tidyverse)
library(rstan)
library(bayesplot)
library(patchwork)
library(posterior)
rstan_options(auto_write = TRUE)  # reuse compiled Stan models
options(mc.cores = 1)             # run the Stan chains one after another
color_scheme_set("purple")       # colors for the diagnostic plots
```

Goals

- Interpret trace plots and identify evidence of initialization effects, slow movement, and disagreement between chains.
- Explain how dependence affects the precision of a Monte Carlo average.
- Calculate and interpret effective sample size and Monte Carlo standard error, and use them to plan a longer run.
- Distinguish initialization bias, Monte Carlo variation, and posterior uncertainty.
- Compare several chains using plots and $\widehat{R}$, and explain the limits of agreement.
- Evaluate a Stan fit and justify whether its posterior summaries are precise enough for a stated purpose.

4.1 Inspecting a run

We begin with the Normal transition rule from Chapter 3. At each step, combine part of the current value with a new random draw:

$$\theta^{(t+1)} = \phi\theta^{(t)} + \sqrt{1-\phi^2}\,\varepsilon_t, \qquad \varepsilon_t \sim \mathrm{N}(0,1).$$

Each new ε_t is drawn independently. The parameter ϕ, with $-1 < \phi < 1$, controls how much of the current value carries over to the next. With $\phi = 0$, the next value is a fresh standard Normal draw. With ϕ close to one, successive values tend to stay close together. This rule is called a **first-order autoregressive process**, or **AR(1) process**.

These choices of ϕ preserve the same standard Normal target. To check this, suppose the current state is standard Normal. A weighted sum of independent Normal variables is also Normal, as we saw in Chapter 3. The next state has mean zero and variance

$$\phi^2 \times 1 + (1-\phi^2) \times 1 = 1.$$

It is therefore standard Normal too. Changing ϕ changes how the chain moves, while keeping its stationary distribution fixed.

The following function stores N states, including the initial state in `theta[1]`, and makes $N-1$ transitions:

```
ar1_chain <- function(phi, N, init = 0) {
  theta <- numeric(N)
  theta[1] <- init
  for (t in seq_len(N - 1)) {
    theta[t + 1] <- phi * theta[t] + sqrt(1 - phi^2) * rnorm(1)
  }
  theta
}
```

We generate three chains, with $\phi = 0$, 0.5, and 0.95. Each starts with a draw from the target, so its distribution is stationary from the start. This lets us study dependence without also introducing an effect of a fixed starting point.

```
set.seed(84735)
phis <- c(0, 0.5, 0.95)
N_ar1 <- 20000
ar1_chains <- lapply(phis, function(phi) {
  ar1_chain(phi, N_ar1, init = rnorm(1))
})
```

A **trace plot** shows the sampled values in the order they were produced. To draw one with `ggplot2`, put each chain in a tibble with an iteration column, stack the tibbles, and label each by its value of ϕ. In Figure 4.1, the three chains all fluctuate around zero, but they do so quite differently.

```
ar1 <- map2(ar1_chains, phis, \(theta, phi) tibble(iteration = seq_along(theta),
  theta, phi)) |>
  list_rbind() |>
  mutate(phi = factor(phi, labels = paste("phi =", phis)))
ar1 |>
  filter(iteration <= 500) |>
  ggplot(aes(iteration, theta)) +
  geom_line(color = "#552E6B", linewidth = 0.3) +
  facet_wrap(~ phi, ncol = 1) +
  labs(x = "Iteration", y = expression(theta))
```

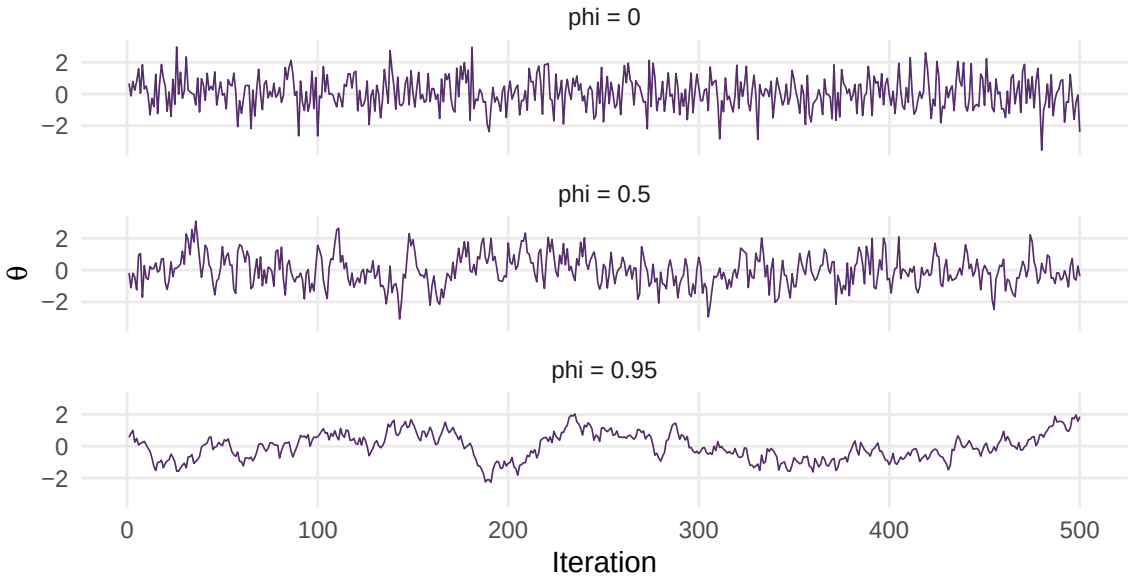


Figure 4.1: The first 500 iterations of three chains that all have the N(0, 1) stationary distribution. With $\phi = 0$ the draws are independent. With $\phi = 0.95$ the chain wanders slowly, and a short window can give an unrepresentative picture of the target.

With $\phi = 0$, values move frequently between the two sides of zero. Most lie near the center, with occasional values farther away, as we expect for a standard Normal distribution. This is useful behavior when estimating the mean: the run samples both sides without spending long stretches on either one. With $\phi = 0.95$, values tend to stay above or below zero for stretches of time. This chain has the same standard Normal stationary distribution, but a short stretch may contain mostly positive or mostly negative values. Its average can therefore be far from zero.

4.1.1 The starting point

So far we have drawn the initial state from the target. Usually we cannot do that: obtaining posterior draws is the problem we are trying to solve. Instead, we choose a starting value and let the chain move from there. What happens if that value is far from the region where most of the target probability lies?

Consider four chains with $\phi = 0.99$, started at -6, -3, 3, and 6. The standard Normal target puts most of its probability between -2 and 2. Figure 4.2 shows how these chains move from their starts toward that region.

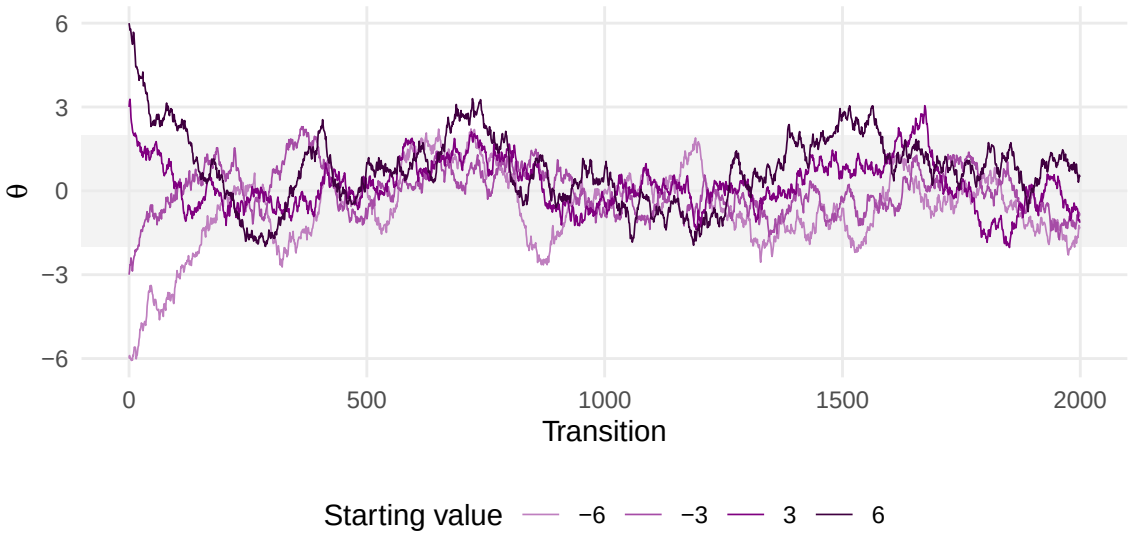


Figure 4.2: Four AR(1) chains with phi = 0.99, started at -6, -3, 3, and 6. The shaded band marks the interval from -2 to 2. Early values reflect the chosen starting points; later values fluctuate through a similar region.

Early values can pull the estimated mean toward the starting point. **Burn-in** is the practice of discarding an initial part of the run to reduce this influence. A trace plot can reveal that early values are still moving away from the starting point. It cannot establish exactly when the effect of initialization has become negligible.

Some sampling algorithms also adjust their settings during an initial **warm-up** phase. Stan, which we use later in this chapter, does this. Its warm-up therefore serves two purposes: allowing movement away from the start and learning suitable sampling settings. Posterior summaries use the subsequent phase, with those settings fixed.

4.1.2 Warning patterns

The same questions arise with a posterior whose shape we do not know. Does the chain keep drifting? Does it remain near one value for long stretches? Do different chains explore different regions? Figure 4.3 illustrates these three patterns on a proportion scale.

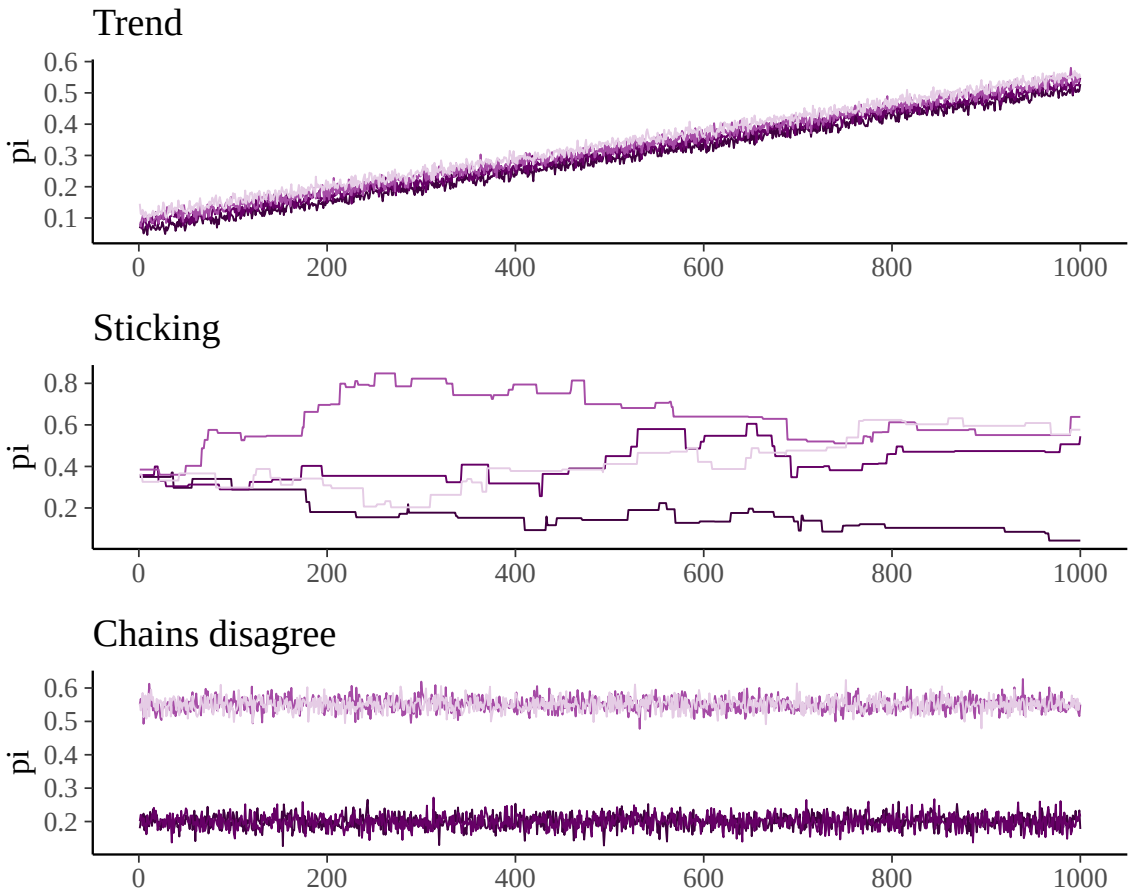


Figure 4.3: Three exaggerated synthetic warning patterns. Top: upward drift. Middle: repeated values and slow movement. Bottom: different centers across records. Real sampling problems can be much subtler, as in the exercises.

In the **trend** example, later values are consistently higher than earlier values, so the average depends strongly on where we stop. In the **sticking** example, long flat stretches contribute little new information. In the **disagreement** example, each record looks stable on its own, but the records give different answers about where the distribution lies.

These are deliberately constructed illustrations. The first adds an upward trend to random fluctuations. The second usually repeats its previous value and occasionally moves. The third generates values around two different centers. Their purpose is to make the warning patterns easy to recognize; the exercises contain subtler examples.

For an actual sampler, the pattern guides the next step. A chain still moving away from its start may need more warm-up. Slow movement may call for different sampling settings. Persistent separation between chains calls for investigation of why they are not exploring the same regions. We will encounter these problems when we build samplers in the coming chapters.

Quiz yourself!

A friend runs one very long chain of 100,000 iterations rather than four chains of 25,000. Their trace plot is a flat band of noise and they conclude that the chain has converged. Which of the three warning patterns could their design not have detected, and why?

4.2 Dependence and the accuracy of an average

Consecutive draws in a Markov chain are usually correlated: Where the chain is now tells you something about where it will be next. A chain in which that correlation dies away quickly is said to **mix well**; a chain that stays correlated over many iterations mixes poorly. Mixing affects how precisely we can estimate posterior summaries from a chain of a given length.

4.2.1 How far does the dependence persist?

The AR(1) traces show that nearby values can resemble one another. Compare each value with the next one, then with the value two steps later, and so on. The separation is called the **lag**. Lag 1 compares neighbors; lag 10 compares values ten iterations apart. The correlation at a given lag is called **autocorrelation**.

For a stationary chain, the lag-k autocorrelation is

$$\rho_k = \mathrm{Corr}(\theta^{(t)}, \theta^{(t+k)}).$$

A value near one means that high values tend to be followed by high values, and low values by low values, at that lag. A negative value indicates a tendency to lie on opposite sides of the mean. The **autocorrelation function** (ACF) plots these correlations against the lag.

For our AR(1) process, $\rho_k = \phi^k$: each step carries forward a fraction ϕ of the preceding state. Thus when $\phi = 0.95$, lag-one correlation is 0.95, lag-two correlation is 0.95^2, and lag-ten correlation is $0.95^{10} \approx 0.60$. Values ten steps apart still tend to resemble each other.

R estimates the autocorrelations from a simulated chain with `acf()`. For example, `acf(ar1_chains[[3]])` plots them for the chain with $\phi = 0.95$. Figure 4.4 compares all three chains.

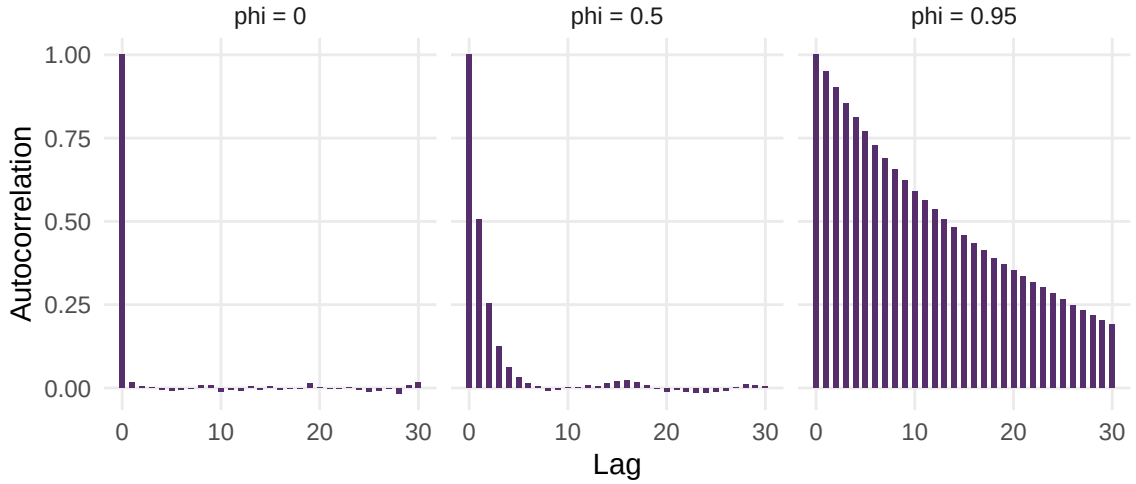


Figure 4.4: Estimated autocorrelations of the three Normal chains. Dependence disappears immediately when phi = 0, falls quickly when phi = 0.5, and persists across many lags when phi = 0.95. Lag zero compares every value with itself and therefore has correlation one.

4.2.2 Repeating the simulation

To see what this means for an estimate, imagine repeating each simulation 200 times and recording its average each time. We use 2,000 states per run, drawing each initial state from the standard Normal target. The target mean is zero; the spread of the 200 averages tells us how precisely each type of chain estimates it. Each value in the histograms below is an average from a whole run.

The calculation for $\phi = 0.95$ is:

```r
set.seed(84735)
means_95 <- replicate(200, mean(ar1_chain(0.95, 2000, init = rnorm(1))))
```

Figure 4.5 compares these averages with those from $\phi = 0$ and $\phi = 0.5$.

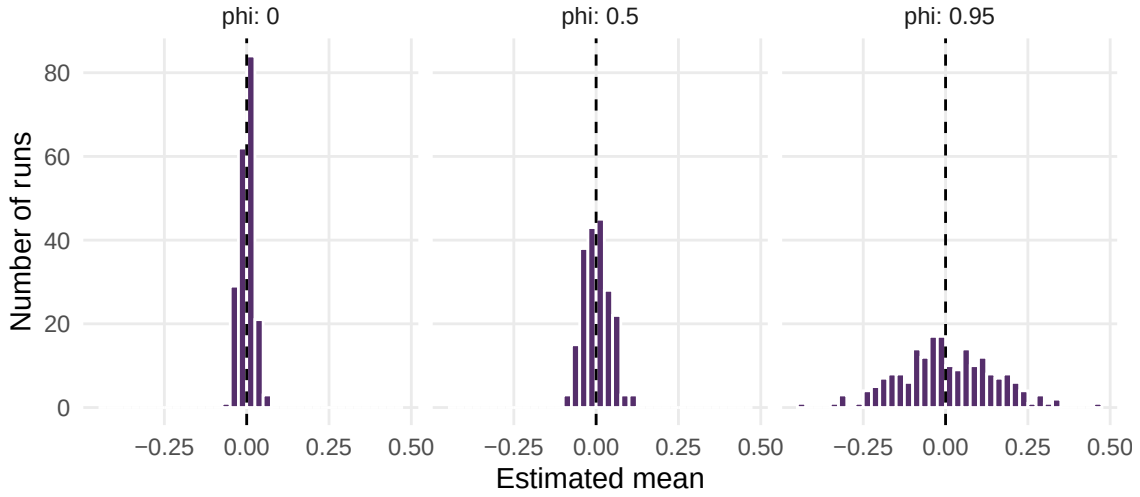


Figure 4.5: Estimated means from 200 independent runs for each value of phi, with 2,000 states per run. All three sets of estimates are centered near the true mean, zero, but stronger dependence gives more variation between runs. The same horizontal scale is used in all panels.

The chain with $\phi = 0.95$ can spend a substantial part of a run above or below zero. Its average therefore varies much more between runs than the average of independent draws. Each run contains the same number of values, but the resulting estimates are not equally precise. This is the cost of positive autocorrelation.

4.3 Effective sample size and Monte Carlo standard error

Suppose we estimate a target mean μ by the average of N states,

$$\bar{\theta} = \frac{1}{N} \sum_{t=0}^{N-1} \theta^{(t)}.$$

For independent draws, Chapter 1 gave $\mathrm{Var}(\bar{\theta}) = \sigma^2/N$, where σ^2 is the target variance. With positively correlated draws, neighboring values often pull the average in the same direction. The covariance terms account for this shared movement and increase the variance of the average.

For a stationary chain, the corresponding large-N approximation is

$$\mathrm{Var}(\bar{\theta}) \approx \frac{\sigma^2}{N} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right). \tag{4.1}$$

This assumes a finite target variance and autocorrelations that die away fast enough for their sum to be finite. The factor in parentheses describes the effect of dependence. For independent draws, all the ρ_k are zero and the factor is one. Positive autocorrelations make it larger.

We can express this loss of precision as an **effective sample size** (ESS): the number of independent draws that would give approximately the same spread of estimated means across repeated runs. Writing $\mathrm{Var}(\bar{\theta}) \approx \sigma^2/N_{\mathrm{eff}}$ gives

$$N_{\mathrm{eff}} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}.$$

For the AR(1) chain, $\rho_k = \phi^k$. Summing the geometric series gives

$$N_{\text{eff}} \approx \frac{N}{1 + 2\sum_{k=1}^{\infty} \phi^k} = \frac{N}{1 + 2\phi/(1-\phi)} = N\frac{1-\phi}{1+\phi}.$$

With $\phi = 0.95$, our 20,000-state chain has an effective sample size of approximately $20{,}000 \times 0.05/1.95 \approx 513$. Its average is about as precise as an average of 513 independent standard Normal draws.

In most applications we do not know the autocorrelations. The function `ess_basic()` in the `posterior` package estimates ESS from the sampled values. By default it first splits each chain into two halves, a safeguard we return to when comparing chains; `split = FALSE` keeps each of our single chains whole. Compare the estimates with the known AR(1) formula:

```
tibble(phi = phis,
       ess_formula = N_ar1 * (1 - phi) / (1 + phi),
       ess_estimated = sapply(ar1_chains, ess_basic, split = FALSE))
#> # A tibble: 3 x 3
#>     phi ess_formula ess_estimated
#>   <dbl>       <dbl>         <dbl>
#> 1  0         20000         19079.
#> 2  0.5        6667.         6706.
#> 3  0.95        513.          558.
```

The columns need not agree exactly. The formula uses known correlations and a long-run approximation; the function uses correlations estimated from a particular path. Repeating the simulation gives different ESS estimates, especially when dependence is strong. Section C.1 derives the exact variance for a chain of N states and explains the approximation.

4.3.1 Expressing precision in the units of the estimate

ESS tells us how much precision we have relative to independent sampling. To express that precision in the units of the mean, take the square root of its variance:

$$\text{MCSE}(\bar{\theta}) \approx \frac{\sigma}{\sqrt{N_{\text{eff}}}}. \tag{4.2}$$

This is the **Monte Carlo standard error** (MCSE). It estimates the standard deviation of the averages we would obtain across repeated runs, as illustrated in Figure 4.5.

For the standard Normal target, $\sigma = 1$. The three 20,000-state runs therefore have approximate MCSEs of $1/\sqrt{20{,}000} = 0.007$ when $\phi = 0$, $1/\sqrt{6{,}667} = 0.012$ when $\phi = 0.5$, and $1/\sqrt{513} = 0.044$ when $\phi = 0.95$.

An estimated mean of 0.04 is thus less than one MCSE from zero for the slowest chain, but more than five MCSEs away for independent draws. An error of 0.04 would thus be quite ordinary for the slowest chain and unusual for independent draws. `mcse_mean()` estimates the MCSE from the draws:

```
tibble(phi = phis,
       estimated_mean = sapply(ar1_chains, mean),
       mcse = sapply(ar1_chains, mcse_mean))
#> # A tibble: 3 x 3
#>     phi estimated_mean    mcse
#>   <dbl>          <dbl>   <dbl>
#> 1  0          -0.00160 0.00721
#> 2  0.5        -0.0211  0.0122
#> 3  0.95        0.0517  0.0417
```

An MCSE is not a bound on the error in one run. Differences of one or two MCSEs from the target mean are unsurprising, and larger differences sometimes occur. We use MCSE to judge the expected scale of numerical error and to decide whether further simulation is worthwhile.

Suppose we need an MCSE of 0.02 for the Normal mean. We need approximately $N_{\text{eff}} = (1/0.02)^2 = 2{,}500$. For $\phi = 0.95$, this requires

$$N \approx 2{,}500 \frac{1 + 0.95}{1 - 0.95} = 97{,}500$$

stored states. Our current 20,000-state run is too short for that purpose. With $\phi = 0.5$, the same target precision requires about 7,500 states. Improving mixing can therefore save substantial computation.

More generally, when the sampler's behavior stays similar, multiplying the retained run length by four approximately halves the MCSE. A pilot run with MCSE e suggests multiplying its length by $(e/e_{\text{target}})^2$, where e_{target} is the MCSE we want to achieve. This is a planning estimate: check the MCSE again after the longer run.

4.3.2 Would thinning help?

The chain with $\phi = 0.95$ is highly correlated at lag 1 but less so at lag 10, since $0.95^{10} \approx 0.60$. That suggests a fix: Keep every tenth draw and throw the rest away. The retained draws are less correlated, so surely they are better? This is called **thinning**.

But lower correlation is only part of the comparison. We also have fewer draws. The values we discard were correlated with their neighbors, but they still contributed to estimating the target mean. After keeping every tenth value, we have 2,000 states with lag-one correlation 0.95^{10}. Their theoretical ESS is

$$2{,}000 \frac{1 - 0.95^{10}}{1 + 0.95^{10}} \approx 502,$$

slightly below the full run's ESS of 513. The two effects nearly cancel, leaving a slightly less precise average.

We can make the same comparison with estimated ESS:

```
theta_95 <- ar1_chains[[3]]
c(full = ess_basic(theta_95, split = FALSE),
  thinned = ess_basic(theta_95[seq(1, N_ar1, by = 10)], split = FALSE))
#>     full  thinned
#> 557.8409 521.3920
```

An estimated ESS can occasionally be larger after thinning because it is itself estimated with noise. In this example, the theoretical ESS decreases slightly. A practical reason to thin is storage: retaining every value can be costly in a large model. Thinning can also reduce the cost of later calculations. Otherwise, keep the full run.

Quiz yourself!

Two chains target the same posterior. Their stationary autocorrelations are $\rho_k = 0.9^k$ for chain A and $\rho_k = 0.3^k$ for chain B. If you can afford 10,000 iterations, which chain gives the more precise estimate of the posterior mean, and roughly by how much?

4.3.3 Bias and variation are different problems

Starting a chain far from the region where most posterior values lie can pull its estimated mean toward the starting value. This is **initialization bias**. Discarding enough early values can reduce this effect.

Even after the starting point has little influence, repeated simulations give different estimates. This is **Monte Carlo variation**. Retaining more draws generally reduces that variation, although strongly dependent draws improve precision more slowly than independent draws.

These two sources of error are related. A longer run can also reduce the influence of the starting point, because the early values make up a smaller share of the average. But running longer only helps if the chain explores the posterior: A chain that misses an important region can give a very precise estimate of the wrong answer (Margossian & Gelman, 2026).

We can see both effects by repeating the Normal simulation from the fixed starting value 6, using $\phi = 0.99$. In Figure 4.6 we vary how many early states we discard and how many later states we retain. Each panel contains the estimated means from 200 runs.

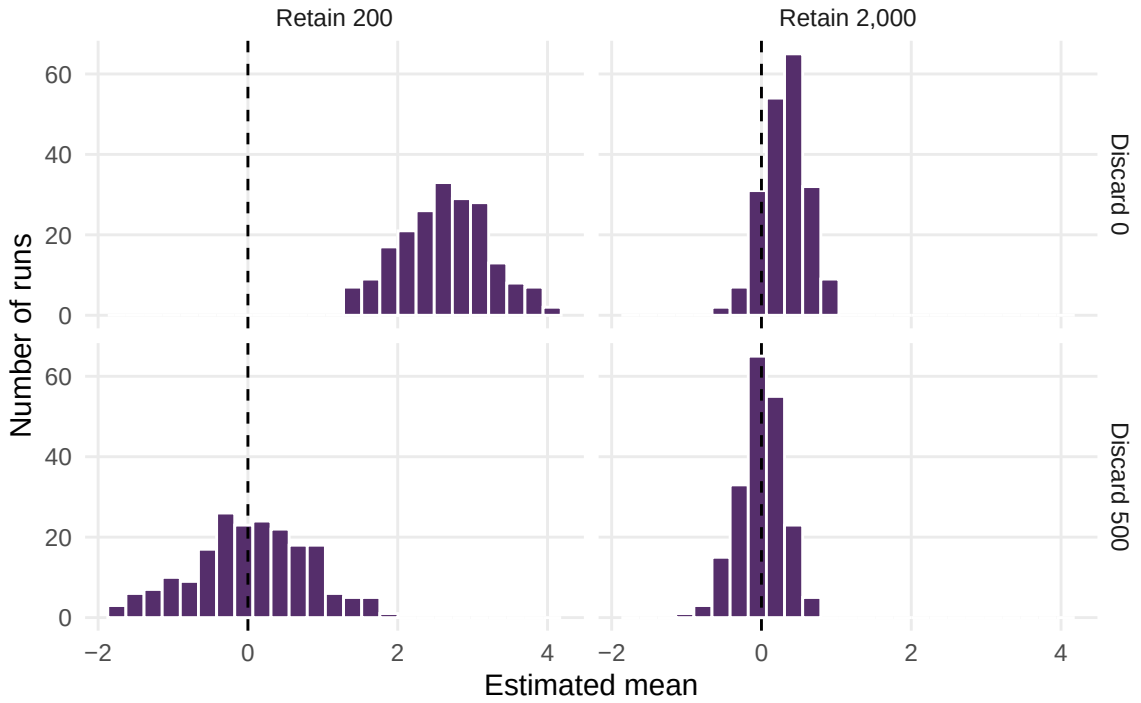


Figure 4.6: Repeated estimates of the mean of a standard Normal target, using an AR(1) chain with phi = 0.99 started at 6. Each panel contains 200 independent runs. The dashed line is the exact mean, zero. Discarding more initial states reduces the displacement of the estimates; retaining more states reduces their spread.

In the upper panels, early values near 6 pull the estimates above zero. Discarding 500 states largely removes that shift. Compare the columns as well: Retaining 2,000 states gives a narrower spread of estimates than retaining 200. Burn-in addresses the influence of initialization; more retained values improve precision once the chain is exploring the target satisfactorily.

A small MCSE answers the precision question. It does not establish that initialization bias is negligible or that the chain has visited every important region. We therefore need to compare chains as well as assess the precision within a run.

Quiz yourself!

A fit reports a posterior mean of 12.4 with posterior standard deviation 3.0 and $N_{\text{eff}} = 36$. What is the MCSE of the posterior mean? How would you communicate its numerical uncertainty? How many effective draws would give a mean MCSE of 0.05?

4.4 Comparing several chains

A single chain cannot reveal that another starting point would lead to a different answer. Several chains let us make that comparison. If they explore the same distribution sufficiently well, their means and ranges should be similar. Persistent differences are a reason to investigate.

Consider two sets of four records in Figure 4.7. In the first set, every record consists of standard Normal draws. In the second, two records fluctuate around −2 and two around 2, each with standard deviation one.

These illustrate agreement and disagreement about location. Each matrix below has 1,000 rows and four columns, one column per record. We create the second set by shifting two columns down by two and two up by two.

```
set.seed(84735)
agree <- matrix(rnorm(4000), ncol = 4)
separated <- matrix(rnorm(4000), ncol = 4)
separated[, 1:2] <- separated[, 1:2] - 2
separated[, 3:4] <- separated[, 3:4] + 2
```

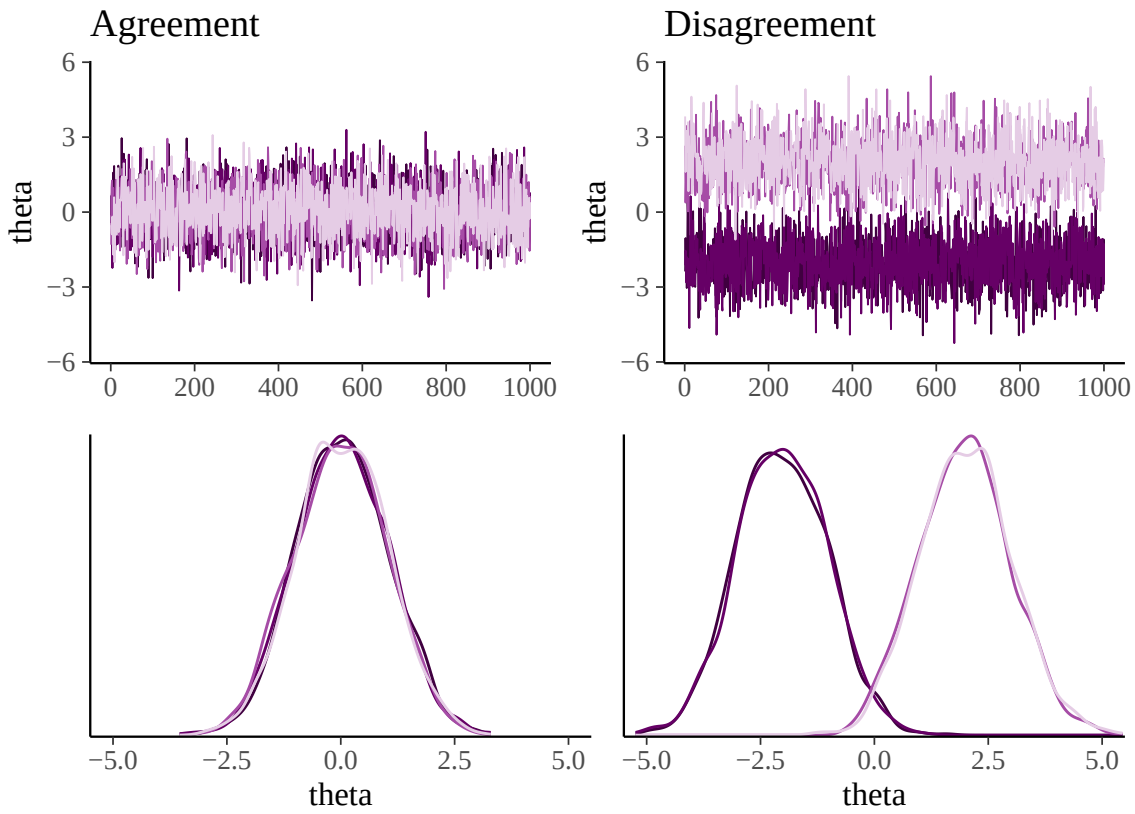


Figure 4.7: Records that agree (left column) and records that disagree about location (right column). Traces show the values over time; density curves summarize the distribution within each record. Each individual record fluctuates rapidly in both sets.

The individual records in both sets have little autocorrelation. That tells us about dependence within each record; it does not tell us whether the records agree with one another. In the second set, pooling the values would hide the fact that no individual record has covered the combined distribution.

4.4.1 The idea behind R-hat

The diagnostic $\widehat{R}$, pronounced **R-hat**, compares variation between chains with variation within chains (Gelman & Rubin, 1992). It is close to one when these give compatible pictures of the distribution, and larger when the chains disagree.

Suppose there are m chains with $L = N/m$ retained values each, where N is the total number of retained values. Write $\bar{\theta}_j$ and s_j^2 for the sample mean and sample variance of chain j. The average of the chain

variances is

$$W = \frac{1}{m}\sum_{j=1}^{m} s_j^2.$$

This describes how much values vary within the chains. To compare the chains, calculate the sample variance of their means and multiply it by the chain length:

$$B = \frac{L}{m-1}\sum_{j=1}^{m}(\bar{\theta}_j - \bar{\theta}_\cdot)^2, \qquad \bar{\theta}_\cdot = \frac{1}{m}\sum_{j=1}^{m}\bar{\theta}_j.$$

Here $\bar{\theta}_\cdot$ is the average of the chain means. The basic calculation combines W and B as

$$\widehat{\mathrm{var}}^{+} = \frac{L-1}{L}W + \frac{B}{L}, \qquad \widehat{R} = \sqrt{\frac{\widehat{\mathrm{var}}^{+}}{W}}.$$

The numerator includes differences between the chain means, whereas the denominator describes variation within the chains. Large differences between the means can therefore raise the ratio above one.

Return to the disagreeing records in Figure 4.7, stored in `separated`. We calculate a mean and a sample variance for each column. `colMeans()` gives the four means; `apply(separated, 2, var)` applies `var()` to each column:

```
chain_means <- colMeans(separated)
chain_variances <- apply(separated, 2, var)
rbind(mean = chain_means, variance = chain_variances)
#>                [,1]       [,2]      [,3]    [,4]
#> mean     -2.0304360 -2.0191015 1.9859088 2.02687
#> variance  0.9664289  0.9663091 0.9986177 1.01906
```

The sample variances are close to one, but they differ because they come from different random draws. Averaging them gives

$$W \approx \frac{0.966 + 0.966 + 0.999 + 1.019}{4} = 0.988.$$

The four means are approximately -2.030, -2.019, 1.986, and 2.027. We calculate their sample variance and multiply it by $L = 1{,}000$ to obtain B. The code uses the unrounded values for all calculations:

```
L <- nrow(separated)
W <- mean(chain_variances)
B <- L * var(chain_means)
var_plus <- (L - 1) / L * W + B / L
c(W = W, B = B, rhat = sqrt(var_plus / W))
#>           W           B        rhat
#>    0.987604 5417.046370    2.546378
```

With $B \approx 5417.05$, the resulting comparison is

$$\widehat{R} \approx \sqrt{\frac{(999/1000) \times 0.988 + 5417.05/1000}{0.988}} \approx 2.546.$$

The records differ much more from one another than their individual variances suggest. The large R-hat makes that disagreement visible in a single number.

4.4.2 Comparing earlier and later values

Whole-chain averages can miss a shared trend. In the upward-trending example in Figure 4.3, all four records rise from a low level to a high level at about the same rate. Their overall means can be similar even though the traces are still changing.

Split R-hat addresses this by dividing each chain into an earlier half and a later half, then treating the halves as separate chains. Four chains become eight half-chains. In the trend example, the earlier halves have lower means than the later halves, so the comparison reveals the change over time.

We can see the difference by generating a fresh set of four such records, with $L = 1{,}000$ values each as before. Each column adds the same increasing sequence to small random fluctuations. The two calls to `rhat_basic()` compare the basic calculation before and after splitting:

```
set.seed(84735)
trend <- sapply(1:4, function(j) {
  0.05 + 0.015 * j + seq(0, 0.45, length.out = L) +
    rnorm(L, 0, 0.012)
})
c(basic = rhat_basic(trend, split = FALSE),
  split = rhat_basic(trend, split = TRUE))
#>    basic    split
#> 1.010445 2.085559
```

The basic value is 1.0104 despite the obvious upward movement. It is barely above the diagnostic target of 1.01, so it gives at most a weak warning. Splitting raises the value to about 2.086: the difference between the earlier and later halves now produces a much clearer warning.

For practical use, `posterior::rhat()` performs this split and also checks for differences in spread (Vehtari et al., 2021). It accepts a matrix with one column per chain. Applied to the two sets in Figure 4.7:

```
c(agreement = posterior::rhat(agree), disagreement = posterior::rhat(separated))
#>    agreement disagreement
#>     1.000479     1.726001
```

Use $\widehat{R} < 1.01$ as a diagnostic target for each monitored parameter. A value of 1.04 calls for investigation even if the software uses a looser warning threshold. Inspect the traces and density curves to see what differs. If the chains agree, assess whether their MCSEs are small enough for the quantities you want to report.

Rank normalization compares the ranks of the sampled values, reducing the influence of a few extreme values. **Folding** compares their distances from a common center. This helps reveal chains that have similar means but different spreads—a disagreement that comparing means alone can miss. Section C.3 explains both calculations.

4.5 A worked analysis: Dutch-language radio

Suppose you want to know what proportion of the songs played on Dutch radio station NPO 3FM are sung in Dutch. Let π denote that proportion. You choose a Beta$(2, 5)$ prior and listen to $n = 20$ randomly selected tracks, seven of which are in Dutch. The Beta-Binomial update from Chapter 2 gives

$$\pi \mid y \sim \text{Beta}(9, 18).$$

We will estimate its mean and a 95% credible interval by simulation. The exact posterior gives us a useful check on the results. For this analysis, suppose we want the MCSE of the mean to be at most 0.005 on the proportion scale, half a percentage point in the reported proportion, and the MCSE of each interval endpoint to be at most 0.01, one percentage point. The endpoints get the looser tolerance because relatively few draws fall near them, so they are harder to pin down than the mean. We will check the mean and both interval endpoints against these tolerances.

4.5.1 Generating the chains with Stan

Stan (Carpenter et al., 2017) generates posterior draws from a model specification. We tell it what is observed, which parameters are unknown, and which prior and likelihood define the model. Its sampling algorithm is explained in Chapter 10 (Hamiltonian Monte Carlo). Here we use it to obtain chains that we can inspect with the tools just developed.

The R package `rstan` fits a model written in Stan's language. The model has three blocks: `data` declares the observations, `parameters` declares the unknown proportion, and `model` specifies the prior and likelihood.

```
radio_model <- "
data {
  int<lower = 0> n;                  // number of tracks sampled
  int<lower = 0, upper = n> y;       // number of tracks in Dutch
}
parameters {
  real<lower = 0, upper = 1> pi;     // proportion of tracks in Dutch
}
model {
  y ~ binomial(n, pi);               // likelihood
  pi ~ beta(2, 5);                   // prior
}
"
```

Here `int` means an integer count and `real` a continuous quantity. The bounds restrict `pi` to proportions between zero and one. The two statements in `model` describe the Binomial likelihood and Beta prior; Stan uses them to evaluate the log posterior.

We ask for four chains of 2,000 iterations, discarding the first 1,000 of each as warm-up. The remaining 1,000 values per chain give $N = 4{,}000$ retained draws:

```
set.seed(84735)
radio_data <- list(n = 20, y = 7)
radio_fit <- stan(model_code = radio_model, data = radio_data,
                  chains = 4, iter = 2000, warmup = 1000,
                  seed = 84735, refresh = 0)
```

The first call compiles the model before sampling, which can take longer than the simulation itself. `as.array()` then arranges the retained draws by iteration, chain, and parameter. We select `pi`, leaving a matrix with one column per chain. The two blank positions in `radio_array[, , "pi"]` keep every retained iteration and every chain:

```
radio_array <- as.array(radio_fit)
pi_mat <- radio_array[, , "pi"]
dim(pi_mat)
#> [1] 1000    4
```

Keep this structure when computing diagnostics. Flattening the matrix into one long vector would lose the chain boundaries.

Quiz yourself!

You call `stan()` with `chains = 3` and `iter = 5000`, leaving warm-up at its default of half the iterations. How many draws do you retain per chain and in total? What changes if you set `warmup = 500`?

4.5.2 Do the chains explore similar values?

The `bayesplot` package provides one function for each graphical check. `mcmc_trace()` shows movement over time, `mcmc_acf()` shows dependence within each chain, and `mcmc_dens_overlay()` draws a density curve for each chain. The argument `pars = "pi"` selects the proportion we want to examine. Figure 4.8 brings these three plots together with `patchwork`, where `/` stacks plots and `|` places them side by side. Read them as complementary views of the same simulation.

```
(mcmc_trace(radio_array, pars = "pi") + legend_none()) /
  (mcmc_acf(radio_array, pars = "pi", lags = 20) |
     (mcmc_dens_overlay(radio_array, pars = "pi") + legend_none()))
```

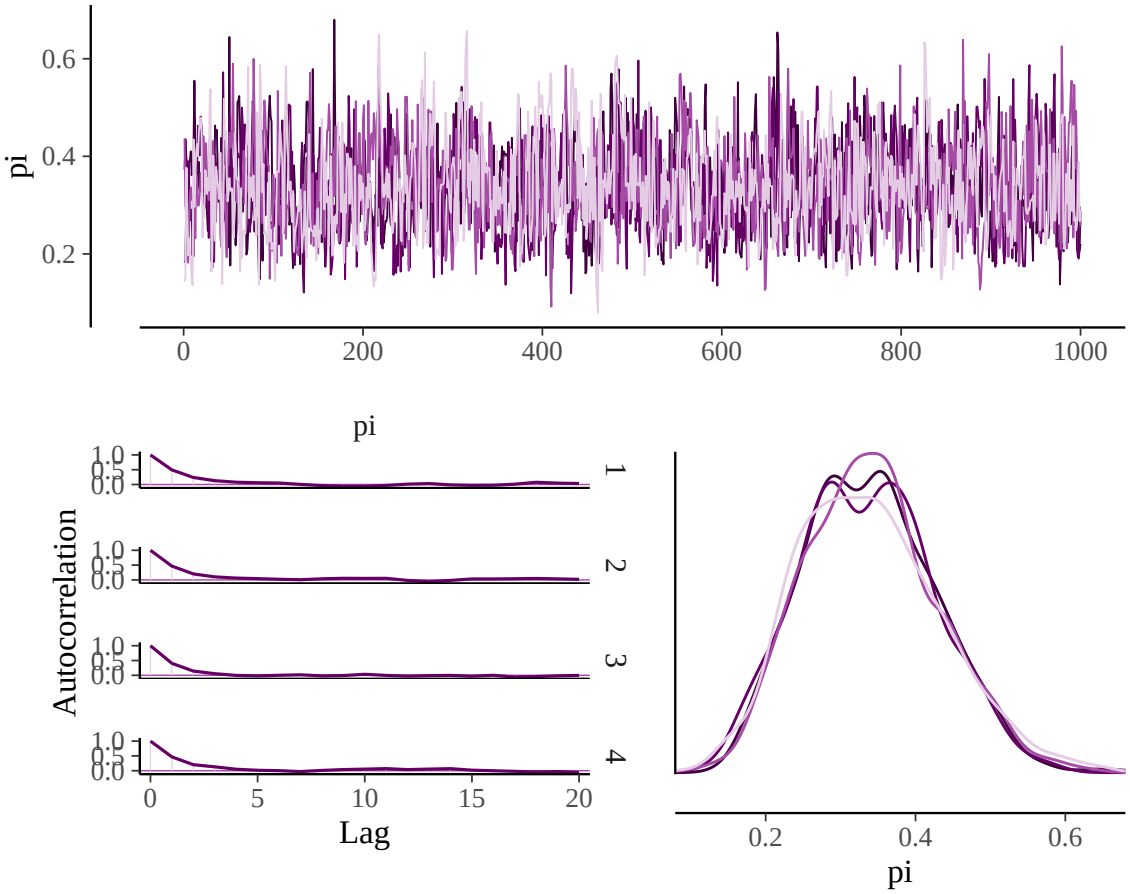


Figure 4.8: The radio posterior from four Stan chains. Top: retained values over time. Bottom left: autocorrelation within each chain. Bottom right: density estimates from the separate chains.

The traces fluctuate through similar ranges, their autocorrelations fall quickly, and the density curves overlap. R-hat provides the corresponding numerical comparison:

```
posterior::rhat(pi_mat)
#> [1] 1.003033
```

For this run it is 1.003, below 1.01. We also ask Stan to check for problems specific to its sampling algorithm:

```
rstan::check_hmc_diagnostics(radio_fit)
#>
#> Divergences:
#> 0 of 4000 iterations ended with a divergence.
#>
#> Tree depth:
#> 0 of 4000 iterations saturated the maximum tree depth of 10.
#>
#> Energy:
#> E-BFMI indicated no pathological behavior.
```

Stan reports no such problems for this fit. Chapter 10 (Hamiltonian Monte Carlo) explains these checks in detail. When a fit does produce warnings, investigate them before relying on the posterior summaries.

4.5.3 Is the mean precise enough?

Calculate the posterior mean, posterior standard deviation, and MCSE of the mean:

```
c(mean = mean(pi_mat), posterior_sd = sd(pi_mat),
  mean_mcse = mcse_mean(pi_mat))
#>         mean posterior_sd    mean_mcse
#>  0.338817635  0.089925277  0.002332468
```

The estimated mean is 0.339, with MCSE 0.0023. This meets our target of 0.005. The exact mean is $9/27 = 1/3$; the difference between the simulated and exact means is 0.0055, or 2.4 MCSEs. That is a little larger than the one or two MCSEs we called unsurprising, but well within the range that repeated runs produce; it is not a sign that anything is wrong.

Keep the two standard deviations separate. The posterior standard deviation, about 0.09, describes uncertainty about the unknown proportion of Dutch-language tracks. The MCSE describes uncertainty in our numerical estimate of the posterior mean. Running the chain longer can reduce the second without changing the first.

4.5.4 Are the interval endpoints precise enough?

A precise mean does not ensure a precise credible interval. The endpoints are the 2.5% and 97.5% quantiles: they leave 2.5% of the posterior probability below the interval and 2.5% above it. We need to assess their numerical error separately. `mcse_quantile()` calculates the MCSE at specified probabilities:

```
interval_precision <- tibble(
  probability = c(0.025, 0.975),
  estimate = as.numeric(quantile(pi_mat, c(0.025, 0.975))),
  exact = qbeta(probability, 9, 18),
  mcse = as.numeric(mcse_quantile(pi_mat, probs = probability))
)
interval_precision
#> # A tibble: 2 x 4
#>   probability estimate exact    mcse
#>         <dbl>    <dbl> <dbl>   <dbl>
#> 1       0.025    0.180 0.172 0.00436
#> 2       0.975    0.522 0.518 0.00458
```

Each row gives an endpoint estimate, its exact value, and its MCSE. The MCSEs are 0.0044 and 0.0046. Both are below 0.01, so this run also meets our requirement for the interval. The estimated 95% credible interval is [0.180, 0.522]. The MCSEs describe how much its endpoints would change between simulations; they do not describe uncertainty about π itself.

If an endpoint's MCSE had been 0.02, a run about four times as long would be a reasonable starting plan for reaching 0.01. We would then check both the chain behavior and the achieved precision again. Tail quantiles often need many effective draws because relatively few sampled values lie near the endpoints. There is no single ESS threshold that ensures a chosen decimal precision for every posterior.

4.5.5 Reading bulk and tail ESS in a summary

Software summaries often include `ess_bulk` and `ess_tail`. These give broad checks on sampling efficiency in the main body and tails of the distribution (Vehtari et al., 2021). For the radio fit:

```
c(ess_bulk = ess_bulk(pi_mat), ess_tail = ess_tail(pi_mat))
#>  ess_bulk  ess_tail
#> 1482.134 1532.167
```

Bulk ESS uses the relative ordering of the draws to assess sampling through the main body of the distribution. **Tail ESS** is the smaller of the ESS estimates for the 5% and 95% quantiles. A large bulk ESS with a small tail ESS is a reason to look more closely at the tails, especially when reporting a credible interval.

Practical diagnostic targets

- Aim for **R-hat below 1.01** for every monitored parameter.
- With four chains, aim for **both bulk ESS and tail ESS of at least 400 in total** (Vehtari et al., 2021).
- Check the **MCSE of each summary you intend to report**. Meeting the ESS target does not necessarily give enough precision for your purpose.

Use these targets alongside the trace plots and sampler checks. They do not guarantee that every important posterior region has been explored.

4.5.6 What would we report?

Under the Beta-Binomial model, the posterior mean proportion of Dutch-language tracks is approximately 0.34, with a 95% credible interval from 0.18 to 0.52.

R-hat is 1.003, below 1.01. Bulk and tail ESS are 1482 and 1532, both above 400. The MCSE of the mean is 0.0023, below our tolerance of 0.005, and the MCSEs of the interval endpoints are 0.0044 and 0.0046, both below our tolerance of 0.01. Together with the satisfactory plots and absence of Stan sampler warnings, these results give us a reason to stop sampling for the present analysis.

Figure 4.9 compares the simulated distribution with the known posterior. In a more complex model this exact check may be unavailable, but the diagnostic questions remain the same.

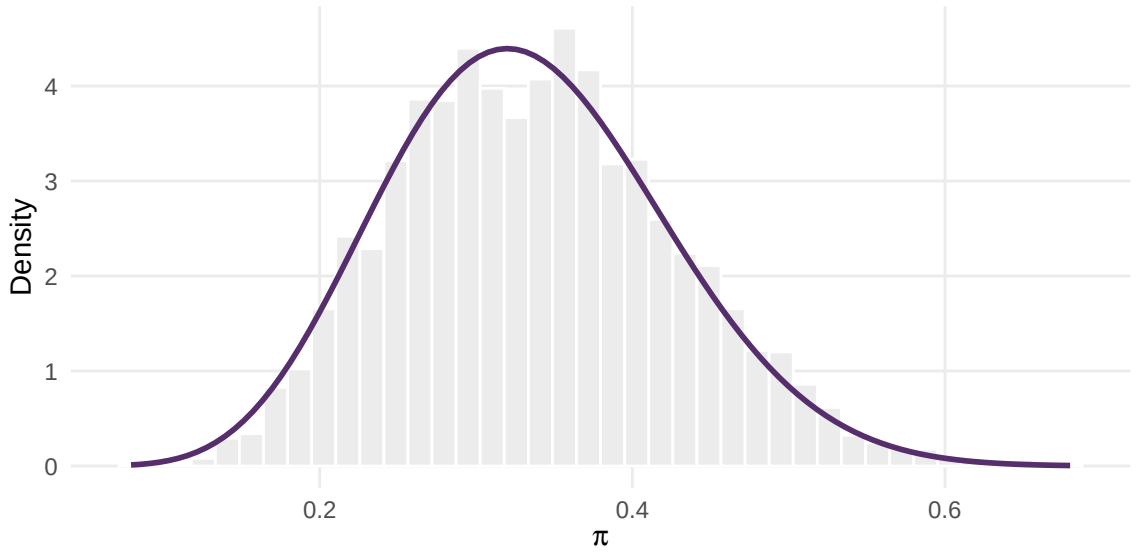


Figure 4.9: Histogram of the 4,000 Stan draws for the proportion of Dutch-language tracks, with the exact Beta(9, 18) posterior overlaid.

Choosing the next step

- **Early values still move away from the starts:** consider more warm-up and inspect the new runs.
- **Chains explore similar values, but the required summaries have large MCSEs:** retain more draws or improve mixing.
- **Chains remain in different regions, or the sampler reports problems:** investigate the cause before relying on a longer run.
- **Plots and diagnostics are satisfactory, and the intended summaries meet their precision targets:** report the estimates with their numerical precision.

4.6 What a reassuring run can miss

Agreement over a short interval does not establish that a longer run will behave the same way. Figure 4.10 illustrates a record that looks stable for its first 1,000 values but later moves to a different level.

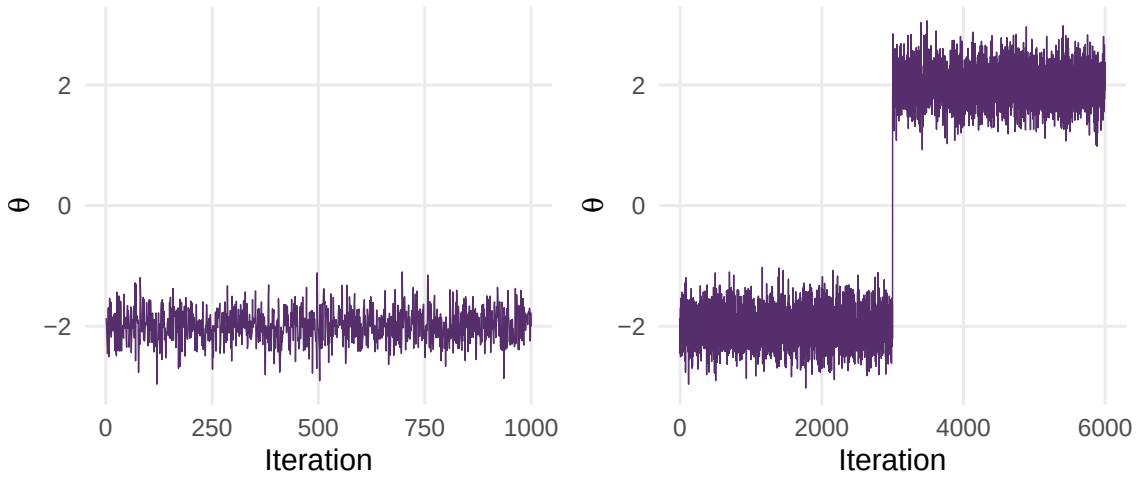


Figure 4.10: A synthetic example of a late change. The first 1,000 values (left) look stable. The longer run (right) reveals a move to a different level after 3,000 values.

The late change might represent arrival in a region with more posterior probability, or movement between two modes. One jump does not tell us which explanation applies. We need to examine the longer run and, when several modes are present, see whether the chains move between them repeatedly.

Several chains can also agree because they have all missed the same region. Suppose the target is an equal mixture of $\mathrm{N}(-3, 1)$ and $\mathrm{N}(3, 1)$. Its mean is zero and half its probability lies on each side of zero. What would we see if every chain stayed near the left mode?

We can illustrate that output with four AR(1) records centered at -3. These sample the left Normal component; they are not samplers for the full mixture. Figure 4.11 compares their output with the whole target.

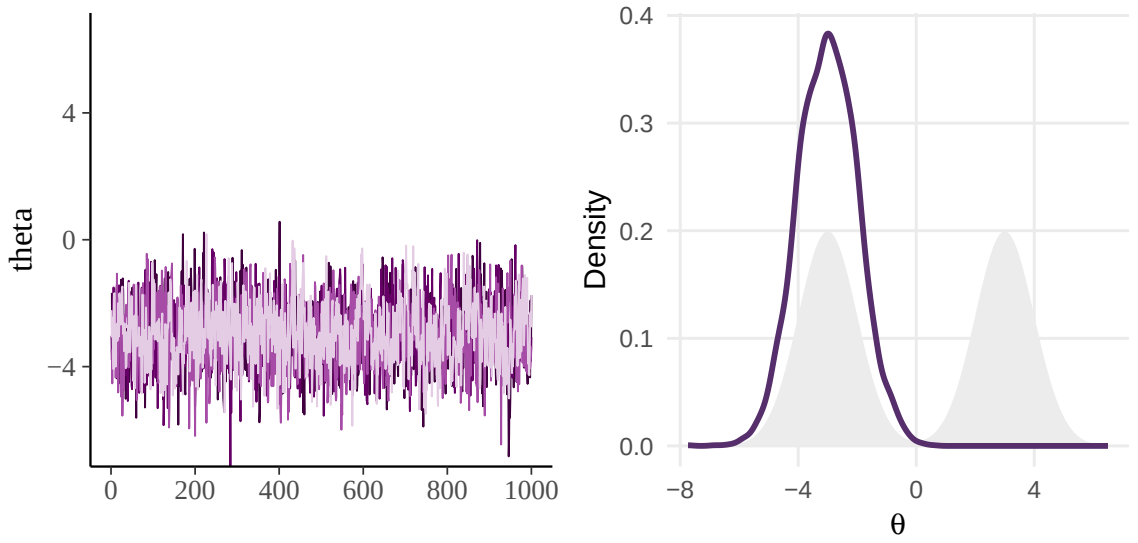


Figure 4.11: Four records representing exploration of only the left mode of a bimodal posterior. Left: the trace looks healthy and the chains agree. Right: the density estimated from the chains (purple) against the true posterior (gray). The records agree around the left mode, but their estimated mean is near -3 rather than the full posterior mean of zero.

The traces look stable and R-hat is 1.002. Nevertheless, the estimated mean is near -3 rather than zero. Agreement has not revealed the missing mode. An MCSE calculated from these records describes variation

within the explored region and misses the much larger error caused by incomplete exploration.

Dispersed starting values help because different chains may find different regions. Knowledge of the model also matters: a mixture model, for example, may suggest several modes worth investigating. Chapter 5 shows this difficulty with an actual sampler. No diagnostic based only on visited values can tell us everything about a region that was never visited.

A separate question is whether the model describes the observed data well. Even an accurate simulation can represent the posterior of an unsuitable model. Posterior predictive checks address that question by comparing observations simulated from the fitted model with the data.

4.7 Chapter summary

A finite simulation needs both adequate exploration and adequate precision. Trace plots reveal movement over time, and comparisons between chains can expose regions explored from one starting point but not another. Burn-in reduces the influence of initialization; Stan's warm-up also adjusts its sampling settings.

Autocorrelation explains why a long chain can give an imprecise average. Effective sample size expresses that precision relative to independent draws, and MCSE expresses the numerical uncertainty in the units of the summary. More retained draws generally reduce Monte Carlo variation, but they do not reduce posterior uncertainty about the parameter.

R-hat compares chains and their halves. Use it alongside plots and sampler checks, then assess the MCSE of each mean, probability, or interval endpoint you intend to report. Satisfactory diagnostics support a decision to use the results; they cannot prove that no important region has been missed.

4.8 Quiz answers

Quiz 1. Disagreement between chains. A single chain confined to one mode can produce a stable-looking trace. Additional chains started elsewhere may reveal a different region. Drift and flat stretches can be visible within one chain; comparison between chains requires more than one.

Quiz 2. If the ACF decays geometrically, $\rho_k = \rho_1^k$, then $N_{\text{eff}} = N(1-\rho_1)/(1+\rho_1)$. Chain A is worth $10{,}000 \times 0.1/1.9 \approx 526$ independent draws and chain B $10{,}000 \times 0.7/1.3 \approx 5{,}385$, about ten times as many. The Monte Carlo standard error of chain B's posterior mean is therefore about $\sqrt{10} \approx 3.2$ times smaller.

Quiz 3. The mean MCSE is $3.0/\sqrt{36} = 0.5$. Communicate the estimate with its numerical uncertainty, for example an estimated posterior mean of 12.4 with MCSE 0.5, rather than claiming that its first decimal is stable. A target MCSE of 0.05 requires $N_{\text{eff}} = (3.0/0.05)^2 = 3{,}600$, one hundred times as many effective draws. This target does not guarantee stable rounding.

Quiz 4. Each chain runs 5,000 iterations, of which the first 2,500 are warm-up by default, leaving 2,500 draws per chain and $3 \times 2{,}500 = 7{,}500$ in total. With `warmup = 500`, only 500 iterations per chain are discarded, so you keep 4,500 per chain and 13,500 in total. Whether 500 is adequate requires checking adaptation, sampler warnings, and the retained chains; a trace alone cannot settle it.

4.9 Exercises

4.9.1 Conceptual exercises

Exercise 4.1 (Warm-up or burn-in?) Explain the difference between burn-in and Stan's warm-up. Why are adaptation draws excluded even when initialization is near the posterior mode? For a fixed transition rule, distinguish starting at the mode from drawing the initial state from the exact posterior.

Which establishes stationarity, and why?

Exercise 4.2 (Thinning by the numbers) A chain of $N = 20{,}000$ draws has autocorrelations $\rho_k = 0.9^k$.

a. Compute its effective sample size.
b. The chain is thinned by keeping every fifth draw. What are the autocorrelations of the thinned chain, and what is its effective sample size?
c. A colleague argues that the thinned chain is better because its draws are "almost independent." Explain what they have confused.

Exercise 4.3 (Four messages) A classmate is running their first Stan models and sends you four messages during the afternoon. Reply to each in two or three sentences, stating what the diagnostic says and what they should do next.

a. "My $\widehat{R}$ is 1.00 and the trace plot is a flat band. Am I done?"
b. "My $\widehat{R}$ is 1.04, but the software has not warned me. Does that mean the chains are adequate?"
c. "My `ess_bulk` is 3,000 but my `ess_tail` is 90. I only care about the posterior mean, so can I ignore the tail?"
d. "My $\widehat{R}$ is 1.6. I'll just run the chains ten times longer."

Exercise 4.4 (How precise is precise enough?) A posterior mean is estimated as 0.412 with posterior standard deviation 0.08 from a chain with $N_{\text{eff}} = 250$.

a. Compute the mean MCSE and give an approximate interval of plus or minus two MCSEs for the numerical uncertainty in the estimated posterior mean. Does this support claiming that its rounding to two decimal places is stable?
b. How many effective draws are needed to make the MCSE smaller than 0.001?
c. Explain in one paragraph why increasing N_{eff} shrinks the MCSE but does not shrink the posterior standard deviation, and why a Bayesian should care about the difference.

4.9.2 Practical exercises

Exercise 4.5 (Diagnose these chains) Figure 4.12 shows trace plots of four sets of chains, labelled A to D, and Figure 4.13 shows the corresponding density overlays. The patterns are subtler than the ones in Figure 4.3. For each set, decide whether the chains look healthy, and if not, name the pathology and the diagnostic that reveals it most clearly. Then say, for each unhealthy set, what further information would be needed to judge whether a longer run would help. The records illustrate patterns; do not assume that their generating rules are valid posterior samplers.

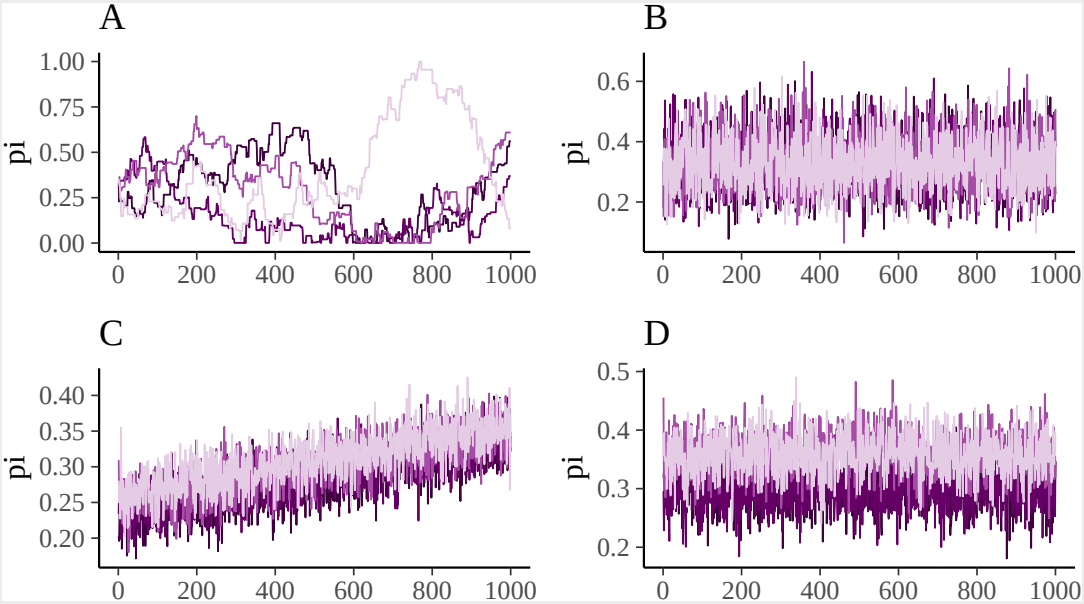


Figure 4.12: Trace plots for Exercise 4.5.

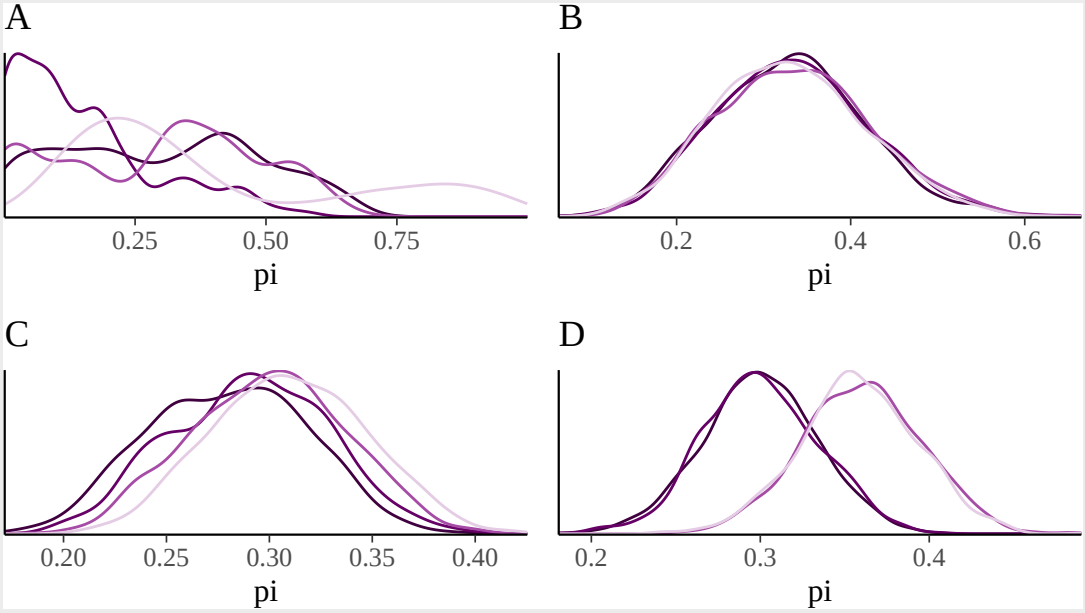


Figure 4.13: Density overlays for Exercise 4.5.

Exercise 4.6 (A mixing laboratory) Use `ar1_chain()` from this chapter.

a. For $\phi \in \{0, 0.3, 0.6, 0.9, 0.99\}$, simulate a chain with $N = 20{,}000$ stored states, including its initial state, and compute `ess_basic()`. Draw that initial state from $\mathrm{N}(0, 1)$ to isolate dependence from initialization. Plot the effective sample size against ϕ and add the curve $N(1-\phi)/(1+\phi)$.

b. For $\phi = 0.99$, plot the trace of the first 1,000 iterations. Describe what you see and relate it to the effective sample size you computed.

c. For each ϕ, compute the MCSE of the posterior mean using `mcse_mean()`. Since the true mean is zero, compare the MCSE with the actual error $|\bar{\theta} - 0|$ of each chain. Repeat the simulation with a few seeds. Is the MCSE an honest description of the error?

Exercise 4.7 ($\widehat{R}$ by hand; optional) Optional extension: Use `rhat_by_hand()` and `split_halves()` from Section C.2.

a. Simulate four records of $N = 1{,}000$ stored states from `ar1_chain()` with $\phi = 0.95$, each started at `init=0`. Compute unsplit and split basic R-hat by hand. Compare them with `rhat_basic(split=FALSE)` and `rhat_basic(split=TRUE)`, and also report `posterior::rhat()`. Are these fixed-start records stationary at time zero?
b. Now start the four chains at $-10, -5, 5$, and 10. Repeat the same comparisons. Then discard the first 200 iterations of each chain and compute them again. Explain what changed.
c. Construct four chains that have the same mean but where two chains have three times the spread of the other two, as in Section C.3. Compare `rhat_basic()` and `posterior::rhat()` and identify any difference in the values relative to 1.01, and explain which of the two ingredients of the rank-normalized $\widehat{R}$, rank normalization or folding, is responsible.

4.9.3 Applied exercises

Exercise 4.8 (An undercooked bicycle counter) A bicycle counter on Amsterdam's Weesperstraat records 23, 19, 25, 21, and 17 bicycles in five one-minute windows. Assume a constant rate λ, with $Y_i \mid \lambda \sim \text{Pois}(\lambda)$ and a Gamma$(20, 1)$ prior (shape 20, rate 1). The exact posterior is Gamma$(125, 6)$.

Write this model in Stan as `bikes_model`, using an array of observed counts in the data block, `array[n] int<lower = 0> y;`, and a positive real parameter `lambda`. Fit it with `chains = 4`, `iter = 60`, and `seed = 84735`.

a. Run the full diagnostic workflow. Which checks fail? Distinguish warnings about the agreement or precision of the chains from warnings about Stan's sampling algorithm; Chapter 10 explains the latter in detail.
b. Fit the model at `iter = 60, 100, 200, 400, 800, 1600, 3200`, using the same seed. Record R-hat, bulk ESS, tail ESS, mean MCSE, and sampler warnings. Report the first tested setting with R-hat below 1.01, both ESS values at least 400, no sampler warnings, and satisfactory plots. If needed, extend the schedule. Explain why this is not a universal minimum or a guarantee that every larger setting passes.
c. Estimate $P(\lambda > 22 \mid y)$ with an indicator average: give each draw value one if $\lambda > 22$ and zero otherwise, then average. Apply `mcse_mean()` to this indicator matrix to assess numerical precision, keeping one column per chain. Use a pilot run to plan a run with indicator MCSE at most 0.0025, then check the achieved MCSE and diagnose the final fit. Compare the result with the exact Gamma-posterior probability. Explain why this numerical tolerance does not guarantee stable two-decimal rounding.

Exercise 4.9 (Gen X at the MoMA) The `moma_sample` data in the `bayesrules` package records, for a random sample of 100 artists represented in the Museum of Modern Art in New York, whether each was born in the Generation X years or later (the variable `genx`). Let π be the proportion of such artists in the museum's collection.

a. Choose a Beta prior for π that reflects your own expectation, and justify it in one sentence.
b. Write the Beta-Binomial model in Stan and fit it with four chains. Run the full diagnostic workflow and report $\widehat{R}$, `ess_bulk`, `ess_tail`, and the MCSE of the posterior mean.
c. Compare the Stan posterior with the exact Beta posterior, both graphically and by posterior mean

and 95% credible interval. Are the differences within what the MCSE leads you to expect?

d. Fit the model again from four dispersed values strictly inside $(0, 1)$; the endpoints are not valid finite initial values on Stan's unconstrained scale. Does anything in the diagnostics change? Should it?

Part III

Sampling Algorithms

Chapter 5

The Metropolis Algorithm

Suppose you ask 20 first-year psychology students how many hours they slept last night. You would like to learn the average number of hours, μ, that first-year students sleep. You choose a prior and a sampling model, then combine them to obtain an unnormalized posterior. How can you generate draws from it?

Chapter 2 introduced conjugate updates and grid approximation. Chapter 4 used Stan to generate posterior draws and examined their accuracy. Here we construct a sampler ourselves: the **Metropolis algorithm**. It proposes a new parameter value, compares the posterior density there with the density at the current value, and decides whether to move. Repeating this procedure produces a Markov chain designed to have the posterior as its stationary distribution. Stan usually uses Hamiltonian Monte Carlo, developed in Chapter 10 (Hamiltonian Monte Carlo); Metropolis gives us a simpler algorithm with which to study how proposals and acceptance decisions work.

Run the following setup before working through the examples.

```
library(tidyverse)
library(posterior)
library(patchwork)
library(mvtnorm)
```

Goals

- Explain proposal symmetry and calculate Metropolis acceptance probabilities, including decisions on the log scale.
- Implement a Metropolis sampler in R and check its output against a known posterior.
- Verify detailed balance and distinguish stationarity from the conditions needed to explore the target.
- Explain the step-size trade-off and tune a sampler using the acceptance rate, trace plots, and the effective sample size.
- Sample from a posterior that has no closed form by writing its log density from a likelihood and a prior.
- Diagnose implementation errors using the states recorded after acceptance and rejection.
- Extend the algorithm to several parameters and see why plain Metropolis struggles as the number of parameters grows.

5.1 The idea: propose, compare, accept or reject

We want a Markov chain whose stationary distribution has a chosen density $f(\theta)$, such as a posterior density. The Metropolis algorithm (Metropolis et al., 1953) uses evaluations of this density and a **proposal distribution** for candidate next states. We need to be able to evaluate the target up to a constant; conjugacy is not required.

In this chapter, a proposal is drawn from a Normal distribution centered on the current state:

$$\theta^\star \sim \mathrm{N}\big(\theta^{(t)}, c^2\big), \qquad c > 0.$$

The standard deviation c is the **step size**. It controls how far proposals tend to fall from the current state. For example, with $c = 0.5$, proposals one unit above and one unit below the current value are equally far from the center and have the same proposal density.

Metropolis requires more than this left–right symmetry around one state. Write $q(b \mid a)$ for the density of proposing b when the chain is at a. The proposal must be **symmetric between states**:

$$q(b \mid a) = q(a \mid b).$$

A Normal proposal with the same step size at every state has this property. Proposing 7 from a current value of 6 involves a displacement of $+1$; proposing 6 from 7 involves a displacement of -1. These have equal density under the centered Normal noise. The target densities at 6 and 7 may nevertheless differ. Proposal symmetry concerns how candidates are generated; the acceptance step accounts for their different target densities.

Compare the proposed and current values using

$$r = \frac{f(\theta^\star)}{f(\theta^{(t)})}.$$

If $r \geq 1$, the target density is at least as high at the proposal, and we accept it. If $r < 1$, we accept with probability r. A proposal with half the current target density is therefore accepted half the time. If it is rejected, the next state equals the current state, which is recorded again.

The Metropolis algorithm

Given a target density $f(\theta)$ that we can evaluate up to a constant, a current state $\theta^{(t)}$, and a symmetric proposal distribution with step size c:

1. **Propose.** Draw $\theta^\star \sim \mathrm{N}(\theta^{(t)}, c^2)$.
2. **Compare.** Compute $r = f(\theta^\star)/f(\theta^{(t)})$.
3. **Accept or reject.** If $r \geq 1$, set $\theta^{(t+1)} = \theta^\star$. Otherwise draw $u \sim \mathrm{Unif}(0, 1)$ and set $\theta^{(t+1)} = \theta^\star$ if $u < r$, and $\theta^{(t+1)} = \theta^{(t)}$ if not.

Repeat for $t = 0, 1, 2, \ldots$, starting at a value where the log target is finite. After T iterations, the recorded states are $\theta^{(1)}, \ldots, \theta^{(T)}$; the starting value $\theta^{(0)}$ is specified separately.

The normalizing constant cancels in r. For a posterior,

$$\frac{f(\theta^\star \mid y)}{f(\theta^{(t)} \mid y)} = \frac{f(y \mid \theta^\star) f(\theta^\star)/f(y)}{f(y \mid \theta^{(t)}) f(\theta^{(t)})/f(y)} = \frac{f(y \mid \theta^\star) f(\theta^\star)}{f(y \mid \theta^{(t)}) f(\theta^{(t)})}.$$

We can therefore evaluate likelihood times prior at the two values without calculating $f(y)$.

Figure 5.1 compares two proposals for a standard Normal target. The proposal in the left panel has a higher target density and is always accepted. The proposal in the right panel has a lower target density and may be accepted or rejected. Allowing moves to lower-density regions lets the chain explore the spread of the distribution, rather than only seeking its peak.

5.1.1 Following three decisions

Consider the unnormalized standard Normal target $f(\theta) \propto e^{-\theta^2/2}$, starting at $\theta^{(0)} = 0$. Suppose the proposal and Uniform draws are those in Table 5.1. These are three illustrative decisions, with each proposal made from the state left by the preceding decision.

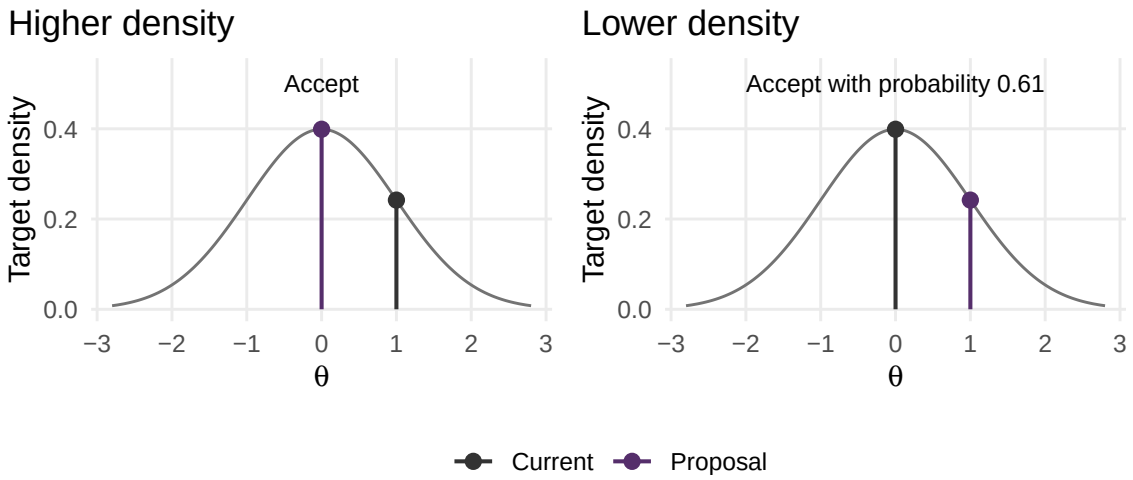


Figure 5.1: Two Metropolis comparisons for a standard Normal target. Vertical lines mark the target density at the current and proposed values. Left: moving from 1 to 0 gives a ratio about 1.65 and is always accepted. Right: moving from 0 to 1 gives a ratio about 0.61 and is accepted with probability 0.61. The curve gives density heights to compare; the chain's states are values on the horizontal axis.

Table 5.1: Three Metropolis decisions for a standard Normal target.

Iteration	Current state	Proposal	Ratio r	Uniform draw u	Decision	Recorded state
1	0	1	$e^{-1/2} \approx 0.607$	0.80	Reject	0
2	0	1	$e^{-1/2} \approx 0.607$	0.40	Accept	1
3	1	0.5	$e^{3/8} \approx 1.455$	0.90	Accept	0.5

In the first row, $0.80 > 0.607$, so the proposal is rejected. The second proposal is consequently made from 0 again. This time $0.40 < 0.607$, so the chain moves to 1. In the third row the target density increases, and the proposal is accepted regardless of the Uniform draw. The three recorded states are $(0, 1, 0.5)$. Keeping the first recorded 0 is part of the algorithm: one iteration has passed even though the chain did not move.

Quiz yourself!

The chain is at $\theta^{(t)} = 1$ where $f(1) = 0.24$, and the proposal is $\theta^\star = 2.5$ where $f(2.5) = 0.02$. Given this proposal, what is its acceptance probability? And if instead the proposal were $\theta^\star = 0.3$ with $f(0.3) = 0.38$?

5.2 A first Metropolis sampler in R

Let's return to the sleep survey. We use the following illustrative answers, in hours:

```
sleep_hours <- c(6.5, 7.2, 5.8, 6.9, 7.5, 6.1, 5.5, 7.0, 6.4, 8.1,
                 6.7, 5.9, 7.3, 6.2, 6.8, 7.7, 5.6, 6.3, 7.1, 6.0)
```

We model the answers as independent given μ, with $Y_i \mid \mu \sim \mathrm{N}(\mu, 1^2)$. The sampling standard deviation of one hour is treated as known. We give μ a $\mathrm{N}(7, 0.5^2)$ prior, expressing an expectation of about seven hours, with most prior probability between six and eight hours.

This is the Normal–Normal model in Chapter 2. To see its posterior, combine the quadratic terms from the likelihood and prior:

$$\begin{aligned}\log f(\mu \mid y) &= -\frac{1}{2}\left[\sum_{i=1}^{n}(y_i-\mu)^2 + \frac{(\mu-7)^2}{0.5^2}\right] + C \\ &= -\frac{1}{2}\left[(n+4)\mu^2 - 2\left(\sum_i y_i + 28\right)\mu\right] + C.\end{aligned}$$

The 4 is the prior precision $1/0.5^2$, and $28 = 7 \times 4$. Completing the square gives a Normal posterior with variance $\tau_n^2 = 1/(n+4)$ and mean $\mu_n = (\sum_i y_i + 28)/(n+4)$. Section D.4 explains this algebra step by step.

```
n <- length(sleep_hours)
tau2_n <- 1 / (1 / 0.5^2 + n / 1^2)
mu_n <- tau2_n * (7 / 0.5^2 + sum(sleep_hours) / 1^2)
c(sample_mean = mean(sleep_hours), posterior_mean = mu_n,
  posterior_sd = sqrt(tau2_n))
#>    sample_mean posterior_mean   posterior_sd
#>      6.6300000      6.6916667      0.2041241
```

With $n = 20$ and $\sum_i y_i = 132.6$, the posterior is

$$\mu \mid y \sim \mathrm{N}\left(\frac{160.6}{24}, \frac{1}{24}\right) \approx \mathrm{N}(6.692, 0.204^2).$$

Its mean lies between the observed mean of 6.63 hours and the prior mean of seven hours. We can generate independent draws from this posterior directly with `rnorm()`. Here it gives us a known answer against which to check Metropolis before changing to a nonconjugate prior.

5.2.1 The log target

The sampler evaluates the log of likelihood times prior. As in Chapter 2, we work on the **log scale** because multiplying many small likelihood contributions can produce a number too small for R to represent. Their logarithms can still be added and compared:

$$\ell(\mu) = \sum_{i=1}^{n} \log f(y_i \mid \mu) + \log f(\mu).$$

This differs from the normalized log posterior by a constant. R's density functions compute its terms with `log = TRUE`:

```
log_posterior <- function(mu, y = sleep_hours) {
  sum(dnorm(y, mean = mu, sd = 1, log = TRUE)) +   # log likelihood
    dnorm(mu, mean = 7, sd = 0.5, log = TRUE)       # log prior
}
log_posterior(6.5)
#> [1] -24.22456
log_posterior(9)
#> [1] -87.72456
```

The larger value means a higher posterior density. To compare 6.5 and 9, subtract their log targets: the difference is about 63.5. The density ratio is the exponential of that difference, although we do not need to compute that exponential to make an acceptance decision.

Metropolis on the log scale

Starting from a current state with finite log target:

1. Draw $\theta^\star \sim \mathrm{N}(\theta^{(t)}, c^2)$.
2. Calculate $\log r = \ell(\theta^\star) - \ell(\theta^{(t)})$.
3. Draw $u \sim \mathrm{Unif}(0, 1)$. Accept when $\log u < \log r$; otherwise keep the current state. Record the resulting state in either case.

Because $\log u < 0$, every proposal with $\log r \geq 0$ is accepted. When $r < 1$, taking logs preserves the comparison $u < r$.

For the first two rows of Table 5.1, $\log r = -0.5$. The first Uniform draw gives $\log(0.80) \approx -0.223$, which is greater than -0.5, so we reject. The second gives $\log(0.40) \approx -0.916$, which is smaller, so we accept. The log calculation makes exactly the same decisions.

5.2.2 The sampler

Here is the algorithm of the previous section as an R function. It takes the log target, a starting value, the number of iterations, and the step size. It returns a list with the draws in a tibble and the fraction of proposals that were accepted. The `n_iter` argument gives the number T of recorded states, and any extra arguments in `...` are passed on to `log_target`, for example the data. The function draws both Normal proposals and Uniform values for the acceptance decisions.

```
metropolis <- function(log_target, init, n_iter, step_size, ...) {
  draws <- numeric(n_iter)
  current <- init
  current_lp <- log_target(current, ...)
  if (!is.finite(current_lp)) stop("log_target() is not finite at init")
  n_accept <- 0

  for (t in seq_len(n_iter)) {
    proposal <- rnorm(1, mean = current, sd = step_size)   # 1. propose
    proposal_lp <- log_target(proposal, ...)
    if (is.nan(proposal_lp)) stop("log_target() returned NaN; return -Inf outside the
      support")
    log_r <- proposal_lp - current_lp                      # 2. compare (on the log
  scale)
    if (log(runif(1)) < log_r) {                           # 3. accept ...
      current <- proposal
      current_lp <- proposal_lp
      n_accept <- n_accept + 1
    }
    draws[t] <- current                                    # ... or stay and record
  }

  list(
    draws = tibble(iteration = seq_len(n_iter), theta = draws),
    accept_rate = n_accept / n_iter
  )
}
```

A few remarks on how the code maps onto the algorithm. Step 2 computes $\log r$ rather than r, which is just a subtraction. Step 3 compares $\log u$ with $\log r$, which is the same as comparing u with r, and it handles both cases of the algorithm at once: if $r \geq 1$ then $\log r \geq 0 > \log u$ and the proposal is always accepted. We also store the log target of the current state, `current_lp`, so that each iteration evaluates the target only

once, for the proposal. That matters when the target is expensive. The two `stop()` lines are guards: the chain must start where the log target is finite, and a log target that returns `NaN` for a proposal cannot be compared with $\log u$; Section 5.6 explains when that happens.

Let's run it for 10,000 iterations, starting at $\mu = 5$ with a step size of 0.5:

```
set.seed(84735)
fit <- metropolis(log_posterior, init = 5, n_iter = 10000, step_size = 0.5)
fit$accept_rate
#> [1] 0.4375
head(fit$draws)
#> # A tibble: 6 x 2
#>   iteration theta
#>       <int> <dbl>
#> 1         1  5.33
#> 2         2  5.98
#> 3         3  6.10
#> 4         4  6.10
#> 5         5  6.12
#> 6         6  6.12
```

About 44% of the proposals were accepted. The `theta` column stores the sampled values of μ; `iteration` counts the updates. A repeated value usually records a rejection. Figure 5.2 shows both the sequence and its distribution, with the exact posterior overlaid on the histogram.

```
p_trace <- ggplot(fit$draws, aes(iteration, theta)) +
  geom_line(color = "#552E6B", linewidth = 0.3) +
  labs(x = "Iteration", y = expression(mu))

p_hist <- ggplot(filter(fit$draws, iteration > 500), aes(theta)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                 fill = "#ECECEC", color = "white") +
  stat_function(fun = dnorm, args = list(mean = mu_n, sd = sqrt(tau2_n)),
                color = "#552E6B", linewidth = 1) +
  labs(x = expression(mu), y = "Density")

p_trace | p_hist
```

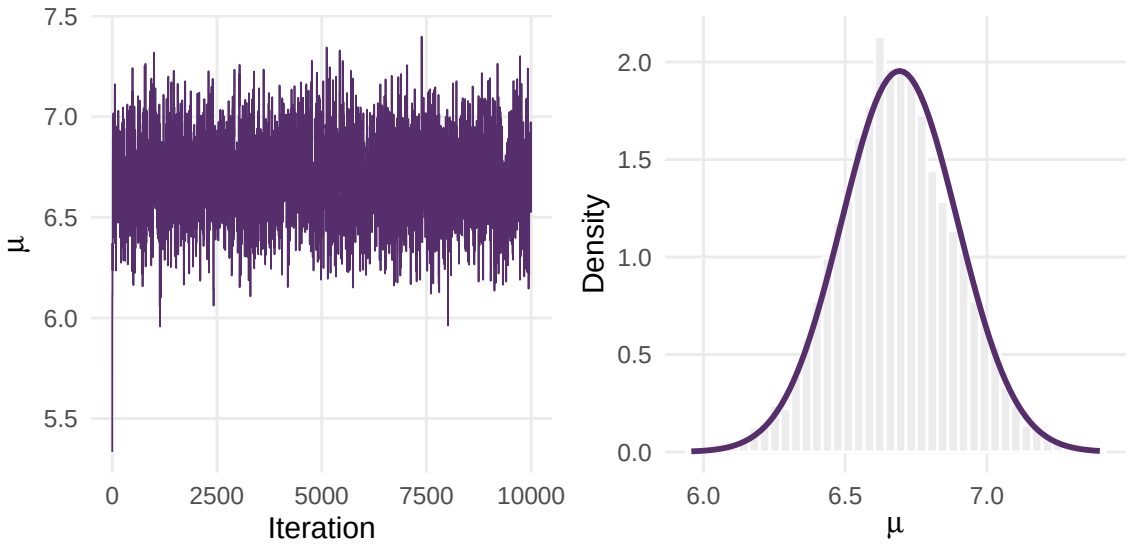


Figure 5.2: Left: trace of the Metropolis chain for the mean hours of sleep. Right: histogram of the draws after discarding the first 500, with the exact Normal posterior overlaid.

The retained draws approximate the exact posterior. We can compare the mean, standard deviation, and central 95% interval as well:

```
draws <- fit$draws |> filter(iteration > 500)
draws |>
  summarize(mean = mean(theta), sd = sd(theta),
            lower = quantile(theta, 0.025), upper = quantile(theta, 0.975))
#> # A tibble: 1 x 4
#>    mean    sd lower upper
#>   <dbl> <dbl> <dbl> <dbl>
#> 1  6.68 0.204  6.29  7.09
c(mean = mu_n, sd = sqrt(tau2_n),
  lower = qnorm(0.025, mu_n, sqrt(tau2_n)), upper = qnorm(0.975, mu_n, sqrt(tau2_n)))
#>      mean        sd     lower     upper
#> 6.6916667 0.2041241 6.2915907 7.0917426
```

As in Chapter 4, the starting value can affect early estimates. Here the chain starts at five hours, well below the main posterior region, and moves toward that region during its first updates. We discard 500 iterations before computing the displayed histogram and summaries. This choice is generous for this example; reaching the posterior region once does not by itself establish that enough warm-up has been discarded. The remaining draws are still dependent, so their numerical precision must be assessed using ESS and MCSE.

5.3 Why it works

The comparison with the exact sleep posterior checks one implementation. Detailed balance explains why the acceptance rule preserves the target distribution.

Suppose the target density at a is twice as high as at b. With a symmetric proposal, a move from a to b is accepted with probability $1/2$, whereas the reverse proposal is always accepted. These different acceptance probabilities compensate for the difference in target density.

For the general argument, consider distinct values a and b with positive target density. Write $K(a, b)$ for the density of moving from a to b in one update. The candidate must first be proposed and then accepted,

giving

$$K(a, b) = q(b \mid a) \min\left\{1, \frac{f(b)}{f(a)}\right\}.$$

Multiplying by the starting density simplifies the expression:

$$\begin{aligned} f(a)K(a, b) &= q(b \mid a) \min\{f(a), f(b)\} \\ &= q(a \mid b) \min\{f(b), f(a)\} \\ &= f(b)K(b, a). \end{aligned}$$

The second line uses proposal symmetry. The minimum of the two density values is also unchanged when we reverse their order. Rejections contribute a probability of staying at the current state; that contribution balances itself. Thus Metropolis satisfies detailed balance, and the target distribution is stationary. Section C.4 derives the acceptance probabilities by solving the balance equation.

Stationarity and exploration

Detailed balance establishes stationarity. To learn about the whole target from a run, we also need **irreducibility** and **aperiodicity**, as discussed in Chapter 3.

For the continuous targets in this chapter, a Normal proposal with fixed $c > 0$ can propose into every region with positive target probability, so the chain is irreducible. Rejections also allow a state to persist, so these chains are not locked into a fixed cycle and are aperiodic. With sufficiently long runs, their averages estimate target expectations, provided those expectations exist. A suitable step size is still needed for useful precision in a practical run.

The balance argument uses the target through a ratio, so multiplying it by a positive constant changes nothing. It also holds for every fixed $c > 0$: changing the step size changes the proposal density in both directions equally. The argument establishes the stationary distribution; the next section examines how efficiently a run explores it.

5.4 Step size and efficiency

The step size c does not affect whether the chain has the right stationary distribution, but it has everything to do with how quickly the chain explores it. Let's run the sleep sampler with three step sizes: one tiny, one large, and the one we used above. We start each near the posterior mean and compare the same number of updates. Figure 5.3 pairs each trace with a histogram so that we can inspect both movement and the resulting posterior approximation.

```
set.seed(84735)
step_sizes <- c(0.02, 0.5, 10)
fits <- map(step_sizes, \(c) metropolis(log_posterior, init = 6.7,
                                        n_iter = 10000, step_size = c))
```

```
tibble(step_size = step_sizes, accept_rate = map_dbl(fits, "accept_rate"))
#> # A tibble: 3 x 2
#>   step_size accept_rate
#>       <dbl>       <dbl>
#> 1      0.02      0.969
#> 2      0.5       0.435
#> 3     10         0.0245
```

The three chains have the same stationary distribution, but they are not equally useful. With $c = 0.02$ almost every proposal is accepted, yet each move is so small that the chain takes hundreds of iterations to

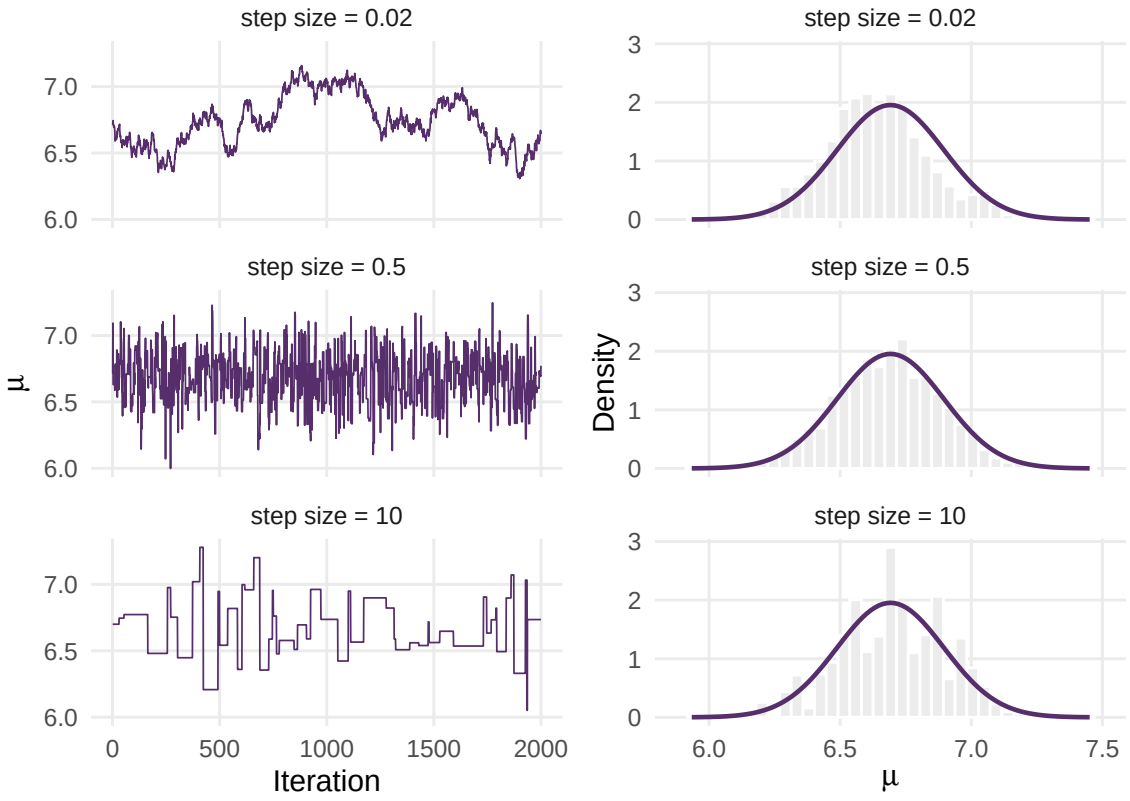


Figure 5.3: Three step sizes for the sleep posterior. Left: the first 2,000 iterations. Right: histograms of all 10,000 states with the exact posterior density overlaid. Small steps move slowly; large steps give repeated states. Both can yield an uneven approximation in a run of this length.

cross the posterior; consecutive draws are nearly identical. With $c = 10$ most proposals land far outside the posterior and are rejected, so the chain repeats the same value for dozens of iterations at a time. Both chains produce draws that are highly **autocorrelated**, and highly autocorrelated draws carry little information each. With $c = 0.5$, proposals are about 2.45 posterior standard deviations wide, allowing larger moves while retaining a moderate acceptance rate. The behavior resembles the AR(1) comparison in Chapter 4: persistent sequences give less stable averages than sequences that explore the target more rapidly. Here the dependence results from the proposal and acceptance decisions, rather than from a chosen AR coefficient.

Chapter 4 gave us the tools to make this precise. Figure 5.4 shows how strongly the sampled values remain associated as their separation in iterations increases:

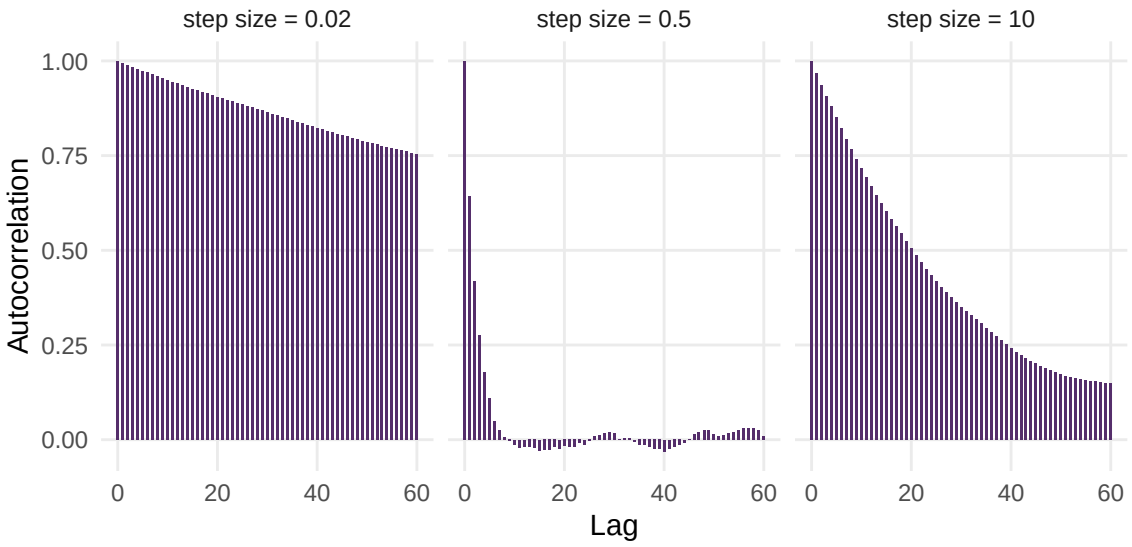


Figure 5.4: Autocorrelation of the draws for the three step sizes. For the tiny and the large step sizes the autocorrelation decays slowly; for the middle one it drops quickly toward zero.

For the posterior mean, ESS expresses the precision of the dependent average in terms of an equivalent number of independent draws:

```
tibble(step_size = step_sizes,
       ess = map_dbl(fits, \(f) ess_basic(f$draws$theta)))
#> # A tibble: 3 x 2
#>   step_size    ess
#>       <dbl>  <dbl>
#> 1      0.02   24.6
#> 2      0.5  2270.
#> 3     10     144.
```

The middle chain is worth roughly 16 times as many independent draws as the better of the other two, for the same number of iterations and target evaluations.

5.4.1 Tuning by hand

How do we choose c? There is no formula that works for every target, but we can compare several trial values using traces, ESS, and the acceptance rate. Use runs of equal length and assess the summaries you intend to report; an estimated ESS from a single trial is itself variable. Since a useful step size depends on the target's scale, it is convenient to think of c relative to the posterior standard deviation, here about 0.2.

```r
set.seed(84735)
tuning <- tibble(step_size = exp(seq(log(0.02), log(10), length.out = 15))) |>
  mutate(fit = map(step_size, \(c) metropolis(log_posterior, init = 6.7, n_iter =
    10000, step_size = c)),
         ess = map_dbl(fit, \(f) ess_basic(f$draws$theta)),
         accept_rate = map_dbl(fit, "accept_rate")) |>
  select(-fit)
```

Figure 5.5 compares the results across the trial step sizes. The horizontal axis is logarithmic so that both small and large step sizes remain visible.

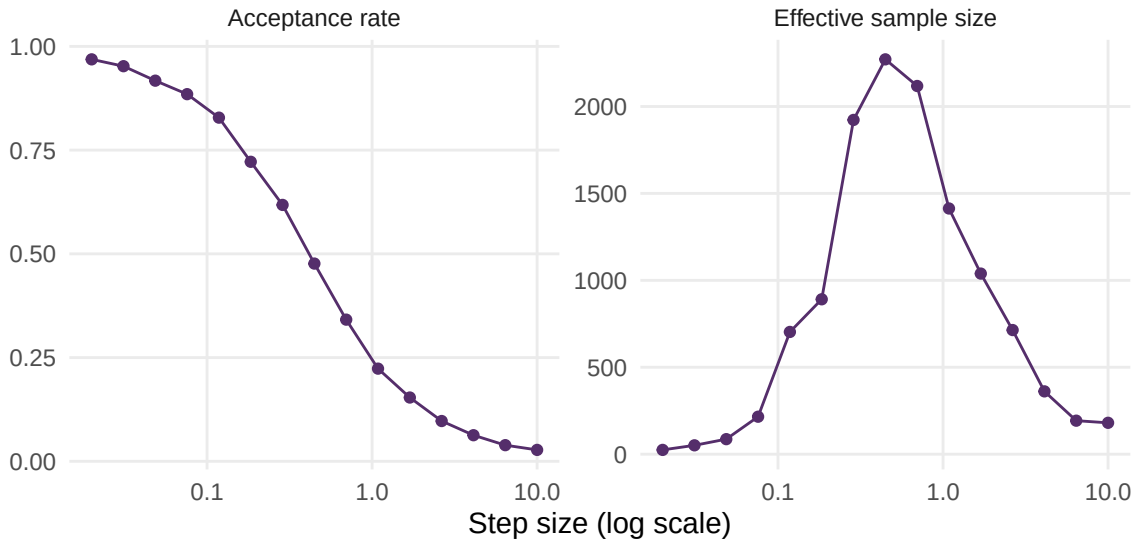


Figure 5.5: Effective sample size (out of 10,000 draws) and acceptance rate as a function of the step size, on a log scale. The effective sample size peaks where the acceptance rate is somewhere between 0.3 and 0.6.

The acceptance rate is useful feedback about the step size, but it must be interpreted with the traces and ESS. For a one-dimensional Normal target with Normal random-walk proposals, studies of optimal scaling suggest an acceptance rate near 0.44. For many similarly scaled parameters updated together, the corresponding high-dimensional result is about 0.234 (Roberts et al., 1997; Roberts & Rosenthal, 2001). These results describe the balance between accepted move sizes and rejection, the same balance illustrated here. They are tuning guides, not convergence criteria or universal optima for every posterior.

Choosing a step size

Start with trial runs at a few step sizes. If most proposals are accepted but the trace moves slowly, try a larger step. If long repeated stretches dominate, try a smaller one. Compare the resulting ESS for equal run lengths, together with the traces. Acceptance around 0.4–0.5 is a useful starting guide for one approximately Normal parameter; about 0.25 is a guide for many similarly scaled parameters updated together.

After tuning, keep the step size fixed for the run used to estimate posterior summaries. Chapter 6 (Metropolis–Hastings and Adaptive Proposals) develops an adaptive procedure that adjusts it during warm-up.

Quiz yourself!

A colleague reports that their Metropolis chain accepted 99% of its proposals and concludes that the sampler works beautifully. What do you tell them?

5.5 Targets beyond the Normal

So far we have used Metropolis on a posterior we could have written down. The point of the algorithm is that it works exactly the same when we cannot.

5.5.1 A robust prior for the sleep example

The $\mathrm{N}(7, 0.5^2)$ prior for μ is confident: it puts almost no mass below six hours. Suppose we want to express the same central belief but allow for the possibility that we are badly wrong. A common device is a **Cauchy prior**, $\mu \sim \mathrm{Cauchy}(7, 0.5)$, which has the same location, seven hours, but much heavier tails. Its scale parameter 0.5 controls the spread; it is not a standard deviation, because the Cauchy distribution has no finite variance. This prior is not conjugate to the Normal likelihood, so the Normal posterior formula no longer applies. We can still evaluate likelihood times prior, changing the prior term to `dcauchy()`:

```
set.seed(84735)
log_posterior_cauchy <- function(mu, y = sleep_hours) {
  sum(dnorm(y, mean = mu, sd = 1, log = TRUE)) +
    dcauchy(mu, location = 7, scale = 0.5, log = TRUE)
}
fit_cauchy <- metropolis(log_posterior_cauchy, init = 6.7, n_iter = 10000, step_size =
  0.5)
fit_cauchy$accept_rate
#> [1] 0.4444
```

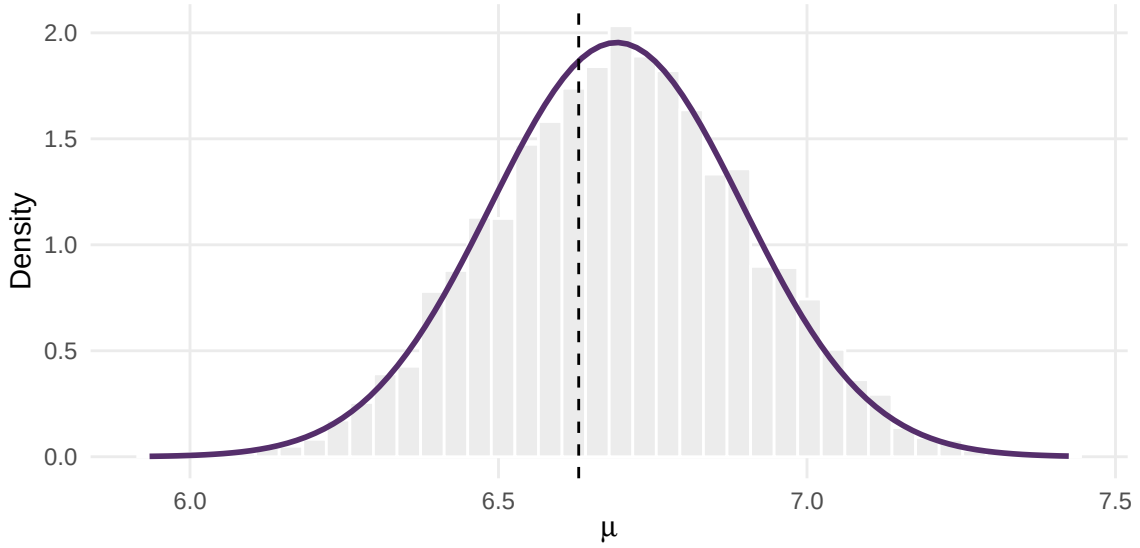


Figure 5.6: Posterior for the mean hours of sleep under the Normal prior (exact, purple line) and under the Cauchy prior (Metropolis draws, histogram). With 20 observations the data dominate and the two posteriors nearly coincide, and the dashed line marks the sample mean. The two priors give similar posteriors for these data.

The proposal and acceptance code is unchanged. The histogram in Figure 5.6 compares the new posterior draws with the exact posterior under the Normal prior. The two priors give similar results for these data, although their tail behavior differs substantially. A numerical grid or integration can provide another check in this one-parameter example. For a model with several parameters, the same acceptance rule applies to a proposed vector, as developed in Section 5.7.

5.5.2 A target with two modes

Let's look at a target that is chosen to make Metropolis struggle: an equal mixture of two Normals, centered at -5 and 5, each with standard deviation 1 (Hanada & Matsuura, 2022, Section 4.4). Posteriors with two separated modes occur in practice, for instance in mixture models and in models where the sign of a parameter is not identified.

```
set.seed(84735)
log_bimodal <- function(theta) {
  a <- log(0.5) + dnorm(theta, -5, 1, log = TRUE)
  b <- log(0.5) + dnorm(theta, 5, 1, log = TRUE)
  m <- pmax(a, b)
  m + log(exp(a - m) + exp(b - m))
}
fit_small <- metropolis(log_bimodal, init = -5, n_iter = 10000, step_size = 0.5)
fit_large <- metropolis(log_bimodal, init = -5, n_iter = 10000, step_size = 5)
```

The function adds the two weighted component densities on a stable log scale, using the rescaling idea from Chapter 2. The target is highest near -5 and 5, with a wide low-density region between them. Before inspecting Figure 5.7, consider why a chain with short proposals may spend a long time near just one mode.

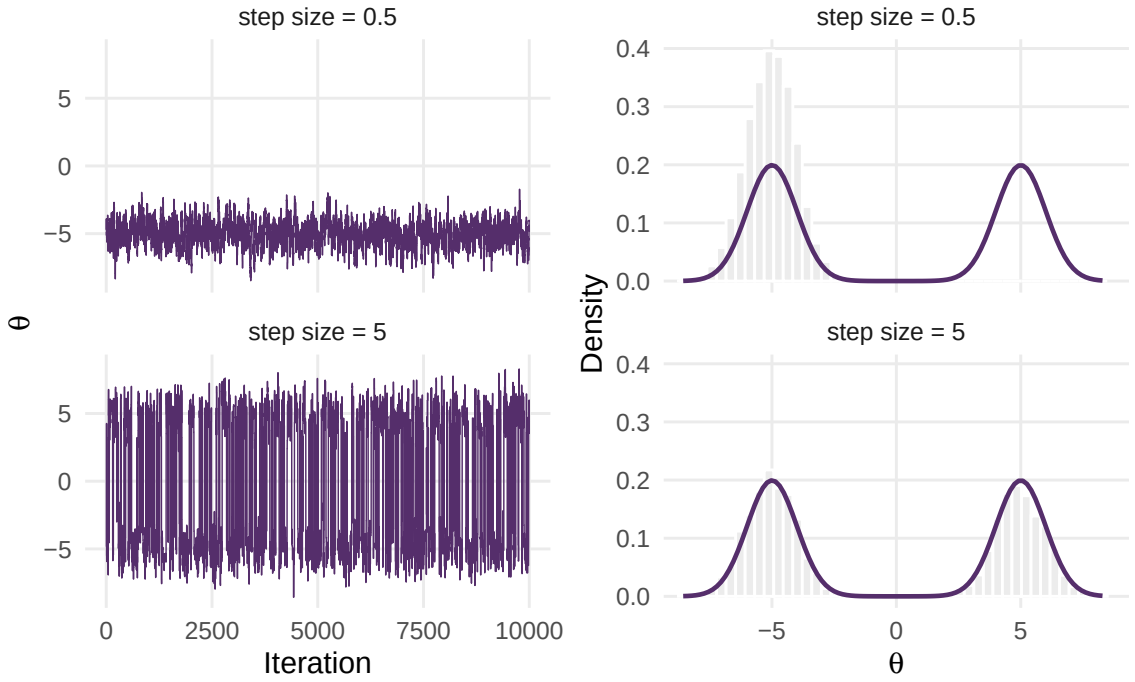


Figure 5.7: Metropolis on an equal mixture of two Normal distributions centered at -5 and 5. Each row uses the same chain for its trace (left) and histogram (right); the purple curve is the full target density. With step size 0.5 the chain never leaves the mode where it started; with step size 5 it moves between the modes. Compare the time spent on either side of zero with the equal weights of the two target components.

The small-step chain accepted 85% of its proposals, yet it crossed zero 0 times in 10,000 iterations; the large-step chain accepted 29% and crossed zero 393 times. An apparently satisfactory acceptance rate can therefore accompany poor exploration: accepted moves within one mode do not tell us how often the chain crosses to the other. To travel from -5 to 5 with step size 0.5, the chain would have to accept a long sequence of moves through the region around zero, where the target density is less than 10^{-5} of its value at a mode; that sequence is so improbable that it does not occur in a run of this length. The target assigns equal probability to its negative and positive halves. The fractions of stored values above zero are 0 for the

small step and 0.47 for the large step, compared with the target value 0.5.

This is the missing-region problem in Chapter 4. Several chains started near different modes can reveal disagreement that one trace misses. Even agreement and a low R-hat cannot guarantee that an unvisited region does not exist. In this example the target is known, so the traces and histograms can be compared directly with both modes.

5.6 Common mistakes

Each update has a proposal, a comparison, and a recorded state. Checking those three parts separately helps locate an implementation error (Hanada & Matsuura, 2022, Section 4.7).

Calculate log densities directly. Using `log(dnorm(...))` first calculates a density that may already have rounded to zero. Use `dnorm(..., log = TRUE)` and the corresponding log options for the likelihood and prior. Then subtract log targets instead of dividing tiny numbers.

Record the state after every decision. In Table 5.1, rejecting the first proposal records another zero. Saving only accepted proposals removes time spent waiting at a state and changes the distribution represented by the saved values. For the sleep target, those accepted moves alone tend to give a histogram that is too wide.

Propose from the current state. After the first rejection in that table, the next proposal is still centered at zero. Centering it at the rejected value would implement a different transition rule. Restarting at the initial value after a rejection would also change the algorithm.

Keep target evaluations consistent. `log_target(theta)` must return the same log density when called again with the same inputs. Store the current log target and update it on acceptance, as the code does. This saves a second evaluation per iteration. Random noise inside the evaluator needs methods beyond this Metropolis implementation.

Reject values outside the support. For a positive parameter, a Normal proposal may be negative. The log target should return `-Inf` there, giving acceptance probability zero. For example, `if (lambda <= 0) return(-Inf)` belongs before evaluating a Poisson likelihood at rate `lambda`: a negative rate would otherwise produce `NaN`, which cannot be compared with $\log u$. The guard in `metropolis()` turns such a value into the error `log_target() returned NaN; return -Inf outside the support`, which points at the missing check instead of failing inside the comparison. Start at a value with a finite log target. Parameter transformations in Chapter 6 (Metropolis–Hastings and Adaptive Proposals) offer another way to handle boundaries.

Check exploration across chains. A good acceptance rate within one mode can hide failure to visit another. Use dispersed starts, traces, R-hat, ESS, and MCSE as in Chapter 4, and interpret them in light of the model.

Quiz yourself!

A student modifies `metropolis()` so that it "saves time" by only storing a draw when a proposal is accepted. They run it on the sleep example and get a histogram that is wider than the exact posterior. Explain what happened.

5.7 Metropolis in more than one dimension

Almost every real model has several parameters. The Metropolis algorithm generalizes with no change to the logic: propose a new *vector* $\theta^\star$ by adding Normal noise to every component of the current vector, compute the ratio of the target at the two vectors, and accept or reject the whole vector at once. Our function needs three edits: `init` becomes a vector, `rnorm()` draws one perturbation per component, and the draws are stored in a matrix. The `...` argument again forwards extra arguments to `log_target`.

```
metropolis <- function(log_target, init, n_iter, step_size, ...) {
  d <- length(init)
  draws <- matrix(NA_real_, nrow = n_iter, ncol = d)
  current <- init
  current_lp <- log_target(current, ...)
  if (!is.finite(current_lp)) stop("log_target() is not finite at init")
  n_accept <- 0

  for (t in seq_len(n_iter)) {
    proposal <- rnorm(d, mean = current, sd = step_size)
    proposal_lp <- log_target(proposal, ...)
    if (is.nan(proposal_lp)) stop("log_target() returned NaN; return -Inf outside the
      support")
    if (log(runif(1)) < proposal_lp - current_lp) {
      current <- proposal
      current_lp <- proposal_lp
      n_accept <- n_accept + 1
    }
    draws[t, ] <- current
  }

  colnames(draws) <- if (d == 1) "theta" else paste0("theta_", seq_len(d))
  list(
    draws = as_tibble(draws) |> mutate(iteration = seq_len(n_iter), .before = 1),
    accept_rate = n_accept / n_iter
  )
}
```

This version is a drop-in replacement for the one-dimensional sampler, since a vector of length one is still a vector, and it is the version that the rest of the book uses. Let's try it on a bivariate Normal target with correlation 0.9, which illustrates the elongated shape that posterior dependence can produce.

```
Sigma <- matrix(c(1, 0.9, 0.9, 1), 2, 2)
log_bvn <- function(theta) dmvnorm(theta, mean = c(0, 0), sigma = Sigma, log = TRUE)

c(along = exp(log_bvn(c(1, 1)) - log_bvn(c(0, 0))),
  across = exp(log_bvn(c(1, -1)) - log_bvn(c(0, 0))))
#>        along       across
#> 5.9077775e-01 4.539993e-05
```

From $(0, 0)$, the proposed vectors $(1, 1)$ and $(1, -1)$ change both coordinates by one unit. Positive correlation makes $(1, 1)$ much more compatible with the target: its acceptance probability is about 0.59, whereas $(1, -1)$ is accepted with probability only about 0.000045. Both proposals are equally plausible under our independent Normal perturbations. The difference comes from the target's shape.

```
set.seed(84735)
fit_bvn <- metropolis(log_bvn, init = c(0, 0), n_iter = 10000, step_size = 0.5)
fit_bvn$accept_rate
#> [1] 0.5465
```

Figure 5.8 shows the resulting draws against contours of equal target density. The elongated contours show combinations of the parameters that receive similar density.

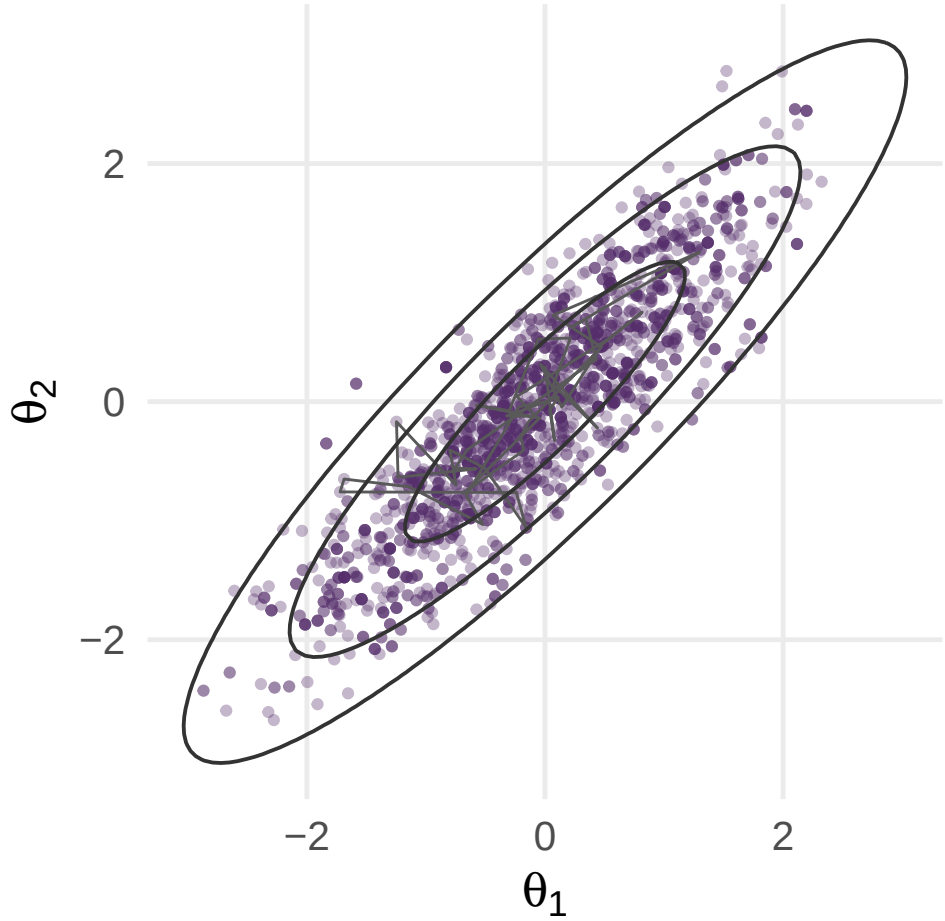


Figure 5.8: A bivariate Normal target with correlation 0.9. Dots show the first 2,000 draws and the path connects the first 80 updates; the contours, drawn on top, enclose 50%, 90%, and 99% of the target probability. The draws spread farther along the diagonal than across it.

It works, but look at the acceptance rate and at the shape of the target. The proposal adds independent noise to both coordinates, so it proposes into the thin directions of the ridge as often as along it, and proposals that move too far across the narrow direction have low acceptance probabilities. The step size that gives a good acceptance rate is dictated by the *narrowest* direction of the target, and it is then far too small for the widest direction. As the number of parameters grows, this problem compounds.

The following comparison keeps the step size at 1 while increasing the number of coordinates. Each target has independent standard Normal coordinates, so this isolates the cost of changing many coordinates at once. We start with an independent target draw to avoid confusing this comparison with initial movement toward the target region.

```
set.seed(84735)
log_std_normal <- function(theta) sum(dnorm(theta, log = TRUE))

dims <- c(1, 2, 5, 10, 20, 50)
by_dim <- tibble(d = dims) |>
  mutate(fit = map(d, \(k) metropolis(log_std_normal, init = rnorm(k), n_iter = 5000,
    step_size = 1)),
         accept_rate = map_dbl(fit, "accept_rate"),
```

```
        ess = map_dbl(fit, \(f) ess_basic(pull(f$draws, 2))) |>   # first parameter
          column
  select(-fit)

by_dim
#> # A tibble: 6 x 3
#>       d accept_rate    ess
#>   <dbl>       <dbl>  <dbl>
#> 1     1      0.704   528.
#> 2     2      0.555   383.
#> 3     5      0.308   356.
#> 4    10      0.138   185.
#> 5    20      0.0396   40.9
#> 6    50      0.0002    1.87
```

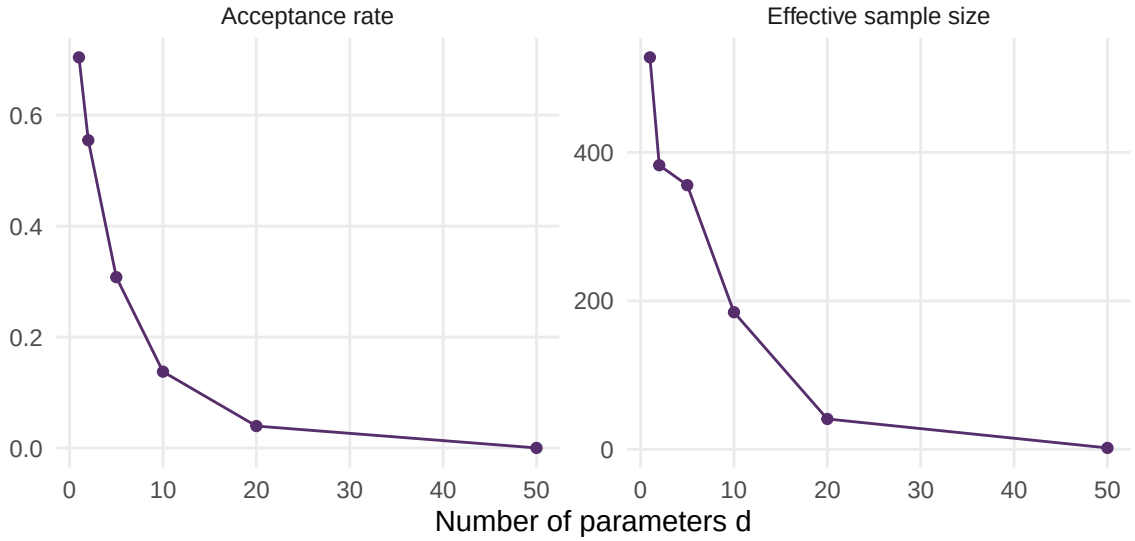


Figure 5.9: Plain Metropolis on a d-dimensional standard Normal target with a fixed step size of 1. As d grows, the acceptance rate collapses and, with it, the effective sample size of the first coordinate (out of 5,000 draws).

A proposed vector must receive one acceptance decision. As more coordinates change at once, their combined change can give a low target density even when each perturbation seems moderate. Figure 5.9 shows the resulting loss of acceptance and precision for the fixed step size.

For high-dimensional targets with independent standard Normal coordinates, optimal-scaling theory suggests $c \approx 2.38/\sqrt{d}$, with acceptance near 0.234 (Roberts et al., 1997). The scale of the target and dependence between its coordinates matter; this is not a universal step-size formula. Smaller steps can improve acceptance here, while the distance explored in each coordinate per update becomes smaller. Shaping proposals to the target (Chapter 6 (Metropolis–Hastings and Adaptive Proposals)), updating parameters separately (Chapter 7 (Gibbs Sampling and Metropolis-within-Gibbs)), and using gradients (Chapter 10 (Hamiltonian Monte Carlo)) offer ways to improve exploration.

5.8 Chapter summary

In this chapter you built the Metropolis algorithm from scratch and learned why it works, when it works well, and where it runs out of steam.

- **Metropolis** turns a target density that we can evaluate up to a constant into a Markov chain that samples from it. Each iteration proposes a candidate near the current state, computes the ratio of the target at the candidate and at the current state, and accepts the candidate if the ratio is at least one, or with probability equal to the ratio otherwise. Rejected proposals leave the chain where it is, and the current state is recorded again.
- The normalizing constant cancels in the ratio, so the algorithm runs on the **unnormalized posterior** $f(y \mid \theta)f(\theta)$.
- With a symmetric proposal, the acceptance rule satisfies **detailed balance** and preserves the target distribution. Irreducibility and aperiodicity are also needed for exploration. For the Normal random-walk proposals here, every fixed positive step size gives the same stationary distribution.
- Efficiency does. A step size that is too small crawls, one that is too large gets rejected; both produce highly autocorrelated draws. **Tune** using traces and ESS, with acceptance rates around 0.4–0.5 in one approximately Normal dimension or around 0.25 in many similarly scaled dimensions as guides.
- Always work on the **log scale**, always record repeated states, and never trust a single chain to have found every mode.
- The algorithm extends to vectors without change, but a proposal that perturbs every parameter blindly becomes inefficient as the number of parameters grows. Chapter 6 (Metropolis–Hastings and Adaptive Proposals), Chapter 7 (Gibbs Sampling and Metropolis-within-Gibbs), and Chapter 10 (Hamiltonian Monte Carlo) develop ways to improve proposals and exploration.

5.9 Quiz answers

Quiz 1. Given the proposal 2.5, its acceptance probability is $\min(1, 0.02/0.24) \approx 0.083$. Given the proposal 0.3, the ratio is $0.38/0.24 > 1$, so it is always accepted. These probabilities concern accepting an already generated proposal; they do not describe the probability of proposing that exact value from a continuous distribution.

Quiz 2. A high acceptance rate alone does not establish useful exploration. For this Normal random-walk sampler, 99% acceptance suggests checking whether the step size is so small that the chain moves slowly. Inspect the trace and ESS, and compare a trial run with a larger step. The aim is precise estimates from good exploration, rather than a particular acceptance rate by itself.

Quiz 3. An accepted-moves-only record omits the waiting time created by rejections. For example, a state held for five iterations should contribute five times to a posterior average; the modified record gives it just one entry. The saved values therefore represent a different distribution. For this Normal target and random-walk proposal, omitting waiting times gives too much relative weight to the tails and can produce the wider histogram.

5.10 Exercises

5.10.1 Conceptual exercises

Exercise 5.1 (Accept or reject?) A Metropolis chain with a symmetric proposal is at $\theta^{(t)} = 2$. For each proposed value below, the unnormalized target is given at the current state and at the proposal. State the probability that the proposal is accepted.

a. $f(2) = 0.6$, $f(\theta^\star) = 0.9$
b. $f(2) = 0.6$, $f(\theta^\star) = 0.15$
c. $f(2) = 0.6$, $f(\theta^\star) = 0.6$
d. $f(2) = 0.6$, $f(\theta^\star) = 0$

Exercise 5.2 (The missing constant) Explain, in words and in one line of algebra, why the Metropolis algorithm never needs the normalizing constant $f(y)$ of the posterior. Then name one quantity from Chapter 2 for which the normalizing constant *is* needed, and explain why Metropolis draws do not directly give it to you.

Exercise 5.3 (Detailed balance with a twist) For a standard Normal target and a symmetric Normal random-walk proposal with positive step size, suppose someone uses a modified acceptance rule: always accept if $r \geq 1$, and accept with probability r^2 if $r < 1$. Does this chain satisfy detailed balance with respect to f? If not, describe in words how its stationary distribution differs from f.

Exercise 5.4 (Reading a trace plot) Three long Metropolis runs use the same one-dimensional Normal target and symmetric Normal random-walk proposals with step sizes 0.01, 1, and 100. Their acceptance rates are 0.02, 0.98, and 0.45, but the labels are lost. Give the most plausible matching, explain the expected trace behavior, and state what further information you would inspect before choosing a step size.

5.10.2 Practical exercises

Exercise 5.5 (Gamma-Poisson by Metropolis) As shown in Chapter 2 and Section D.3, a Poisson rate λ with a Gamma(s, r) prior has a Gamma$(s + \sum y_i, r + n)$ posterior. Take $s = 3$, $r = 1$, and the data $y = (2, 0, 4, 1, 3, 2)$.

a. Write `log_posterior(lambda)` for this model. What should it return for negative `lambda`?
b. Run `metropolis()` for 10,000 iterations from `init = 1` with step size 0.5. Report the acceptance rate and compare the histogram of the draws with the exact Gamma posterior.
c. Repeat with step sizes 0.05 and 5. Compare acceptance rates, trace plots, and effective sample sizes.

Exercise 5.6 (Warm-up) Run the sleep sampler with step size 0.5 from the starting values $\mu^{(0)} = 0$, $\mu^{(0)} = 6.7$, and $\mu^{(0)} = 20$, for 2,000 iterations each.

a. Plot the three traces on the same axes. How many iterations does each chain need to reach the posterior region?
b. For each chain, compute the posterior mean from all 2,000 draws and from the draws after discarding the first 200. Which estimate is affected more by the starting value, and why?

Exercise 5.7 (Two chains catch a mode) Return to the bimodal target `log_bimodal()` with step size 0.5.

a. Run two chains of 5,000 iterations, one from -5 and one from 5. Compute the posterior mean of θ from each chain separately and from the two chains pooled.
b. Compute $\widehat{R}$ for θ using `posterior::rhat()` on a matrix with one column per chain. What does the value tell you?
c. Compare step sizes 0.5, 1, 2, and 5 using the two chains at each value. Report $\widehat{R}$, acceptance rates, and the fraction of states above zero in each chain. Which settings give more convincing exploration, and why does $\widehat{R} < 1.01$ alone not settle the question?

Exercise 5.8 (Two parameters) Extend the sleep example by treating σ as unknown, with prior $\sigma \sim \text{Exponential}(1)$, so that the parameter vector is (μ, σ).

a. Write `log_posterior(theta)` where `theta = c(mu, sigma)`. Make sure it returns `-Inf` when `sigma` is not positive.
b. Run the vector version of `metropolis()` from `init = c(7, 1)` for 20,000 iterations. Try step sizes 0.1, 0.3, and 1. Compare traces and the ESS of both parameters after discarding the first 500 iterations, and explain your choice of step size.
c. Plot the joint draws of (μ, σ) and report a 95% credible interval for σ. How does the interval for μ compare with the one where σ was fixed at 1?

5.10.3 Applied exercises

Exercise 5.9 (Ghosts and age) The `pulse_of_the_nation` data in the `bayesrules` package record, among other things, whether each of 1,000 respondents believes in ghosts and their age. Consider the logistic regression

$$Y_i \mid \beta_0, \beta_1 \sim \text{Bernoulli}(\pi_i), \qquad \log \frac{\pi_i}{1-\pi_i} = \beta_0 + \beta_1 \,\text{age}_i,$$

with $Y_i = 1$ for a believer, and independent $\text{N}(0, 1)$ priors on β_0 and β_1 after centering and scaling age.

a. Write `log_posterior(theta)` for `theta = c(beta_0, beta_1)`. Use `dbinom(y, 1, plogis(eta), log = TRUE)` for the log likelihood and check that it returns a finite number at `theta = c(0, 0)`.
b. Run the vector version of `metropolis()` for 20,000 iterations from `c(0, 0)`. Compare a few step sizes, using acceptance between 0.2 and 0.4 as an initial guide. Choose using traces and the ESS of both coefficients after discarding the first 500 iterations, and report your comparison.
c. Plot the joint draws of (β_0, β_1). Are the two coefficients correlated in the posterior? Would they be if age were not centered?
d. Report a 95% posterior credible interval for β_1 and the posterior probability that older respondents are less likely to believe in ghosts.
e. Fit the same model with `rstan` (see Chapter 4 for the workflow) and compare the posterior means and intervals.

References

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., & Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, *76*(1), 1–32.

Dickey, J. M., & Lientz, B. P. (1970). The weighted likelihood ratio, sharp hypotheses about chances, the order of a markov chain. *Annals of Mathematical Statistics*, *41*(1), 214–226.

Gelman, A., & Rubin, D. B. (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, *7*(4), 457–472.

Hanada, M., & Matsuura, S. (2022). *MCMC from scratch: A practical introduction to markov chain monte carlo*. Springer.

Jeffreys, H. (1961). *Theory of probability* (3rd ed.). Oxford University Press.

Johnson, A. A., Ott, M. Q., & Dogucu, M. (2022). *Bayes rules! An introduction to applied Bayesian modeling*. CRC Press.

Lee, M. D., & Wagenmakers, E.-J. (2013). *Bayesian cognitive modeling: A practical course*. Cambridge University Press.

Logan, G. D., & Cowan, W. B. (1984). On the ability to inhibit thought and action: A theory of an act of control. *Psychological Review*, *91*(3), 295–327. https://doi.org/10.1037/0033-295X.91.3.295

Margossian, C. C., & Gelman, A. (2026). For how many iterations should we run Markov chain Monte Carlo? In R. V. Craiu, D. Vats, G. L. Jones, S. Brooks, A. Gelman, & X.-L. Meng (Eds.), *Handbook of Markov chain Monte Carlo* (2nd ed., pp. 110–131). CRC Press. https://doi.org/10.1201/9781003453420-4

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, *21*(6), 1087–1092.

Norris, J. R. (1997). *Markov chains*. Cambridge University Press.

Robert, C. P., & Casella, G. (2004). *Monte Carlo statistical methods* (2nd ed.). Springer.

Roberts, G. O., Gelman, A., & Gilks, W. R. (1997). Weak convergence and optimal scaling of random walk metropolis algorithms. *The Annals of Applied Probability*, *7*(1), 110–120.

Roberts, G. O., & Rosenthal, J. S. (2001). Optimal scaling for various Metropolis–Hastings algorithms. *Statistical Science*, *16*(4), 351–367.

van den Bergh, D., Clyde, M. A., Raftery, A. E., & Marsman, M. (2026). *Reversible jump MCMC with no regrets: Bayesian variable selection using mixtures of mutually singular distributions*.

Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., & Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, *16*(2), 667–718.

Wagenmakers, E.-J., Lodewyckx, T., Kuriyal, H., & Grasman, R. P. P. P. (2010). Bayesian hypothesis testing for psychologists: A tutorial on the savage–dickey method. *Cognitive Psychology*, *60*(3), 158–189.

News

Versions of this website, newest first. Every version remains available through the version menu in the sidebar, and every change to the text and code is recorded in the GitHub repository.

Development version

Not yet released. Online in the development version:

- the welcome page;
- Chapter 1, Probability and Simulation;
- Chapter 2, Bayesian Inference, Revisited;
- Chapter 3, Markov Chains;
- Chapter 4, Diagnosing Markov Chains;
- Chapter 5, The Metropolis Algorithm;
- Appendices A–D: software setup, notation, technical notes on MCMC (including the acceptance-probability derivation), and conjugate-posterior derivations.

Appendix A

Software Setup

Everything in this book runs in R. Most of it needs nothing beyond a handful of CRAN packages, because the point of the book is that you write the samplers yourself. The one exception is Stan, which we use in Chapter 4 and Chapter 10 (Hamiltonian Monte Carlo), and which needs a C++ compiler on your machine. This appendix walks you through the installation in the order that causes the fewest problems, gives you a small model to confirm that Stan works, and tells you what to do if it does not.

If you took the Bayesian Statistics course on this machine, you probably already have everything. Skip to Section A.4 and run the test model; if it passes, you are done.

A.1 R and RStudio

Install R first and RStudio second; RStudio is an editor and needs R to be there already.

1. **R (4.4 or later).** Download the installer for your operating system from cran.r-project.org. On macOS, pick the build that matches your processor (Apple silicon or Intel). If you already have R, check its version with `R.version.string` and upgrade if it is older than 4.4, because several packages we use no longer support older versions.
2. **RStudio.** Download RStudio Desktop from posit.co. Any recent version is fine.

Both are free. Quarto, which we use to build this book, ships inside RStudio; you do not need it for the course.

A.2 R packages

The packages below cover the whole book. Copy this block into the R console and run it once. It downloads binaries on macOS and Windows and takes a few minutes; on Linux it compiles from source and takes longer.

```
install.packages(c(
  "tidyverse",    # data handling and plots
  "bayesrules",   # data sets from Bayes Rules!
  "bayesplot",    # trace plots and other MCMC graphics
  "posterior",    # effective sample size, R-hat, draws objects
  "coda",         # classic MCMC diagnostics
  "mvtnorm",      # multivariate Normal densities and draws
  "patchwork",    # combining ggplot2 figures
  "rstan",        # Stan
  "bgms",         # Bayesian graphical models, Chapters 7-9
  "lars"          # the diabetes data, Chapters 8-9
))
```

`rstan` is on CRAN like the others, and on macOS and Windows it installs as a pre-built binary without compiling anything. That is the easy half. The hard half is that *using* `rstan` means compiling each Stan model into C++ the first time you run it, and for that your machine needs a C++ toolchain. The next section installs it.

A.3 Installing Stan

`rstan` is the R interface to Stan (Carpenter et al., 2017). The Stan program you write is translated to C++ and compiled, so you need a compiler that R knows how to call. What that means depends on your operating system.

macOS. Install the Xcode Command Line Tools by opening the Terminal app and running

```
xcode-select --install
```

A dialog asks you to confirm; the download is a few gigabytes and takes a while. You do not need the full Xcode application from the App Store. If you already have the tools, the command says so and you can move on.

Windows. Install **Rtools** from CRAN, choosing the version that matches your R version: Rtools 4.4 for R 4.4.x, Rtools 4.5 for R 4.5.x, and so on. The CRAN download page for Windows lists them under "Rtools". Accept the default installation location. A mismatch between the Rtools version and the R version is the most common reason that Stan will not compile on Windows, so check `R.version.string` before you download.

Linux. You need `g++` and `make`, which on Debian and Ubuntu come with the `build-essential` package, plus the R development headers (`r-base-dev`). `install.packages("rstan")` then compiles `rstan` and its dependencies from source, which takes ten to twenty minutes. The Stan website, mc-stan.org, has distribution-specific notes.

On all three platforms, restart R after installing the toolchain, and confirm that R can see it:

```
pkgbuild::has_build_tools(debug = TRUE)
```

The answer should be `TRUE`. If it is `FALSE`, the toolchain is not installed or R cannot find it, and no Stan model will compile until that is fixed; on Windows, an Rtools version mismatch shows up here.

Two settings worth putting in every Stan script

```
library(rstan)
options(mc.cores = parallel::detectCores())    # run the chains in parallel
rstan_options(auto_write = TRUE)               # keep the compiled model on disk
```

The first line runs one chain per processor core instead of one after the other. The second saves the compiled C++ next to your Stan file so that the same model is not recompiled every time you fit it. Both are the settings *Bayes Rules!* uses, and every Stan chunk in this book assumes them.

A.4 Checking your installation

Here is the model you will meet again in Chapter 4: the Beta-Binomial. Suppose 14 of 20 students in a seminar say they use a spaced-repetition app, and suppose a Beta$(2, 2)$ prior for the proportion π who do. The posterior is Beta$(16, 8)$, with mean $16/24 \approx 0.667$, and that number is what we check against.

Run the following in a fresh R session. The first time you run it, it will take about a minute, and you will see a line saying that the model is being compiled, possibly followed by compiler warnings. The warnings are normal; only an error that stops the run is a problem.

```
library(rstan)
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

bb_model <- "
data {
  int<lower=0> n;
  int<lower=0, upper=n> y;
}
parameters {
  real<lower=0, upper=1> pi;
}
model {
  pi ~ beta(2, 2);
  y ~ binomial(n, pi);
}
"

fit <- stan(model_code = bb_model, data = list(n = 20, y = 14),
            chains = 4, iter = 2000, seed = 84735)
print(fit)
```

Success looks like this. `print(fit)` starts with a header saying "4 chains, each with iter=2000; warmup=1000" and "total post-warmup draws=4000", followed by a table with a row for `pi` and a row for `lp__`. The `mean` for `pi` is 0.67 give or take 0.01, `Rhat` is 1, and `n_eff` is somewhere in the low thousands. If you see that, Stan compiles and runs on your machine, and you can use every chapter of the book.

Two things that look alarming but are not: `rstan` may print long compiler warnings about "unused variable" or "deprecated" declarations while compiling, and on macOS the very first compilation can take noticeably longer than a minute. Two things that are actual failures: an error mentioning `make`, `g++`, `clang`, or "compilation failed" means the toolchain is missing or not found, so go back to the previous section; an error mentioning `stanc` or a line number in the model is a mistake in the Stan code, so check that you copied the model exactly.

A.5 If rstan will not compile

On some machines, most often a managed university laptop or a very new operating system that CRAN's binaries do not yet target, `rstan` installs fine but refuses to compile any model, and the error message does not point to a fix. Rather than lose an afternoon, switch to `cmdstanr`. It is a second R interface to the same Stan, developed by the Stan team, that talks to a separate CmdStan installation instead of compiling inside R's own toolchain. It is less convenient to install but more robust once it runs.

`cmdstanr` is not on CRAN. Install it from the Stan team's R-universe repository, let it repair the toolchain, and then let it download and build CmdStan:

```
install.packages("cmdstanr",
                 repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
library(cmdstanr)
check_cmdstan_toolchain(fix = TRUE)    # installs what the compiler is missing, if it
  can
install_cmdstan(cores = 2)             # downloads and builds CmdStan; several minutes
cmdstan_version()                      # a version number means it worked
```

On Windows, `check_cmdstan_toolchain(fix = TRUE)` also needs Rtools, so install that first. On macOS

it needs the Xcode Command Line Tools, so the previous section still applies; what `cmdstanr` sidesteps is the integration between the compiler and R's own build settings, which is where `rstan` usually fails.

All Stan code in this book is written for `rstan`, and translating it takes two lines. Where the book writes

```
fit <- stan(model_code = bb_model, data = bb_data, chains = 4, iter = 2000, seed =
  84735)
```

you instead compile and sample in two steps:

```
mod <- cmdstan_model(write_stan_file(bb_model))
fit <- mod$sample(data = bb_data, chains = 4, iter_warmup = 1000, iter_sampling =
  1000, seed = 84735)
```

Note that `cmdstanr` counts warm-up and sampling iterations separately, while `rstan`'s `iter` includes warm-up (by default, the first half). Everything after sampling can be the same, because the `posterior` and `bayesplot` packages work with both kinds of fit. To get a draws object that the book's diagnostic code can use:

```
draws <- posterior::as_draws_array(as.array(fit))   # rstan
draws <- fit$draws()                                # cmdstanr
```

From here on, `posterior::summarise_draws(draws)` and `bayesplot::mcmc_trace(draws)` behave identically. If you use `cmdstanr`, `fit$summary()` plays the role of `print(fit)`, and `fit$draws(format = "df")` gives you the draws as a data frame.

A.6 Reproducibility: seeds and sessions

Every method in this book is a simulation, so running the same code twice gives different numbers unless you fix the random number generator. Two seeds matter.

R's seed. `set.seed()` fixes the state of R's random number generator, so that every `rnorm()`, `runif()`, and `sample()` call that follows is reproducible. The book sets `set.seed(84735)` at the top of every chapter, a nod to *Bayes Rules!*. Set it once, at the top of your script, not before every sampler call: re-seeding inside a loop gives you the same "random" numbers again and again.

Stan's seed. Stan runs outside R and has its own generator, so `set.seed()` does not reach it. Pass `seed =` to `stan()` (or to `$sample()` in `cmdstanr`), as the test model above does. With the same seed, the same model, the same data, and the same number of chains, Stan produces the same draws, though not necessarily across different Stan versions or operating systems.

```
set.seed(84735)
x <- rnorm(3)
set.seed(84735)
identical(x, rnorm(3))
```

```
[1] TRUE
```

Reproducible does not mean correct. A seed makes a bug repeatable, which is useful for finding it, but when you want to know whether a sampler works, run it with *several* seeds and check that the conclusions do not change. Chapter 4 makes this idea precise.

Finally, when you report a problem or hand in an assignment, include the output of `sessionInfo()`. It lists your R version, your operating system, and the version of every loaded package, which is usually the first thing anyone helping you will ask for.

```
sessionInfo()
```

Setup checklist

1. R 4.4 or later, and RStudio.
2. The package block above, run once.
3. A C++ toolchain: Xcode Command Line Tools (macOS), Rtools matching your R version (Windows), or `build-essential` (Linux).
4. The Beta-Binomial test model compiles and reports a posterior mean for `pi` near 0.67.
5. If step 4 fails: `cmdstanr` from the Stan R-universe, `check_cmdstan_toolchain(fix = TRUE)`, `install_cmdstan()`.
6. Seeds: `set.seed()` for R, `seed =` for Stan; `sessionInfo()` when asking for help.

Appendix B

Notation

This appendix collects the symbols used throughout the book. We follow *Bayes Rules!* (Johnson et al., 2022) wherever it has a convention, so that nothing you learned there has to be unlearned, and add the symbols that Markov chains and samplers need.

B.1 Conventions

Every density is f. The prior, the likelihood, the posterior, and the density of a proposal are all written $f(\cdot)$, and the arguments tell you which one is meant: $f(\theta)$ is the prior, $f(y \mid \theta)$ the likelihood, and $f(\theta \mid y)$ the posterior. We do not use $p(\cdot)$ or $\pi(\cdot)$ for densities, because π is a proportion parameter in *Bayes Rules!* and $\mathbf{p}$ is a momentum in Chapter 10 (Hamiltonian Monte Carlo). When a chapter builds a sampler for a generic target, that target is written $f(\theta)$, and you should read it as "the posterior, up to a constant we cannot compute."

Work on the log scale. Every sampler in this book takes a function that returns $\log f(\theta)$, never $f(\theta)$, and the code calls it `log_target` or `log_posterior`. The reason is numerical: a likelihood of 2,000 observations is a product of 2,000 small numbers and underflows to zero, while its logarithm is a sum of 2,000 moderate numbers. On the log scale a ratio becomes a difference, $\log r = \log f(\theta^\star) - \log f(\theta^{(t)})$, and the acceptance test becomes $\log u < \log r$ (Chapter 5). R's `d*` functions all take `log = TRUE` for exactly this purpose.

Vectors are bold, indices are subscripts, iterations are superscripts. A single parameter is θ; a vector of parameters is $\theta = (\theta_1, \ldots, \theta_d)$; a matrix is a bold capital such as $\mathbf{P}$ or Σ. Observations are indexed $i = 1, \ldots, n$, predictors $j = 1, \ldots, p$, and models $m = 1, \ldots, M$. The state of a Markov chain at iteration t is $\theta^{(t)}$, with parentheses so that it is not mistaken for a power; a proposed value is $\theta^\star$. Chains start at $t = 0$ and run for $t = 1, \ldots, T$ iterations. Hats denote estimates computed from draws, as in $\widehat{R}$.

B.2 Data, parameters, and densities

Symbol	Meaning
y, $\mathbf{y} = (y_1, \ldots, y_n)$	observed data; n observations
$\mathbf{X}$, x_{ij}	design matrix with n rows and p predictors
θ, θ	a generic parameter; a vector of d parameters
π	a proportion parameter, as in *Bayes Rules!*
μ, σ, σ^2	a mean, a standard deviation, a variance
$\beta_0, \beta_1, \ldots, \beta_p$	regression coefficients
$f(\theta)$	prior density
$f(y \mid \theta)$, $L(\theta \mid y)$	likelihood; the same function read as a function of θ
$f(\theta \mid y)$	posterior density

Symbol	Meaning
$f(y) = \int f(y \mid \theta) f(\theta)\, d\theta$	marginal likelihood, the normalizing constant of the posterior
$f(\theta \mid y) \propto f(y \mid \theta) f(\theta)$	the unnormalized posterior, which is all a sampler needs
$E[g(\theta) \mid y]$	a posterior expectation, the thing Monte Carlo estimates (Chapter 1)
$\mathrm{BF}_{01} = f(y \mid \mathcal{M}_0) / f(y \mid \mathcal{M}_1)$	Bayes factor for model $\mathcal{M}_0$ against $\mathcal{M}_1$ (Chapter 2)

B.3 Markov chains and samplers

Symbol	Meaning
$\theta^{(t)}$	state of a Markov chain at iteration t; $\theta^{(0)}$ is the starting value
T	number of iterations; draws after warm-up are $\theta^{(t)}$ for $t > T_{\text{warm}}$
$\theta^\star$	a proposed value
$q(\theta^\star \mid \theta)$	proposal density; symmetric in the Metropolis algorithm, general in Metropolis–Hastings
r	acceptance ratio, $f(\theta^\star)/f(\theta^{(t)})$ for a symmetric proposal
$\alpha(\theta, \theta^\star) = \min\{1, r\}$	acceptance probability
$u \sim \mathrm{Unif}(0, 1)$	the uniform draw that decides acceptance
c	step size of a random-walk proposal, $\theta^\star \sim \mathrm{N}(\theta^{(t)}, c^2)$
α^*, γ_t	target acceptance rate and learning rate in Robbins–Monro adaptation (Chapter 6 (Metropolis–Hastings and Adaptive Proposals))
$\mathbf{P}$, P_{ij}	transition matrix of a discrete chain; probability of moving from state i to state j (Chapter 3)
$K(\theta, \theta')$	transition kernel of a continuous chain, the density of moving from θ to θ'
π	stationary distribution of a discrete chain, a row vector with $\pi\mathbf{P} = \pi$
$f(\theta)K(\theta, \theta') = f(\theta')K(\theta', \theta)$	detailed balance with respect to f
$f(\theta_j \mid \theta_{-j}, y)$	full conditional of θ_j given all other parameters (Chapter 7 (Gibbs Sampling and Metropolis-within-Gibbs))
ρ_k	autocorrelation of a chain at lag k (Chapter 4)
ESS	effective sample size, the number of independent draws the chain is worth
$\widehat{R}$	R-hat, the square root of the ratio of between-plus-within to within-chain variance; close to 1 at convergence
MCSE	Monte Carlo standard error of a posterior estimate, roughly $\mathrm{sd}/\sqrt{\mathrm{ESS}}$

B.4 Model uncertainty

Symbol	Meaning
$\mathcal{M}_m$, $m = 1, \ldots, M$	model m out of M candidate models (Chapter 8 (Bayesian Variable Selection))
$\gamma_j \in \{0, 1\}$	inclusion indicator for predictor j: $\gamma_j = 1$ means x_j is in the model
$\gamma = (\gamma_1, \ldots, \gamma_p)$	a model, written as its vector of indicators; there are 2^p of them
$f(\mathcal{M}_m)$, $f(\mathcal{M}_m \mid y)$	prior and posterior model probabilities
$f(y \mid \mathcal{M}_m)$	marginal likelihood of model m
$P(\gamma_j = 1 \mid y)$	posterior inclusion probability of predictor j
$f(\theta \mid y) = \sum_m f(\theta \mid y, \mathcal{M}_m) f(\mathcal{M}_m \mid y)$	model-averaged posterior (Chapter 9 (Bayesian Model Averaging))

B.5 Hamiltonian Monte Carlo

Symbol	Meaning
$\mathbf{p}$	momentum, an auxiliary vector of the same length as θ (Chapter 10 (Hamiltonian Monte Carlo))
$\mathbf{M}$	mass matrix, the covariance of the momentum, $\mathbf{p} \sim \mathrm{N}(\mathbf{0}, \mathbf{M})$
$H(\theta, \mathbf{p}) = -\log f(\theta) + \frac{1}{2}\mathbf{p}^\top \mathbf{M}^{-1} \mathbf{p}$	the Hamiltonian: potential energy plus kinetic energy
$\nabla \log f(\theta)$	gradient of the log target, the `grad_log_target` argument of `hmc()`
ε	leapfrog step size
L	number of leapfrog steps per proposal; trajectory length is εL

Note the three different uses of π-like and γ-like symbols: plain π is a proportion, bold $\boldsymbol{\pi}$ is a stationary distribution, and $\pi = 3.14\ldots$ never appears except inside the Normal density; γ_j with a predictor index is an inclusion indicator, while γ_t with an iteration index is a learning rate. Context and the index should always make clear which is meant.

B.6 Distributions

The book writes distributions in the abbreviations below, parameterized the way *Bayes Rules!* does. The last column gives the R function that computes the density, because you will write log posteriors as sums of `d*(..., log = TRUE)` calls, and R's argument names do not always match the mathematical parameters. Two traps: the Normal is written with its variance σ^2 but `dnorm()` takes the standard deviation `sd`, and the Gamma and Exponential are written with a *rate*, which is what `dgamma()` and `dexp()` take by default, not a scale. The `r*` function with the same name and arguments draws random values.

Notation	Parameters	Density in R
$\mathrm{N}(\mu, \sigma^2)$	mean μ, variance σ^2	`dnorm(x, mean = mu, sd = sigma)`
$\mathrm{N}(\mu, \Sigma)$	mean vector, covariance matrix	`mvtnorm::dmvnorm(x, mean = mu, sigma = Sigma)`
$\mathrm{Beta}(\alpha, \beta)$	shape parameters $\alpha, \beta > 0$; mean $\alpha/(\alpha+\beta)$	`dbeta(x, shape1 = alpha, shape2 = beta)`
$\mathrm{Gamma}(s, r)$	shape s, rate r; mean s/r	`dgamma(x, shape = s, rate = r)`
Inverse-$\mathrm{Gamma}(s, r)$	shape s, rate r; $1/X \sim \mathrm{Gamma}(s, r)$	no base function; `dgamma(1/x, s, r, log = TRUE) - 2 * log(x)`
$\mathrm{Exponential}(\lambda)$	rate λ; mean $1/\lambda$	`dexp(x, rate = lambda)`
$\mathrm{Unif}(a, b)$	lower and upper bounds	`dunif(x, min = a, max = b)`
$\mathrm{Cauchy}(m, s)$	location m, scale s	`dcauchy(x, location = m, scale = s)`
$\mathrm{Laplace}(m, b)$	location m, scale b	no base function; `-log(2 * b) - abs(x - m) / b` on the log scale
$\mathrm{Bernoulli}(\pi)$	success probability π	`dbinom(x, size = 1, prob = pi)`
$\mathrm{Binomial}(n, \pi)$	number of trials n, success probability π	`dbinom(x, size = n, prob = pi)`
$\mathrm{Poisson}(\lambda)$	rate λ; mean and variance λ	`dpois(x, lambda = lambda)`

A quick check of the two traps, which you can run yourself. The $\mathrm{N}(7, 0.5^2)$ prior of Chapter 5 has standard deviation 0.5, and a $\mathrm{Gamma}(2, 4)$ variable has mean $2/4$:

```
dnorm(7, mean = 7, sd = 0.5, log = TRUE)
```

```
[1] -0.2257914
```

```
mean(rgamma(1e5, shape = 2, rate = 4))
```

```
[1] 0.4986272
```

Appendix C

Technical Notes on MCMC

These notes expand on the diagnostics in Chapter 4 and derive the acceptance rule used in Chapter 5 and Chapter 6 (Metropolis–Hastings and Adaptive Proposals). Exercise 4.7 uses the R-hat functions in Section C.2.

C.1 The variance of a dependent average

For N independent draws, the variance of their average is σ^2/N. With dependent draws, the covariances also contribute. For a stationary chain, $\mathrm{Cov}(\theta^{(t)}, \theta^{(t+k)}) = \sigma^2 \rho_k$. There are $N - k$ pairs of values separated by lag k, giving

$$\mathrm{Var}(\bar{\theta}) = \frac{\sigma^2}{N}\left[1 + 2\sum_{k=1}^{N-1}\left(1 - \frac{k}{N}\right)\rho_k\right]. \tag{C.1}$$

This expression is exact for a stationary chain with finite variance. When N is large relative to the lags with appreciable correlation, the weights $1 - k/N$ are close to one. Extending the sum gives the large-N approximation used in Chapter 4:

$$\mathrm{Var}(\bar{\theta}) \approx \frac{\sigma^2}{N}\left(1 + 2\sum_{k=1}^{\infty}\rho_k\right).$$

Positive autocorrelation increases this variance. Negative autocorrelation can reduce it: for example, the AR(1) formula gives $N_{\text{eff}} > N$ when $\phi < 0$. Thinning such a chain can remove the benefit of its negative autocorrelation.

Bias and variation can also be combined in one expression. For an estimate $\hat{\mu}$ of a mean μ,

$$\mathbb{E}[(\hat{\mu} - \mu)^2] = \left(\mathbb{E}[\hat{\mu}] - \mu\right)^2 + \mathrm{Var}(\hat{\mu}). \tag{C.2}$$

The mean squared error on the left is the sum of squared bias and Monte Carlo variance. Bias describes a systematic displacement from the true mean; Monte Carlo variance describes how much the estimates vary between runs (Margossian & Gelman, 2026).

C.2 Calculating basic and split R-hat

For m chains of length L, let s_j^2 and $\bar{\theta}_j$ be the variance and mean within chain j, and $\bar{\theta}_\cdot$ the average of the chain means. Then

$$W = \frac{1}{m}\sum_{j=1}^{m} s_j^2, \qquad B = \frac{L}{m-1}\sum_{j=1}^{m}(\bar{\theta}_j - \bar{\theta}_\cdot)^2.$$

We illustrate the calculation with four records of independent standard Normal draws. The following functions implement the basic R-hat formula and split each column into equal-length first and last halves. For an odd number of values, the middle value is left out.

```r
rhat_by_hand <- function(x) {
  L <- nrow(x)
  W <- mean(apply(x, 2, var))
  B <- L * var(colMeans(x))
  sqrt(((L - 1) / L * W + B / L) / W)
}
split_halves <- function(x) {
  half <- floor(nrow(x) / 2)
  cbind(x[seq_len(half), , drop = FALSE],
        x[seq.int(nrow(x) - half + 1, nrow(x)), , drop = FALSE])
}
set.seed(48407)
normal_chains <- matrix(rnorm(4000), ncol = 4)
rbind(
  unsplit = c(by_hand = rhat_by_hand(normal_chains),
              package = rhat_basic(normal_chains, split = FALSE)),
  split = c(by_hand = rhat_by_hand(split_halves(normal_chains)),
            package = rhat_basic(normal_chains, split = TRUE))
)
#>          by_hand  package
#> unsplit 1.000113 1.000113
#> split   1.000048 1.000048
```

Within each row, the hand and package calculations match because they implement the same formula. The split and unsplit results can also look nearly identical after rounding when both halves explore similar values. `rhat_basic()` splits by default; `split = FALSE` disables that step.

C.3 Rank normalization and folding

Chains can agree about their center and still disagree about their spread. Suppose two records vary narrowly around zero while two others range much farther above and below it. Their means can be similar, so comparing means alone may give little warning.

The following code creates that situation. All four records have an expected mean of zero, but the last two have standard deviation three instead of one:

```r
set.seed(48408)
same_mean <- cbind(rnorm(1000, 0, 1), rnorm(1000, 0, 1), rnorm(1000, 0, 3),
  rnorm(1000, 0, 3))
c(basic = rhat_basic(same_mean), rank_normalized = posterior::rhat(same_mean))
#>           basic rank_normalized
#>        1.000529        1.187145
```

`posterior::rhat()` adds two improvements to the basic split calculation (Vehtari et al., 2021).

Rank normalization replaces values by their positions in an ordered list. For example, the values $(-3, -1, 1, 3)$ have ranks $(1, 2, 3, 4)$. An extremely large value still receives the highest rank, but its numerical size no longer dominates the comparison.

After splitting the chains, we rank all their retained values together. Ties receive their average rank. We then convert the ranks to standard Normal scores. If r is a rank and S the total number of ranked values, the transformation is

$$z = \Phi^{-1}\left(\frac{r - 3/8}{S + 1/4}\right).$$

The expression inside Φ^{-1} turns a rank into a probability between zero and one; Φ^{-1} gives the corresponding standard Normal quantile. For the four-value example, the scores are approximately $(-1.05, -0.30, 0.30, 1.05)$. We calculate basic R-hat on the scores. This makes the comparison less sensitive to extreme values, including those from distributions with very long tails. Bulk ESS uses this same transformation of the split chains before estimating ESS. Tail ESS instead computes ESS for indicators of falling below the pooled 5% quantile and below the pooled 95% quantile, and reports the smaller value.

Folding checks spread more directly. Replace each original value by its distance from the pooled median. In the four-value example the median is zero, so $(-3, -1, 1, 3)$ becomes $(3, 1, 1, 3)$. The sign disappears and distance from the center remains. Wider chains now tend to have larger values than narrower chains. We split and rank-normalize these distances, then calculate R-hat again.

The reported `posterior::rhat()` is the larger of these two results. Figure C.1 shows how folding separates records with different spreads. In this example, ranking the original values alone can miss the problem, while folding reveals it. The code below calculates both parts and compares their maximum with `posterior::rhat()`:

```
rank_normalize <- function(x) {
  ranks <- rank(x, ties.method = "average")
  scores <- qnorm((ranks - 3/8) / (length(ranks) + 1/4))
  matrix(scores, nrow = nrow(x), ncol = ncol(x))
}
rank_component <- rhat_by_hand(rank_normalize(split_halves(same_mean)))
folded <- abs(same_mean - median(same_mean))
fold_component <- rhat_by_hand(rank_normalize(split_halves(folded)))
c(rank_component = rank_component, folded_component = fold_component,
  maximum = max(rank_component, fold_component), package = posterior::rhat(same_mean))
#>   rank_component folded_component          maximum          package
#>         1.000340         1.187145         1.187145         1.187145
```

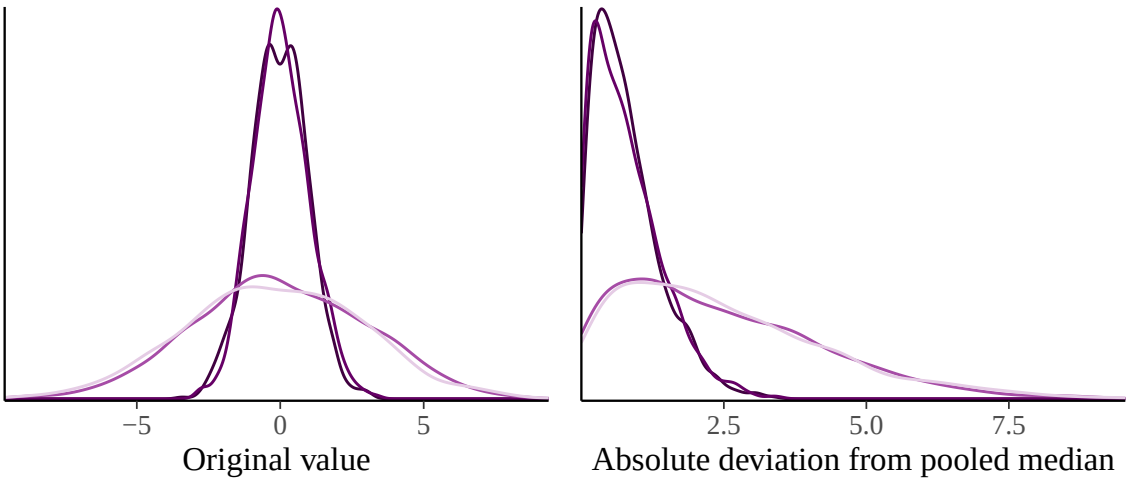


Figure C.1: Folding makes a spread difference easier to compare. Left: records share a center but have different spreads. Right: absolute deviations from the pooled median are systematically larger in the wider records.

C.4 Deriving the Metropolis acceptance probability

The Metropolis rule accepts every proposal to a higher target density and only some proposals to a lower one. How do we obtain these probabilities? We can choose them by solving the detailed balance equation (van den Bergh et al., 2026, Appendix B).

Let a be the current state and b a proposed state. Write α_{fwd} for the probability of accepting the move from a to b, and α_{rev} for the acceptance probability if we instead propose a from b. These are probabilities for two possible directions, not two decisions made in one iteration. For this derivation, assume that both states have positive, finite target density and that the proposal densities in both directions are positive.

C.4.1 Relating the two directions

Detailed balance is satisfied when

$$f(a)q(b \mid a)\alpha_{\text{fwd}} = f(b)q(a \mid b)\alpha_{\text{rev}}. \tag{C.3}$$

For the symmetric proposals in Chapter 5, $q(b \mid a) = q(a \mid b)$, so these factors cancel. Dividing by $f(a)$ leaves

$$\alpha_{\text{fwd}} = r\alpha_{\text{rev}}, \qquad r = \frac{f(b)}{f(a)}. \tag{C.4}$$

Once we choose an acceptance probability in one direction, this equation fixes the probability in the other. The choices must also lie between zero and one.

Since $\alpha_{\text{rev}} \leq 1$, the equation gives $\alpha_{\text{fwd}} \leq r$. We also require $\alpha_{\text{fwd}} \leq 1$. The largest permitted forward probability is therefore $\min(1, r)$. Taking this value gives the Metropolis rule:

$$\alpha_{\text{fwd}} = \min(1, r), \qquad \alpha_{\text{rev}} = \min(1, 1/r). \tag{C.5}$$

If $r < 1$, set $\alpha_{\text{rev}} = 1$ and obtain $\alpha_{\text{fwd}} = r$. The proposal goes to a lower target density, so it is accepted only sometimes. If $r \geq 1$, set $\alpha_{\text{fwd}} = 1$ and obtain $\alpha_{\text{rev}} = 1/r$. The proposal goes to an equal or higher target density and is always accepted. In either case, the equation is satisfied in both directions. Proposals outside the target support are rejected.

C.4.2 A numerical comparison

Suppose $f(a) = 0.2$, $f(b) = 0.1$, and the symmetric proposal density is 0.3 in either direction. Then $r = 0.5$. The two sides of Equation C.3 are

$$\underbrace{0.2 \times 0.3 \times 0.5}_{a \text{ to } b} = \underbrace{0.1 \times 0.3 \times 1}_{b \text{ to } a} = 0.03.$$

The target assigns a higher density to a, but the smaller acceptance probability from a to b compensates for that difference.

There are other balanced choices. For example, $\alpha_{\text{fwd}} = 0.25$ and $\alpha_{\text{rev}} = 0.5$ also satisfy Equation C.4, since $0.25 = 0.5 \times 0.5$. Both sides of the balance equation then equal 0.015. Both directions would produce more rejections. The Metropolis choice makes both acceptance probabilities as large as possible while preserving their required relationship; detailed balance alone does not specify a unique rule.

C.4.3 Allowing asymmetric proposals

When proposal densities differ between directions, retain them in Equation C.3. The same rearrangement gives

$$\alpha_{\text{fwd}} = R\alpha_{\text{rev}}, \qquad R = \frac{f(b)q(a \mid b)}{f(a)q(b \mid a)}.$$

Applying the same bounds yields the **Metropolis–Hastings** acceptance probability

$$\alpha(a, b) = \min\left\{1, \frac{f(b)q(a \mid b)}{f(a)q(b \mid a)}\right\}. \tag{C.6}$$

The reverse proposal density is in the numerator and the forward proposal density is in the denominator. This adjustment accounts for a proposal mechanism that favors one direction. Chapter 6 (Metropolis–Hastings and Adaptive Proposals) develops examples of such proposals. With a symmetric proposal, their ratio is one and the expression reduces to the rule derived above.

Appendix D

Deriving Conjugate Posteriors

In the modern art example in Chapter 2, a Beta prior and a Binomial likelihood give a Beta posterior. The same kind of calculation gives the other two updates in Table 2.1. Here we work through all three, beginning with the probability functions and ending with the posterior parameters.

D.1 What we need to recognize

Bayes' rule says that the posterior is proportional to the likelihood times the prior. To identify its distribution, we first keep all the factors that depend on the unknown parameter. Their product is a **kernel**: it gives the shape of a density without necessarily including the constant that makes its integral equal to one.

For example, multiplying a function of π by 10 changes its height at every π by the same factor. Once we divide by its integral, that factor cancels. Multiplying by π, however, changes the relative heights at different values of π and therefore changes the distribution. This is why we can set aside factors that are constant with respect to the parameter, but must retain factors that vary with it.

We then compare the kernel with a familiar density on the same range of possible parameter values. If they match, the familiar density supplies both the distribution's name and its normalizing constant. On the log scale, constant factors become additive terms. We write these as C, allowing its value to change as we collect terms. Such constants may depend on the observed data; they must not depend on the parameter whose posterior we are finding.

The constants can be set aside when identifying the posterior. Computing the marginal likelihood requires the full likelihood and prior, including their constants, as in the model comparisons in Chapter 2.

D.2 A Binomial likelihood and a Beta prior

Let y be the observed number of successes in n trials, with common success probability π. Under the Binomial model,

$$f(y \mid \pi) = \binom{n}{y} \pi^y (1-\pi)^{n-y}, \qquad 0 < \pi < 1.$$

Here n and y are known. The factor $\binom{n}{y}$ counts how many trial sequences give y successes; it does not change as we vary π.

Assign π a Beta(α, β) prior, with $\alpha > 0$ and $\beta > 0$. Its density is

$$f(\pi) = \frac{1}{B(\alpha, \beta)} \pi^{\alpha-1} (1-\pi)^{\beta-1}, \qquad 0 < \pi < 1.$$

The Beta function $B(\alpha, \beta)$ is the integral of $\pi^{\alpha-1}(1-\pi)^{\beta-1}$ over $(0, 1)$. Dividing by it makes the prior integrate to one. Since α and β are fixed prior parameters, it too is constant with respect to π.

Multiplying the likelihood and prior, and setting aside these constant factors, gives

$$\begin{aligned} f(\pi \mid y) &\propto \pi^{y}(1-\pi)^{n-y}\pi^{\alpha-1}(1-\pi)^{\beta-1} \\ &= \pi^{\alpha+y-1}(1-\pi)^{\beta+n-y-1}. \end{aligned}$$

The second line uses the rule that exponents add when we multiply powers with the same base. The y successes add y to the exponent of π; the $n-y$ failures add $n-y$ to the exponent of $1-\pi$.

A Beta density with parameters a and b has exponents $a-1$ and $b-1$. Matching them gives $a=\alpha+y$ and $b=\beta+n-y$. The range is still $(0,1)$, so

$$\pi \mid y \sim \text{Beta}(\alpha+y,\ \beta+n-y).$$

We can now write the normalized posterior explicitly:

$$f(\pi \mid y) = \frac{\pi^{\alpha+y-1}(1-\pi)^{\beta+n-y-1}}{B(\alpha+y, \beta+n-y)}, \qquad 0<\pi<1.$$

The denominator uses the **updated** parameters. Reusing the prior's normalizing constant would not make this new density integrate to one.

For the modern art data, $\alpha=4$, $\beta=6$, $n=100$, and $y=14$. The posterior is therefore Beta$(18,92)$. Its mean is $18/(18+92)\approx 0.164$, between the prior mean 0.4 and the sample proportion 0.14. More generally, for $n>0$,

$$E[\pi \mid y] = \frac{\alpha+y}{\alpha+\beta+n} = \frac{\alpha+\beta}{\alpha+\beta+n}\frac{\alpha}{\alpha+\beta} + \frac{n}{\alpha+\beta+n}\frac{y}{n}.$$

The last expression is a weighted average of the prior mean and the sample proportion. The weights add to one, and the data receive more weight as n increases while the prior stays fixed.

D.3 A Poisson likelihood and a Gamma prior

Now suppose $y_1,\dots,y_n$ are counts from n observation periods of equal length. We model them as independent given a common expected count λ:

$$Y_i \mid \lambda \sim \text{Pois}(\lambda), \qquad i=1,\dots,n.$$

For a single observation, the probability is $\lambda^{y_i}e^{-\lambda}/y_i!$. Conditional independence lets us multiply these probabilities to obtain the joint likelihood:

$$\begin{aligned} f(\mathbf{y} \mid \lambda) &= \prod_{i=1}^{n} \frac{\lambda^{y_i}e^{-\lambda}}{y_i!} \\ &= \frac{\lambda^{\sum_{i=1}^{n} y_i}e^{-n\lambda}}{\prod_{i=1}^{n} y_i!}. \end{aligned}$$

There are two combinations to notice. Multiplying the powers of λ adds the observed counts in the exponent. Multiplying n copies of $e^{-\lambda}$ gives $e^{-n\lambda}$. The factorials in the denominator involve only the observed counts, so the likelihood kernel is

$$\lambda^{\sum_i y_i}e^{-n\lambda}, \qquad \lambda>0.$$

Use a Gamma(s,r) prior with shape $s>0$ and **rate** $r>0$:

$$f(\lambda) = \frac{r^{s}}{\Gamma(s)}\lambda^{s-1}e^{-r\lambda}, \qquad \lambda>0.$$

Here $\Gamma(s)$ is the Gamma function; together with r^s it supplies the normalizing constant. To recognize the distribution we need the power $s-1$ and the coefficient r in the exponential. The rate convention matters: a Gamma density written using a scale parameter would instead have $e^{-\lambda/\text{scale}}$.

Multiplying the two kernels gives

$$\begin{aligned} f(\lambda \mid \mathbf{y}) &\propto \lambda^{\sum_i y_i} e^{-n\lambda} \lambda^{s-1} e^{-r\lambda} \\ &= \lambda^{s+\sum_i y_i - 1} e^{-(r+n)\lambda}. \end{aligned}$$

This has the Gamma form again: the shape is $s + \sum_i y_i$, and the rate is $r + n$. Thus

$$\lambda \mid \mathbf{y} \sim \text{Gamma}\left(s + \sum_{i=1}^{n} y_i,\ r + n \right).$$

The sum of the counts updates the shape; the number of observation periods updates the rate. Substituting these two parameters into the Gamma density above gives the normalized posterior:

$$f(\lambda \mid \mathbf{y}) = \frac{(r+n)^{s+\sum_i y_i}}{\Gamma(s + \sum_i y_i)} \lambda^{s+\sum_i y_i - 1} e^{-(r+n)\lambda}, \qquad \lambda > 0.$$

For example, suppose four equal-length task sessions produce counts of $2, 0, 3, 1$ attention lapses, and we use a Gamma$(2, 1)$ prior for the expected count per session. There are $n = 4$ sessions and $\sum_i y_i = 6$ lapses, giving a Gamma$(8, 5)$ posterior. The prior mean is $2/1 = 2$, the observed average is $6/4 = 1.5$, and the posterior mean is $8/5 = 1.6$.

The weighted-average interpretation is visible algebraically as well:

$$E[\lambda \mid \mathbf{y}] = \frac{s + \sum_i y_i}{r + n} = \frac{r}{r+n}\frac{s}{r} + \frac{n}{r+n}\bar{y}.$$

Here s/r is the prior mean and $\bar{y}$ is the average observed count. With the prior fixed, more observation periods give the observed average more weight.

D.4 A Normal likelihood and a Normal prior

Suppose the observations are independent given their common mean μ, with a known sampling variance $\sigma^2 > 0$:

$$Y_i \mid \mu \sim \text{N}(\mu, \sigma^2), \qquad i = 1, \dots, n.$$

Assign μ a $\text{N}(\theta, \tau^2)$ prior, where θ is the prior mean and $\tau^2 > 0$ is the prior variance. Here μ is the unknown parameter. The quantities θ, τ^2, and σ^2 are specified by the model.

The likelihood and prior are

$$\begin{aligned} f(\mathbf{y} \mid \mu) &= (2\pi\sigma^2)^{-n/2} \exp\left[-\frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \mu)^2 \right], \\ f(\mu) &= (2\pi\tau^2)^{-1/2} \exp\left[-\frac{(\mu - \theta)^2}{2\tau^2} \right]. \end{aligned}$$

The factors in front of the exponentials do not depend on μ. Taking logs turns multiplication of the kernels into addition:

$$\log f(\mu \mid \mathbf{y}) = -\frac{1}{2}\left[\frac{\sum_i (y_i - \mu)^2}{\sigma^2} + \frac{(\mu - \theta)^2}{\tau^2} \right] + C.$$

D.4.1 Collecting the terms in the unknown mean

To recognize a Normal density, we want one squared expression of the form $(\mu - m)^2$. First expand the squares already present:

$$\begin{aligned} \sum_{i=1}^{n} (y_i - \mu)^2 &= \sum_{i=1}^{n} y_i^2 - 2\mu \sum_{i=1}^{n} y_i + n\mu^2, \\ (\mu - \theta)^2 &= \mu^2 - 2\theta\mu + \theta^2. \end{aligned}$$

The first line contains $n\mu^2$ because each of the n squared differences contributes one copy of μ^2. The terms $\sum_i y_i^2$ and θ^2 do not depend on μ and can be included in C. Using $\sum_i y_i = n\bar{y}$, the log posterior becomes

$$\log f(\mu \mid \mathbf{y}) = -\frac{1}{2}\left[\left(\frac{n}{\sigma^2} + \frac{1}{\tau^2}\right)\mu^2 - 2\left(\frac{n\bar{y}}{\sigma^2} + \frac{\theta}{\tau^2}\right)\mu\right] + C.$$

D.4.2 Completing the square

To make the next algebraic step easier to follow, name the two coefficients

$$A = \frac{n}{\sigma^2} + \frac{1}{\tau^2}, \qquad h = \frac{n\bar{y}}{\sigma^2} + \frac{\theta}{\tau^2}.$$

The expression in brackets is $A\mu^2 - 2h\mu$. Factor out A and add and subtract $(h/A)^2$ inside the brackets:

$$\begin{aligned} A\mu^2 - 2h\mu &= A\left[\mu^2 - 2\frac{h}{A}\mu\right] \\ &= A\left[\mu^2 - 2\frac{h}{A}\mu + \left(\frac{h}{A}\right)^2 - \left(\frac{h}{A}\right)^2\right] \\ &= A\left(\mu - \frac{h}{A}\right)^2 - \frac{h^2}{A}. \end{aligned}$$

Adding and subtracting the same quantity leaves the expression unchanged, while the first three terms inside the brackets form a square. Since h^2/A does not depend on μ, it joins the constant in the log posterior. We are left with

$$\log f(\mu \mid \mathbf{y}) = -\frac{A}{2}\left(\mu - \frac{h}{A}\right)^2 + C.$$

A Normal log density with mean m and variance v has the form $-(\mu - m)^2/(2v) + C$. Comparing the two expressions shows that $m = h/A$ and $v = 1/A$. The parameter μ can take any real value, as required for a Normal distribution. Thus

$$\begin{aligned} \mu \mid \mathbf{y} &\sim \mathrm{N}(m, v), \\ m = \frac{\theta/\tau^2 + n\bar{y}/\sigma^2}{1/\tau^2 + n/\sigma^2}, &\qquad v = \frac{1}{1/\tau^2 + n/\sigma^2}. \end{aligned}$$

The normalized posterior density is therefore

$$f(\mu \mid \mathbf{y}) = \frac{1}{\sqrt{2\pi v}} \exp\left[-\frac{(\mu - m)^2}{2v}\right].$$

D.4.3 Interpreting the update

The reciprocal of a variance is called **precision**. A small variance corresponds to high precision. The prior precision is $1/\tau^2$. The sample mean has sampling variance σ^2/n, so its precision is n/σ^2. The posterior precision adds these contributions:

$$\frac{1}{v} = \frac{1}{\tau^2} + \frac{n}{\sigma^2}.$$

The posterior mean is a weighted average of θ and $\bar{y}$, with each weight proportional to its precision. A tightly concentrated prior gives θ more weight. More observations, or less sampling variability, give $\bar{y}$ more weight.

For the well-being example in Exercise 2.4, $n = 25$, $\bar{y} = 5.735$, and $\sigma = 0.5$. Taking the prior $\mu \sim \mathrm{N}(6.5, 1^2)$ gives prior precision 1 and sample-mean precision $25/0.5^2 = 100$. Hence

$$m = \frac{1 \times 6.5 + 100 \times 5.735}{101} = \frac{580}{101} \approx 5.743, \qquad v = \frac{1}{101} \approx 0.00990.$$

The posterior mean lies close to the observed mean because the sample mean has much greater precision than the prior. The posterior standard deviation is $\sqrt{v} \approx 0.0995$; this is the value to pass to `dnorm()` as `sd`, whereas the Normal notation in the table uses the variance v.

This derivation treats σ^2 as known throughout. If the sampling variance is also unknown, it needs its own prior, and this one-parameter update no longer describes the full posterior.